

TUGAS AKHIR - SF234801

**OPTIMASI PORTOFOLIO BERBASIS MODEL
MARKOWITZ MENGGUNAKAN METODE
*VARIATIONAL QUANTUM EIGENSOLVER***

PUTRA NAUFAL TABASSAM

NRP 5001221120

Dosen Pembimbing

Dr. Heru Sukamto, M.Si.

NIP 198411092012121001

Dr. Gagus K. Sunnardianto

NIP 198611052019021001

Program Studi Sarjana Fisika

Departemen Fisika

Fakultas Sains dan Analitika Data

Institut Teknologi Sepuluh Nopember

Surabaya

2026

Halaman ini sengaja dikosongkan



TUGAS AKHIR - SF234801

**OPTIMASI PORTOFOLIO BERBASIS MODEL
MARKOWITZ MENGGUNAKAN METODE
*VARIATIONAL QUANTUM EIGENSOLVER***

PUTRA NAUFAL TABASSAM

NRP 5001221120

Dosen Pembimbing

Dr. Heru Sukanto, M.Si.

NIP 198411092012121001

Dr. Gagus K. Sunnardianto

NIP 198611052019021001

Program Studi Sarjana Fisika

Departemen FISIKA

Fakultas Sains dan Analitika Data

Institut Teknologi Sepuluh Nopember

Surabaya

2026

Halaman ini sengaja dikosongkan

diasosiasikan dengan likuiditas yang tinggi dan stabilitas yang lebih baik, sehingga menjadi pilihan utama bagi para investor tingkat institusi. Kapitalisasi pasar juga dapat berfungsi sebagai indikator kualitatif yang esensial untuk mengukur tingkat kepercayaan publik terhadap profil risiko suatu entitas finansial (Schwert, 2003). Selain itu, terdapat metrik kinerja laporan keuangan yang memberikan rincian performa aset dan liabilitas yang sangat mendetail, di mana Gambar 2.1 mengilustrasikan ringkasan posisi keuangan suatu perusahaan sebagai landasan utama dalam menentukan derajat kesehatan fiskal mereka (Abdul Hali & Yuliati, 2020). Terdapat pula variabel modern lainnya seperti ana-

Ikhtisar Keuangan dan Rasio Keuangan

Tabel Ikhtisar Keuangan dan Rasio Keuangan

(dalam jutaan Rupiah)

Uralan	2025	2024*	2023	2022	2021
LAPORAN POSISI KEUANGAN KONSOLIDASIAN					
ASET					
Kas	32.044.482	29.783.642	31.603.784	27.407.478	26.299.973
Giro pada Bank Indonesia	31.929.608	88.878.969	101.909.121	150.935.150	56.426.573
Giro dan Penempatan pada bank lain - Netto	63.488.113	83.448.015	87.545.335	91.869.777	73.012.684
Efek-efek, Wesel Ekspor, Reverse Repo dan Tagihan Lainnya	420.454.320	382.903.830	416.411.206	418.685.107	455.174.902
Kredit yang Diberikan, Piutang Syariah, dan Pembiayaan	1.521.485.857	1.354.640.779	1.266.429.247	1.139.077.065	1.042.867.453
CKPN Kredit yang Diberikan, Piutang Syariah, dan Pembiayaan	(83.058.656)	(81.063.511)	(85.501.888)	(93.087.981)	(87.829.417)
Tagihan Derivatif - Netto	1.167.029	1.087.048	911.683	911.405	730.083
Tagihan Akseptasi - Netto	13.046.341	9.783.690	9.967.710	7.031.064	9.066.005
Penyertaan Saham - Netto	8.834.868	8.076.567	7.305.491	6.506.903	6.071.727
Aset Tetap - Netto	63.294.240	62.477.965	59.678.119	55.216.047	47.970.187
Aset Pajak Tangguhan - neto	8.129.522	12.800.660	15.445.977	18.712.994	16.284.898
Aset Lain-lain - neto	54.555.381	39.369.252	53.709.169	42.374.001	32.022.666
TOTAL ASET	2.135.371.105	1.992.186.906	1.965.414.954	1.865.639.010	1.678.097.734

Gambar 2.1: Contoh ikhtisar laporan posisi keuangan konsolidasian yang merepresentasikan struktur aset dan kesehatan fiskal entitas.

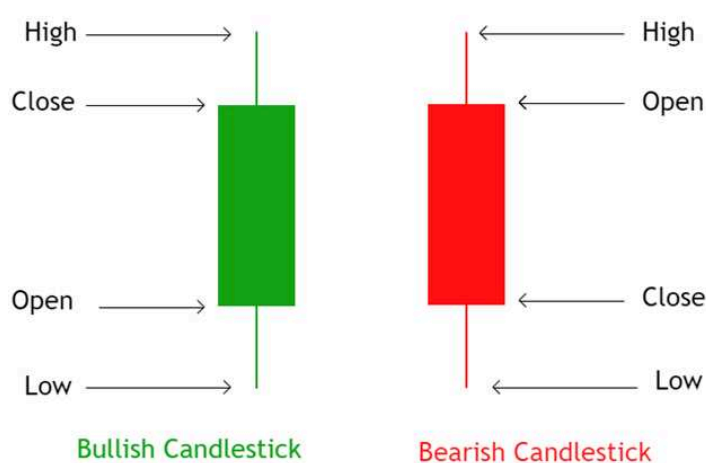
lisis sentimen berita dan data jaringan sosial yang dapat memberikan gambaran mengenai potensi saturasi nilai aset di pasar. Semakin banyak agen yang memiliki sentimen negatif terhadap suatu aset, maka semakin besar potensi penurunan harga aset tersebut di masa depan. Sebaliknya, semakin banyak agen yang memiliki sentimen positif terhadap suatu aset, maka semakin besar potensi kenaikan harga aset tersebut di masa depan.

2.2.2 Representasi Data pada Pergerakan Harga

Di samping analisis fundamental, terdapat analisis teknikal yang berfokus pada pemetaan stokastik aktivitas transaksi ke dalam domain visual untuk memberikan identifikasi pola-pola harga yang sering kali berulang dalam periode waktu tertentu. Pada umumnya, analisis teknikal digunakan oleh para investor untuk membuat keputusan investasi jangka pendek hingga menengah dengan target waktu maksimal 1 tahun. Terdapat berbagai jenis representasi grafis mulai dari *line chart*, *bar chart*, hingga *candlestick* yang digunakan sebagai instrumen untuk mengenali derau harga (*price noise*) dalam bentuk

visual bagi para analis pasar profesional. Prinsip utama dari analisis ini didasarkan pada asumsi bahwa harga pasar saat ini telah mencerminkan seluruh informasi historis dan aksi kolektif para pelaku pasar secara efisien (Schwert, 2003). Visualisasi data tersebut dapat membantu investor dalam melakukan ekstrapolasi tren masa depan berdasarkan pengenalan pola-pola geometris yang telah terbukti secara statistik dalam riwayat perdagangan (Sikalo dkk., 2022).

Representasi *candlestick* yang ditunjukkan pada Gambar 2.2 memberikan visualisasi pergerakan harga dalam interval waktu tertentu yang mencakup nilai *open*, *high*, *low*, dan *close*. Analisis data *candlestick* sering digunakan oleh investor untuk mengidentifikasi tekanan beli maupun jual yang signifikan dalam menentukan potensi pembalikan atau kelanjutan suatu tren harga aset. Saat *candlestick* dideretkan menjadi sebuah deret

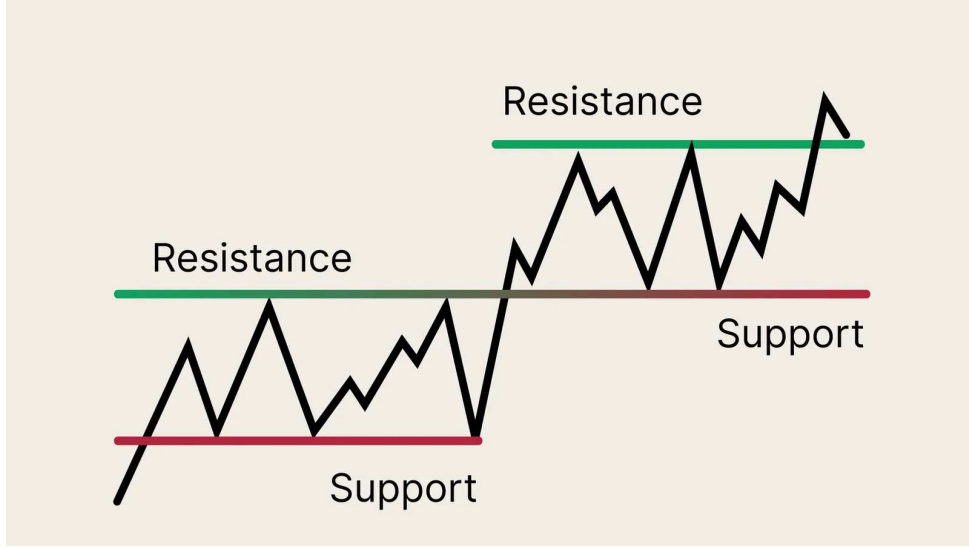


Gambar 2.2: Struktur anatomi *bullish* dan *bearish candlestick* yang merangkum fluktuasi harga dalam satu periode waktu.

waktu, akan terbentuk pola-pola yang merepresentasikan psikologi kolektif pelaku pasar. Titik-titik balik penting dalam grafik di mana pergerakan harga tertahan atau berbalik arah didefinisikan sebagai *Support* dan *Resistance*. *Support* merupakan tingkat harga di mana tekanan beli cukup kuat untuk mengatasi tekanan jual sehingga mencegah harga jatuh lebih jauh, sementara *Resistance* merujuk pada tingkat di mana tekanan jual mendominasi sehingga mencegah harga naik lebih tinggi. Kedua konsep tersebut mencerminkan keseimbangan kekuatan dinamis antara penawaran dan permintaan dalam pasar, sehingga berfungsi sebagai indikator krusial bagi investor dalam pengambilan keputusan teknis. Analogi ilustrasi interaksi pasar tersebut dapat dilihat pada Gambar 2.3, di mana harga sering kali mengalami pantulan pada level *support* sebelum akhirnya berhasil menembus level *resistance* untuk selanjutnya membentuk tren kenaikan yang baru di pasar. Identifikasi pola teknikal tersebut berfungsi sebagai instrumen untuk mendeteksi fenomena perilaku *herding* (*herding behavior*) yang menjadi pemicu utama munculnya volatilitas ekstrem dan perubahan rezim dalam sistem pasar finansial (Hidalgo, 2006).

2.2.3 Analisis Kuantitatif

Analisis kuantitatif merupakan metode yang sering digunakan oleh para analis kuantitatif atau biasa disebut *quants* untuk menentukan alokasi portofolio optimal selain analisis



Gambar 2.3: Representasi visual level *support* dan *resistance* sebagai batas memori sosial dalam dinamika harga aset.

fundamental dan teknikal. Analisis kuantitatif dimulai dari proses transformasi data harga mentah menjadi variabel imbal hasil sederhana (*simple return*) untuk mengurangi efek non-stasioneritas yang sering kali menghambat analisis statistik. Penggunaan imbal hasil logaritmik (*log return*) lebih diutamakan oleh para peneliti karena memungkinkan agregasi nilai secara matematis serta memberikan kemudahan dalam analisis numerik yang kompleks (Saslow, 1999). Imbal hasil sederhana aset ke- i pada hari ke- t ($R_{i,t}$) dihitung sebagai rasio perubahan harga terhadap hargal awal

$$R_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}}. \quad (2.1)$$

Sedangkan imbal hasil logaritmik digunakan untuk menjamin properti stasioneritas pada runtun waktu yang didefinisikan sebagai fungsi logaritma natural dari rasio harga dengan bentuk

$$r_{i,t} = \ln(1 + R_{i,t}) = \ln\left(\frac{P_{i,t}}{P_{i,t-1}}\right). \quad (2.2)$$

Dari kedua data tersebut, dapat diperoleh proses data selanjutnya yakni ekspektasi imbal hasil (μ_i) dan varians (σ_i^2) untuk setiap aset. Ekspektasi imbal hasil diperoleh melalui rata-rata data satu semester perdagangan yang mana memiliki 126 hari perdagangan ($n = 126$) sebagai data pelatihan untuk algoritma dengan rumus

$$\mu_i = \bar{R}_i = \frac{1}{n} \sum_{t=1}^n R_{i,t}. \quad (2.3)$$

Jika menggunakan imbal hasil logaritmik pada aset ke- i , rumusnya menjadi

$$\mu_{i,l} = \bar{r}_i = \frac{1}{n} \sum_{t=1}^n r_{i,t}. \quad (2.4)$$

Sedangkan untuk varians yang merepresentasikan risiko karena semakin besar varians maka semakin besar pula potensi kerugian yang dapat terjadi, diperoleh dengan bentuk

$$\sigma_i^2 = \frac{1}{n-1} \sum_{t=1}^n (R_{i,t} - \bar{R}_i)^2. \quad (2.5)$$

Jika menggunakan imbal hasil logaritmik, rumus variansnya menjadi

$$\sigma_{i,l}^2 = \frac{1}{n-1} \sum_{t=1}^n (r_{i,t} - \bar{r}_i)^2. \quad (2.6)$$

Selain itu, kovariansi yang digunakan untuk mengukur tingkat keterkaitan antara dua aset saham memiliki bentuk

$$\sigma_{ij} = \frac{1}{n-1} \sum_{t=1}^n (R_{i,t} - \bar{R}_i)(R_{j,t} - \bar{R}_j). \quad (2.7)$$

Jika menggunakan imbal hasil logaritmik, rumus kovariansinya menjadi

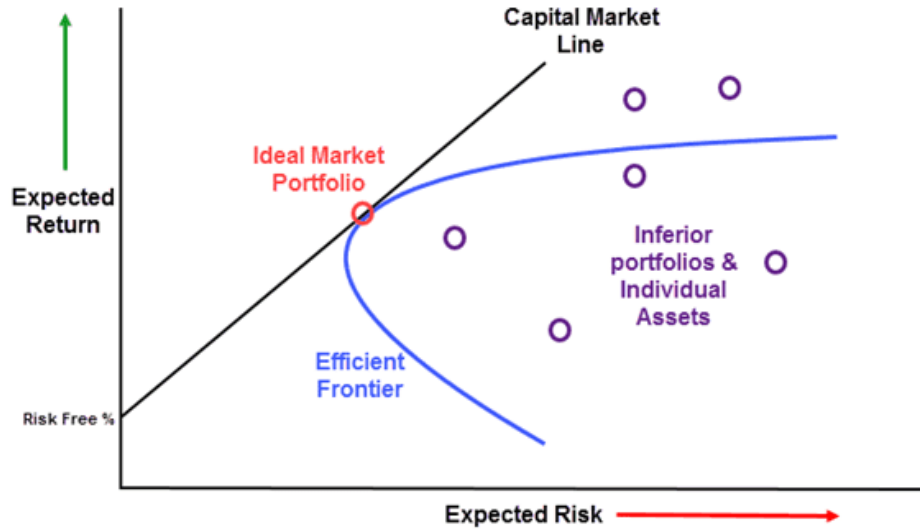
$$\sigma_{ij,l} = \frac{1}{n-1} \sum_{t=1}^n (r_{i,t} - \bar{r}_i)(r_{j,t} - \bar{r}_j). \quad (2.8)$$

Kovariansi positif menunjukkan bahwa kedua aset bergerak searah, sedangkan kovariansi negatif menunjukkan bahwa kedua aset bergerak berlawanan arah. Sebelum masuk ke dalam formulasi optimasi portofolio, perlu dipahami konsep efisiensi alokasi sumber daya melalui kerangka *Pareto Optimum*. Sebuah alokasi portofolio dikatakan optimal secara Pareto jika tidak ada cara untuk meningkatkan ekspektasi imbal hasil tanpa meningkatkan risiko, atau sebaliknya, tidak ada cara untuk menurunkan risiko tanpa menurunkan imbal hasil yang diharapkan.

Dalam ekonomi kesejahteraan, *Pareto Optimum* merepresentasikan titik-titik di mana utilitas sistem tidak dapat ditingkatkan lagi melalui realokasi aset sederhana tanpa merugikan salah satu parameter objektif (Ahmed dkk., 2017). Konsep ini mendasari pembentukan *Efficient Frontier* dalam teori portofolio, yang merupakan kumpulan dari semua portofolio yang menawarkan imbal hasil tertinggi untuk tingkat risiko tertentu (Cohen dkk., 2020). Bagi investor rasional, pemilihan alokasi modal harus selalu berada pada kurva efisiensi ini guna memastikan penggunaan sumber daya finansial yang maksimal di tengah ketidakpastian pasar (Innan dkk., 2025).

2.2.4 Model Markowitz

Model *mean-variance* yang diperkenalkan oleh Harry Markowitz pada tahun 1952 mentransformasikan konsep abstrak *Pareto Optimum* untuk mencari titik keseimbangan antara ekspektasi imbal hasil dan tingkat risiko (Markowitz, 1952). Titik-titik keseimbangan optimal tersebut membentuk sebuah kurva yang dikenal sebagai kurva *Efficient Frontier* sebagaimana divisualisasikan pada Gambar 2.4. Titik-titik yang terletak pada kurva tersebut merepresentasikan konfigurasi portofolio yang paling efisien, di mana setiap pergeseran di sepanjang kurva mencerminkan pertukaran (*trade-off*) antara risiko dan imbal hasil. Jika ingin mendapatkan imbal hasil tinggi, maka investor harus siap



Gambar 2.4: Kurva *Efficient Frontier* sebagai representasi batas efisiensi alokasi aset dalam kerangka kerja model Markowitz.

dengan risiko yang tinggi juga. Namun jika investor menginginkan risiko yang rendah, maka investor harus memiliki aset dengan potensi imbal hasil yang rendah pula. Oleh karena itu, pemilihan titik spesifik pada kurva ini sangat bergantung pada tingkat profil toleransi risiko dari masing-masing investor untuk mencapai utilitas maksimal.

Sebagai bentuk formulasi matematis dari gambar kurva *Efficient Frontier* tersebut, analisis kuantitatif pada model ini bertujuan untuk menentukan bobot alokasi modal yang tepat pada setiap instrumen untuk memaksimalkan utilitas portofolio (Abdul Hali & Yuliati, 2020). Fungsi biaya dalam model Markowitz menggabungkan matriks kovarians yang merepresentasikan risiko dan vektor ekspektasi imbal hasil dari semua aset yang ingin dipilih (Schwert, 2003). Fungsi objektif model Markowitz dapat dinyatakan dalam bentuk fungsi biaya (cost function) sebagai

$$f(\omega) = \sum_{i,j} \sigma_{ij} \omega_i \omega_j - \lambda \sum_i \mu_i \omega_i \quad (2.9)$$

dengan ω_i merepresentasikan bobot untuk setiap aset i , σ_{ij} merupakan elemen matriks kovarians, dan μ_i menunjukkan ekspektasi imbal hasil aset. Parameter λ berfungsi sebagai tingkat toleransi risiko yang mana secara matematis menentukan gradien utilitas investor pada kurva efisiensi tersebut. Untuk mempermudah optimasi model, parameter ω_i yang memiliki nilai kontinu antara 0 hingga 1 dan memiliki syarat $\sum \omega_i = 1$, dapat diubah menjadi variabel biner x_i yang hanya bernilai 0 dan 1 untuk menyatakan aset ke- i dibeli jika bernilai 1 dan tidak dibeli jika bernilai 0. Hubungan perubahan variabel tersebut dinyatakan dengan bentuk

$$\omega_i = \frac{x_i}{K} \quad (2.10)$$

dimana K merepresentasikan jumlah aset yang dipilih dari total pilihan aset yang ada (N). Jika jumlah aset yang dipilih adalah setengah dari pilihan aset maka $K = N/2$.

Sehingga, fungsi objektif dapat diubah menjadi

$$f(\mathbf{x}) = \sum_{i,j} \sigma_{ij} \frac{x_i x_j}{K^2} - \lambda \sum_i \mu_i \frac{x_i}{K}. \quad (2.11)$$

2.3 Teori Permainan dan Permainan Potensial Eksak (*Exact Potential Game*)

Pada dasarnya, manusia tidak bisa hidup dan bekerja sendirian karena manusia sangat membutuhkan orang lain mengingat sifatnya sebagai makhluk sosial yang memiliki ketergantungan interaktif satu sama lain. Namun faktanya, interaksi sosial yang dilakukan oleh manusia tidak selalu bersifat harmonis dan saling menguntungkan, melainkan sering kali terjebak dalam dilema kepentingan yang memicu persaingan strategis. Ketidakpastian hasil dari interaksi ini membutuhkan formalisme ke dalam suatu metode analisis yang mampu memetakan perilaku agen rasional secara matematis dalam menghadapi skenario pilihan untuk mengoptimalkan utilitas kolektif. Oleh karena itu, dalam buku "*Theory of Games and Economic Behavior*", matematikawan John Von Neumann dan Oskar Morgenstern merumuskan formalisme matematika aksiomatik yang mengurai logika strategi di bawah kondisi ketergantungan secara sistematis (John Von Neumann & Oskar Morgenstern, 1953). Paradigma ini kemudian berkembang dan memberikan landasan bagi pemodelan perilaku agen hingga tercetuslah konsep *game theory* atau teori permainan yang menjadi pilar utama dalam analisis interaksi strategis (Galam & Walliser, 2010).

2.3.1 Teori Permainan

Teori permainan merupakan kerangka kerja matematis aksiomatik yang digunakan untuk menganalisis interaksi strategis yang melibatkan sekumpulan agen rasional dalam lingkungan yang kompetitif. Sebuah permainan didefinisikan oleh tiga komponen fundamental yang meliputi himpunan pemain (N), ruang strategi (S), dan fungsi utilitas (U) bagi setiap partisipan yang terlibat. Interaksi antar pemain terjadi ketika keputusan yang diambil oleh satu agen mempengaruhi nilai utilitas atau keuntungan yang diperoleh oleh agen lainnya secara langsung, sehingga menciptakan ketergantungan yang beragam. Terdapat berbagai fenomena interaksi strategis dalam ekosistem ekonomi yang dapat direpresentasikan secara akurat melalui model permainan klasik (Monderer & Shapley, 1996). Model permainan klasik tersebut seperti *Prisoner's Dilemma*, *Stag Hunt*, dan *Battle of the Sexes*, menangkap esensi konflik kepentingan antar agen. Meskipun memiliki narasi konflik yang bervariasi, model-model tersebut secara formal dapat diklasifikasikan sebagai *Exact Potential Game* karena memenuhi syarat penyelarasan antara utilitas marginal pemain dan perubahan fungsi potensial global yang akan dibahas di kedua subbab selanjutnya (Monderer & Shapley, 1996). Dalam kerangka *Prisoner's Dilemma*, pendekatan EPG memungkinkan identifikasi ekuilibrium yang stabil meskipun sering kali bersifat sub-optimal, sementara dalam *Stag Hunt*, metode ini mampu memfasilitasi penemuan titik koordinasi yang optimal bagi seluruh agen (Sarkar & Benjamin, 2018).

Ketiga model permainan tersebut memberikan kerangka umum terhadap dinamika ekuilibrium dalam sistem multi-agen dengan ketergantungan utilitas yang bersifat non-linear (Monderer & Shapley, 1996). Gambar 2.5(a) mengilustrasikan permainan *Stag*

Hunt di mana strategi *Stag* merepresentasikan koordinasi berisiko tinggi dengan imbal hasil maksimal, sedangkan strategi *Hare* merupakan pilihan individualistik yang aman namun menghasilkan utilitas rendah bagi sistem (Sarkar & Benjamin, 2018). Dalam model *Prisoner's Dilemma* yang ditunjukkan pada Gambar 2.5(b), konflik muncul antara pilihan *Cooperation* untuk mencapai keuntungan kolektif optimal berhadapan dengan strategi *Non-cooperation* yang secara individual dominan namun berujung pada ekuilibrium yang bersifat *Pareto-inferior*. Sementara itu, Gambar 2.5(c) merepresentasikan *Battle of the Sexes* yang mensyaratkan agen untuk melakukan koordinasi antara opsi *Dog race (D)* dan *Ballet (B)* meskipun terdapat preferensi imbal hasil yang berbeda di antara para pemain yang terlibat.

				Hunter 2	
				Stag	Hare
Hunter 1	Stag	3, 3	0, 2		
	Hare	2, 0	2, 2		

(a) Stag Hunt

				Player B	
				Cooperation	Non-cooperation
Player A	Cooperation	3, 3	0, 5		
	Non-cooperation	5, 0	1, 1		

(b) Prisoner's Dilemma

				Woman	
				D	B
Man	D	(4, 3)	(2, 2)		
	B	(1, 1)	(3, 4)		

(c) Battle of the Sexes

Gambar 2.5: Matriks imbal hasil pada model permainan klasik: (a) *Stag Hunt*, (b) *Prisoner's Dilemma*, dan (c) *Battle of the Sexes* yang merepresentasikan variasi struktur ekuilibrium (Szabó & Borsos, 2016).

2.3.2 *Exact Potential Game (EPG)* sebagai Algoritma Pencarian Ekuilibrium Nash

Ekuilibrium Nash atau *nash equilibrium* merupakan konsep stabilitas fundamental dalam teori permainan di mana tidak ada satu pun pemain yang dapat meningkatkan utilitasnya dengan mengubah strateginya secara sepihak tanpa adanya perubahan dari pihak lain. Dalam sistem finansial, sebagian besar interaksi strategis antar agen ekonomi direpresentasikan melalui model permainan koordinasi (*coordination game*) yang mana setara dengan struktur *Stag Hunt* (Szabó & Borsos, 2016). Pemilihan model ini didasarkan pada fakta bahwa pasar sangat bergantung pada sinkronisasi keputusan para investor. Kerja sama kolektif untuk mempertahankan aset akan memberikan imbal hasil lebih tinggi dibandingkan pilihan individualis untuk keluar dari pasar (Sarkar & Benjamin, 2018). Namun, identifikasi titik ekuilibrium tersebut sering kali menghadapi kendala komputasi yang berat karena ruang solusi yang bersifat non-konveks serta diskrit, terutama pada sistem yang memiliki jumlah agen sangat besar.

Exact Potential Game (EPG) hadir untuk mengatasi masalah tersebut (Ahmadi dkk., 2023). EPG didefinisikan sebagai kelas permainan strategis khusus di mana perubahan utilitas marginal setiap pemain selaras dengan variasi sebuah fungsi potensial global (Monderer & Shapley, 1996). Secara matematis, eksistensi fungsi potensial (P) menunjukkan bahwa setiap insentif agen untuk meningkatkan keuntungan lokal akan berujung

pada peningkatan nilai potensial global (Y. Yang dkk., 2012). Karakteristik penyelarasan tersebut dapat menyederhanakan masalah optimasi multi-agen yang sangat kompleks menjadi masalah maksimisasi fungsi tunggal yang jauh lebih efisien untuk dikelola secara komputasional. Dalam sistem pasar finansial, EPG berfungsi sebagai algoritma pencarian ekuilibrium Nash yang andal melalui pemanfaatan mekanisme dinamika respons-terbaik (*Best-Response Dynamics*) secara iteratif (Y. Yang dkk., 2012). Sebagai contoh, implementasi *Best-Response Dynamics* oleh (Y. Yang dkk., 2012) memiliki prosedur lewat perumusan sebuah sistem permainan dalam bentuk algoritma yang memanfaatkan variabel ω_n^0 sebagai representasi bobot portofolio awal bagi setiap agen n di dalam himpunan strategi K_n . Indeks q digunakan untuk melacak kemajuan tahapan iteratif, di mana profil strategi kolektif ω^q dievaluasi terhadap fungsi utilitas u_n untuk mendapatkan konvergensi sistem menuju titik stabil. Mekanisme pembaruan sekuensial (Gauss-Seidel) beroperasi dengan mengintegrasikan informasi strategis terbaru dari agen sebelumnya, sementara pembaruan simultan (Jacobi) menggunakan data dari iterasi sebelumnya untuk menghitung respon terbaik secara paralel bagi seluruh partisipan pasar (Freitas dkk., 2024). Struktur operasional algoritma iteratif tersebut dapat ditinjau pada algoritma 1.

Algorithm 1 Iterative Best Response Algorithm (Y. Yang dkk., 2012)

- 1: **Data:** Pilih strategi awal $\omega_n^0 \in K_n$ untuk semua $n = 1, \dots, N$, dan set $q = 0$.
 - 2: **Step 1:** Jika ω^q memenuhi kriteria penghentian yang sesuai: **STOP**.
 - 3: **Step 2:** Perbarui strategi ω_n^{q+1} untuk $n = 1, \dots, N$ menggunakan salah satu metode:
 - 4: **Sequential (Gauss-Seidel) Update:**
 - 5: $\omega_n^{q+1} \leftarrow \arg \min_{\omega_n \in K_n} u_n(\omega_1^{q+1}, \dots, \omega_{n-1}^{q+1}, \omega_n, \omega_{n+1}^q, \dots, \omega_N^q)$
 - 6: **Simultaneous (Jacobi) Update:**
 - 7: $\omega_n^{q+1} \leftarrow (1 - \frac{1}{N})\omega_n^q + \frac{1}{N} \cdot \arg \min_{\omega_n \in K_n} u_n(\omega_1^q, \dots, \omega_{n-1}^q, \omega_n, \omega_{n+1}^q, \dots, \omega_N^q)$
 - 8: **Step 3:** Set $q \leftarrow q + 1$; dan kembali ke **Step 1** (Monderer & Shapley, 1996).
-

2.3.3 Kerangka EPG dari Model Markowitz

Implementasi kerangka kerja *Exact Potential Game* dalam pemilihan aset strategis diawali dengan transformasi fungsi objektif dari kontinu menjadi diskrit versi biner yang telah dijabarkan sebelumnya pada Persamaan (2.11). Formulasi fungsi objektif tersebut selanjutnya dikonversi menjadi representasi fungsi utilitas (U) yang harus dimaksimalkan oleh setiap agen di dalam sistem. Transformasi ini dilakukan dengan mendefinisikan sebuah faktor baru yaitu faktor penghindaran risiko (γ) yang memiliki hubungan $\gamma = 2/\lambda$, sehingga profil nilai utilitas portofolio sistem dapat dinyatakan dengan hubungan

$$U(\mathbf{x}) = -\frac{1}{\lambda}f(\mathbf{x})$$

sehingga menjadi

$$U(\mathbf{x}) = \sum_i \frac{\mu_i}{K} x_i - \frac{\gamma}{2K^2} \sum_{i,j} \sigma_{ij} x_i x_j. \quad (2.12)$$

dengan utilitas individu yang dimiliki oleh agen ke- i didefinisikan sebagai

$$u_i(\mathbf{x}) = \frac{\mu_i}{K} x_i - \frac{\gamma}{2K^2} x_i \left(\sum_{j \neq i} \sigma_{ij} x_j \right) \quad (2.13)$$

Secara matematis, Lenz dan Ising memodelkan material feromagnetik sebagai sekumpulan variabel *spin* biner yang hanya dapat berada pada dua orientasi, yaitu sejajar (+1) atau berlawanan (-1) terhadap suatu arah acuan. Model ini berangkat dari energi potensial sebuah dipol magnetik yang berada di dalam medan magnet eksternal, yaitu

$$E_i = -\boldsymbol{\mu}_i \cdot \mathbf{B}, \quad (2.19)$$

dengan $\boldsymbol{\mu}_i$ menyatakan momen magnetik partikel ke- i dan $\mathbf{B}$ merupakan medan magnet luar. Sistem akan mencapai keadaan energi minimum ketika arah momen magnetik sejajar dengan medan magnet yang diberikan sehingga diberikan tanda negatif pada persamaan. Momen magnetik suatu partikel memiliki nilai yang sebanding dengan operator spin-nya, sehingga dapat dinyatakan sebagai

$$\boldsymbol{\mu}_i = -\gamma \mathbf{S}_i, \quad (2.20)$$

dengan γ adalah rasio giromagnetik dan $\mathbf{S}_i$ merupakan operator spin. Apabila medan magnet hanya diberikan pada arah sumbu- z , yaitu $\mathbf{B} = (0, 0, B_z)$, maka Persamaan (2.19) menjadi

$$E_i = \gamma B_z S_i^z. \quad (2.21)$$

Karena komponen operator spin sepanjang sumbu- z memenuhi hubungan

$$S_i^z = \frac{\hbar}{2} \sigma_i^z, \quad (2.22)$$

maka energi pada Persamaan (2.21) dapat dituliskan sebagai

$$E_i = \frac{\gamma \hbar B_z}{2} \sigma_i^z. \quad (2.23)$$

Dapat didefinisikan pula parameter medan efektif sebagai

$$h_i = -\frac{\gamma \hbar B_z}{2}, \quad (2.24)$$

sehingga kontribusi energi akibat medan magnet eksternal dapat disederhanakan menjadi

$$E_i = -h_i s_i, \quad (2.25)$$

dengan $s_i = \pm 1$ menyatakan orientasi spin pada lokasi ke- i . Selain dipengaruhi oleh medan luar, model Ising juga mengasumsikan bahwa setiap pasangan spin saling berinteraksi. Energi interaksi antar pasangan spin dinyatakan sebagai

$$E_{ij} = -J_{ij} s_i s_j, \quad (2.26)$$

dengan J_{ij} merupakan konstanta kopling antara spin ke- i dan ke- j . Ketika kedua spin memiliki orientasi yang sama, maka nilai $s_i s_j$ akan bernilai 1 yang mana merepresentasikan konfigurasi sejajar sempurna, sedangkan apabila keduanya berlawanan arah maka $s_i s_j$ akan bernilai -1 yang mana merepresentasikan konfigurasi anti-sejajar sempurna.

Dengan menjumlahkan seluruh kontribusi energi medan luar dan interaksi antar pasangan spin, maka diperoleh Hamiltonian Ising pada sistem klasik berbentuk

$$\mathcal{H} = -\sum_{i=1}^N h_i s_i - \sum_{i<j}^N J_{ij} s_i s_j. \quad (2.27)$$

sudah terkonstruksi. Selain itu, komputer kuantum dapat memproses dan memanipulasi informasi dalam ruang Hilbert yang sangat luas menggunakan rangkaian kuantum khusus lewat gerbang kuantum yang sangat berbeda dari gerbang klasik (Rajak dkk., 2023). Komputer kuantum memanfaatkan prinsip superposisi dan keterbelitan kuantum untuk menavigasi ruang solusi dari masalah optimasi kombinatorial kompleks yang mana sangat sulit dijangkau oleh sistem komputasi klasik (Preskill, 2018).

Bab ini akan menguraikan bagaimana sistem qubit dapat menjadi fondasi utama teknis komputer kuantum lalu dilanjutkan dengan gerbang kuantum 1 qubit dan 2 qubit. Kemudian dikenalkan pula keterbelitan kuantum (*Quantum entanglement*) sebagai akibat dari penggunaan gerbang kuantum dan pengukuran kuantum (*Quantum Measurement*) sebagai dasar pengukuran sistem kuantum di akhir rangkaian kuantum (Coyle dkk., 2021). Terakhir, dikenalkan pula tentang pembelajaran mesin kuantum (*Quantum Machine Learning*) dan rangkaian kuantum berparameter (*Parameterized Quantum Circuit*) yang menjadi dasar dari algoritma Variational Quantum Eigensolver (VQE) yang akan digunakan pada tugas akhir ini.

2.5.1 Sistem Qubit

Qubit didefinisikan sebagai satuan informasi dasar dalam komputasi kuantum yang memperluas konsep bit klasik menjadi sistem dua-keadaan mekanika kuantum sehingga mampu eksis dalam kondisi superposisi. Berbeda dengan bit klasik yang hanya memiliki dua status 0 dan 1 secara eksklusif, *qubit* didefinisikan secara matematis melalui vektor kolom di ruang vektor kompleks dua-dimensi $\mathbb{C}^2$ (Nielsen & Chuan, 2010). Qubit dapat direpresentasikan dengan basis komputasi ortogonal $|0\rangle$ dan $|1\rangle$ yang dinyatakan melalui vektor kolom ortonormal dengan bentuk matematis

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \quad (2.42)$$

Keberadaan vektor keadaan *qubit* yang bersifat superposisi dinyatakan pada vektor keadaan ($|\psi\rangle$) dari keadaan qubit dengan bentuk

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (2.43)$$

di mana α dan β merupakan amplitudo probabilitas sistem kuantum berada di keadaan $|0\rangle$ dan $|1\rangle$. Karena sistem fisik harus memenuhi syarat kekekalan probabilitas yang menyatakan bahwa total probabilitas dari keadaan $|0\rangle$ dan $|1\rangle$ harus bernilai 1 yang berarti pasti ada, dengan bentuk persamaan

$$|\alpha|^2 + |\beta|^2 = 1 \quad (2.44)$$

dimana kuadrat dari amplitudo probabilitas menyatakan peluang sistem kuantum berada di keadaan $|0\rangle$ dan $|1\rangle$ yang bernilai 0 hingga 1 (Purwanto, 2016). Sifat superposisi inilah yang memungkinkan *qubit* berada dalam kombinasi linier dari kedua basis keadaan dalam satu waktu yang sama, sehingga memberikan potensi komputasi paralel yang jauh melampaui kapabilitas bit klasik.

Karena amplitudo probabilitas α dan β merupakan bilangan kompleks yang memenuhi syarat normalisasi, keadaan umum sebuah *qubit* dapat diparameterkan menggunakan dua parameter sudut, yaitu sudut polar θ dan sudut fase relatif ϕ . Dengan mengabaikan fase

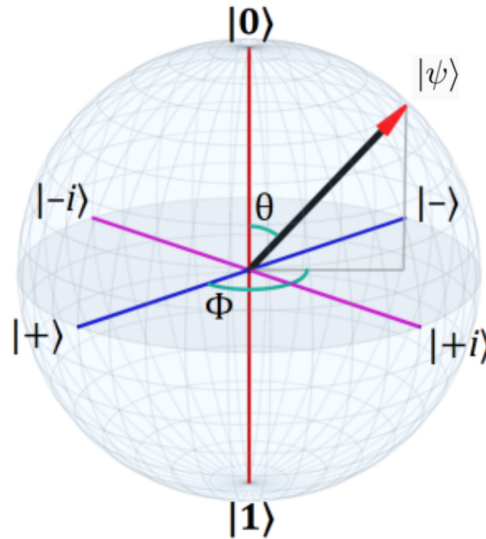
global yang tidak memengaruhi hasil pengukuran, vektor keadaan *qubit* dapat dituliskan dalam bentuk

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle, \quad (2.45)$$

dengan $0 \leq \theta \leq \pi$ dan $0 \leq \phi < 2\pi$. Parameter θ menentukan distribusi amplitudo probabilitas antara keadaan $|0\rangle$ dan $|1\rangle$, sedangkan parameter ϕ menyatakan fase relatif di antara kedua basis tersebut. Keadaan sebuah *qubit* tersebut dapat direpresentasikan menggunakan representasi Bola Bloch yang memvisualisasikan bagaimana vektor keadaan sebuah *qubit* terletak di dalam ruang keadaan 3D (Padilla, 2025). Pada gambar 2.7, vektor keadaan pada sebuah *qubit* direpresentasikan oleh arah garis $|\psi\rangle$ dari tengah menuju permukaan bola (Nakahara, 2008). Vektor tersebut dipengaruhi oleh dua komponen yaitu parameter polar θ yang bergantung pada amplitudo probabilitas dan parameter azimuthal sebagai fase relatif ϕ . Ketika vektor keadaan bergerak dari kutub utara menuju kutub selatan sepanjang sumbu z , probabilitas pengukuran pada basis komputasi berubah secara kontinu dari keadaan $|0\rangle$ menjadi $|1\rangle$. Sementara itu, semua keadaan yang terletak pada ekuator Bola Bloch memiliki probabilitas pengukuran yang sama, yaitu 0,5 untuk $|0\rangle$ dan 0,5 untuk $|1\rangle$, namun dibedakan oleh fase relatif yang dinyatakan oleh sudut ϕ . Jika diformalisasikan secara aljabar, posisi geometris dari vektor keadaan tersebut dapat dipetakan ke dalam koordinat Kartesian tiga dimensi pada ruang bola Bloch (Gong dkk., 2025). Koordinat Kartesian x , y , dan z dari titik ujung vektor keadaan didefinisikan secara unik melalui hubungan sudut polar θ dan sudut azimuthal ϕ dengan hubungan

$$x = \sin\theta \cos\phi, \quad y = \sin\theta \sin\phi, \quad z = \cos\theta. \quad (2.46)$$

Hubungan koordinat spasial tersebut memetakan ruang status Hilbert dua dimensi ke dalam representasi ruang nyata tiga dimensi secara konsisten untuk mempermudah visualisasi fisis.



Gambar 2.7: Representasi Bola Bloch sebagai visualisasi geometris keadaan murni sebuah *qubit*. Posisi vektor keadaan ditentukan oleh sudut polar θ dan sudut azimuthal ϕ , yang dipetakan ke koordinat Kartesian (x, y, z) .

Keempat keadaan tersebut memiliki amplitudo probabilitas yang sama besar pada dua basis komputasi penyusunnya, sehingga pengukuran pada salah satu qubit akan langsung menentukan hasil pengukuran pada qubit lainnya. Sifat inilah yang menjadikan Keadaan Bell sebagai sumber daya fundamental dalam berbagai protokol komputasi dan komunikasi kuantum, seperti teleportasi kuantum, superdense coding, dan distribusi kunci kuantum.

Selain dapat dibangkitkan lewat kombinasi gerbang Hadamard dan CNOT, keterbelitan juga dapat dibangkitkan menggunakan kombinasi gerbang rotasi yang dikombinasikan dengan gerbang CNOT. Hal tersebut dapat terjadi karena gerbang Hadamard memiliki hubungan dengan gerbang rotasi. Secara matematis, gerbang Hadamard ekuivalen dengan kombinasi gerbang rotasi terhadap sumbu-Y dan sumbu-Z. Hubungan tersebut dapat dinyatakan sebagai

$$H = e^{i\pi/2} R_z(\pi) R_y\left(\frac{\pi}{2}\right) \sim R_z(\pi) R_y\left(\frac{\pi}{2}\right) \quad (2.72)$$

Karena fase global tidak memiliki konsekuensi fisik yang dapat diamati, faktor $e^{i\pi/2}$ umumnya dapat diabaikan. Ketika keadaan awal sistem adalah $|0\rangle$, pengaruh $R_z(\pi)$ hanya berupa perubahan fase yang tidak mengubah distribusi probabilitas hasil pengukuran. Oleh karena itu, pembentukan superposisi sering kali cukup dianalisis melalui aksi gerbang rotasi $R_Y(\theta)$. Untuk menunjukkan bahwa gerbang R_Y dapat menghasilkan superposisi yang ekuivalen dengan gerbang Hadamard, aksi gerbang tersebut ditinjau terhadap keadaan dasar $|0\rangle$. Variabel sudut rotasi pada matriks gerbang rotasi terhadap sumbu-Y yang didefinisikan pada persamaan (2.47) disubstitusikan dengan nilai parameter rotasi $\theta = \pi/2$, sehingga diperoleh

$$R_y\left(\frac{\pi}{2}\right) |0\rangle = \begin{pmatrix} \cos\left(\frac{\pi}{2}\right) & -\sin\left(\frac{\pi}{2}\right) \\ \sin\left(\frac{\pi}{2}\right) & \cos\left(\frac{\pi}{2}\right) \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad (2.73)$$

Hasil tersebut menunjukkan bahwa gerbang $R_Y(\pi/2)$ menghasilkan superposisi dengan amplitudo yang sama besar pada basis $|0\rangle$ dan $|1\rangle$. Hal tersebut serupa dengan gerbang Hadamard dalam membangkitkan superposisi awal pada suatu qubit.

Keadaan superposisi tersebut selanjutnya dapat diperluas ke sistem dua qubit dengan menerapkan gerbang $R_Y(\pi/2)$ pada qubit kontrol dan gerbang identitas pada qubit target. Untuk keadaan awal $|00\rangle$, diperoleh

$$\left(R_y\left(\frac{\pi}{2}\right) \otimes I\right) |00\rangle = \frac{|00\rangle + |10\rangle}{\sqrt{2}} \quad (2.74)$$

yang mana identik dengan pembangkitan superposisi melalui penerapan gerbang Hadamard pada qubit pertama. Selanjutnya, dengan menerapkan gerbang CNOT, diperoleh

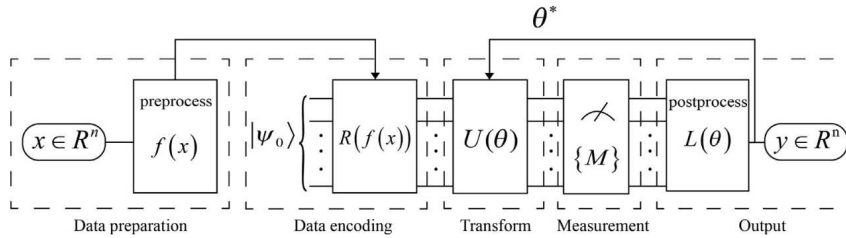
$$\text{CNOT} \left(R_y\left(\frac{\pi}{2}\right) \otimes I\right) |00\rangle = \text{CNOT} \frac{|00\rangle + |10\rangle}{\sqrt{2}} = |\Phi^+\rangle. \quad (2.75)$$

2.5.4 Sirkuit Kuantum Berparameter

Quantum Machine Learning (QML) merupakan bidang multidisipliner yang menggabungkan prinsip komputasi kuantum dengan metode pembelajaran mesin. Salah satu pendekatan yang berkembang dalam QML adalah *Quantum Deep Learning* (QDL), yaitu pemanfaatan model kuantum dengan parameter khusus untuk mempelajari pemodelan

data kompleks. Implementasi model QDL umumnya diwujudkan melalui *Parameterized Quantum Circuit* (PQC), yaitu sirkuit kuantum yang terdiri atas gerbang-gerbang kuantum yang memiliki parameter yang nilainya dapat dioptimasi. PQC menjadi komponen utama dalam keluarga *Variational Quantum Algorithms* (VQA), yaitu algoritma hibrida yang menggabungkan pemrosesan kuantum dan optimasi klasik. Keluarga VQA terdiri dari *Variational Quantum Eigensolver* (VQE), *Quantum Approximate Optimization Algorithm* (QAOA), dan *Variational Quantum Classifier* (VQC) yang mana semuanya memanfaatkan PQC sebagai representasi solusi yang dievaluasi untuk kemudian nilainya diperbarui oleh algoritma optimasi klasik secara iteratif (Chen dkk., 2025).

Sirkuit Kuantum Berparameter (*Parameterized Quantum Circuit* atau PQC) adalah rangkaian gerbang kuantum yang memiliki parameter kontinu, umumnya berupa sudut rotasi. Dalam algoritma variasional, rangkaian PQC berperan sebagai ansatz yang mengeksplorasi parameter pada ruang solusi untuk fungsi gelombang dari suatu permasalahan. Pada umumnya, alur kerja PQC ditunjukkan pada Gambar 2.11. Secara singkat, proses diawali dengan penyandian data klasik $x \in \mathbb{R}^n$ ke dalam keadaan kuantum $|\psi_0\rangle$ melalui fungsi prapemrosesan $f(x)$ dan operasi penyandian $R(f(x))$. Keadaan kuantum yang telah terbentuk kemudian diproses oleh sirkuit variasional $U(\theta)$, yang bertugas mentransformasikan keadaan kuantum melalui serangkaian operasi kuantum berparameter. Parameter θ pada sirkuit ini akan diperbarui secara iteratif selama proses optimasi untuk menghasilkan suatu keadaan kuantum $|\psi(\theta)\rangle$ yang merepresentasikan solusi optimum. Selanjutnya, keadaan akhir diproyeksikan melalui pengukuran $\{M\}$ untuk memperoleh nilai ekspektasi atau probabilitas observabel yang relevan. Hasil pengukuran tersebut kemudian dievaluasi menggunakan fungsi biaya $L(\theta)$, yang digunakan oleh algoritma optimasi klasik untuk menentukan pembaruan parameter hingga diperoleh parameter optimal θ dan keluaran model $y \in \mathbb{R}^n$.



Gambar 2.11: Representasi alur kerja sirkuit kuantum berparameter (PQC) yang mengilustrasikan tahap penyandian data, sirkuit variasional, pengukuran, dan pembaruan parameter oleh algoritma optimasi klasik.

Setelah data dikodekan ke dalam keadaan kuantum $|\psi(\theta)\rangle$, sirkuit variasional $U(\theta)$ diterapkan untuk menghasilkan keadaan kuantum yang bergantung pada parameter-parameter optimasi. Berdasarkan sifat evolusi uniter dalam mekanika kuantum, keadaan keluaran sirkuit dapat dituliskan sebagai

$$|\psi(\theta)\rangle = U(\theta)|\psi_0\rangle. \quad (2.76)$$

di mana operator variasional $U(\theta)$ dibangun dari kombinasi gerbang rotasi satu-qubit dan gerbang keterbelitan dua-qubit. Secara umum, operator tersebut dapat dinyatakan

$$U(\theta) = \prod_{l=1}^L U_{(ent)}^{(l)} U_{(rot)}^{(l)}(\theta^{(l)}) \quad (2.77)$$

dengan $U_{(rot)}^{(l)}$ merepresentasikan lapisan gerbang rotasi berparameter, seperti $Rx(\theta)$, $Ry(\theta)$, dan $Rz(\theta)$, sedangkan $U_{(ent)}^{(l)}$ menyatakan lapisan gerbang keterbelitan, misalnya gerbang CNOT. Struktur berlapis ini memungkinkan sirkuit menghasilkan keadaan kuantum yang semakin kompleks seiring bertambahnya jumlah parameter dan lapisan yang digunakan. Variasi nilai parameter θ akan menghasilkan fungsi gelombang yang berbeda sehingga memungkinkan eksplorasi berbagai kandidat solusi di dalam ruang pencarian.

Kualitas suatu kandidat solusi kemudian dievaluasi melalui pengukuran nilai ekspektasi suatu operator observabel $\hat{O}$, yang umumnya merepresentasikan fungsi biaya atau besaran fisik yang ingin dioptimalkan. Nilai fungsi biaya tersebut dirumuskan sebagai

$$C(\theta) = \langle \psi(\theta) | \hat{O} | \psi(\theta) \rangle. \quad (2.78)$$

Nilai hasil pengukuran tersebut kemudian dikirimkan kepada *optimizer* klasik untuk memperbarui parameter sirkuit, kemudian proses evaluasi dan pembaruan diulang hingga kriteria konvergensi terpenuhi. Proses ini dikenal sebagai *hybrid quantum-classical optimization* atau optimasi hibrida kuantum-klasik (C dkk., 2025).

2.6 Optimasi Parameter pada Sistem Klasik

Dalam pemodelan sistem makroskopik, pencarian titik ekuilibrium pada algoritma kuantum merupakan proses yang sangat rentan terhadap pengaruh derau atau *noise* lingkungan. Komputasi pada perangkat keras kuantum era NISQ sering kali menghasilkan distorsi sinyal informasi yang signifikan sehingga memerlukan metode optimasi parameter yang stabil untuk menavigasi kompleksitas ruang solusi secara kokoh. Algoritma optimasi klasik digunakan sebagai pendukung sistem hibrida untuk melakukan kalibrasi model secara iteratif untuk meminimalkan fungsi biaya yang ditentukan. Oleh karena itu, diperlukan pemilihan metodologi evaluasi gradien dengan mempertimbangkan efisiensi sumber daya dan ketahanan terhadap fluktuasi statistik yang melekat pada pengukuran observabel kuantum skala industri (Spall, 1998).

Metode optimasi yang tepat memungkinkan sistem untuk mencapai konvergensi energi minimum meskipun beroperasi di tengah keterbatasan koherensi sirkuit (Fontana dkk., 2021). Algoritma gradien klasik konvensional sering kali mengalami kendala teknis saat diterapkan langsung pada lanskap energi kuantum yang bergejolak akibat *noise* pengukuran (Sharma dkk., 2020). Ketidakstabilan numerik ini dapat diatasi dengan menerapkan teknik estimasi gradien yang mengeksplorasi properti analitis dari operator uniter tanpa memperburuk eror yang terjadi pada sistem (Wierichs dkk., 2022). Bab ini akan menguraikan implementasi teknis dari metode *Parameter Shift Rule* dan optimasi stokastik *SPSA* sebagai pilar utama dalam pembaruan parameter sirkuit secara efisien dan akurat.

2.6.1 Metode Pergeseran Parameter (*Parameter Shift Rule*)

Dalam setiap proses optimasi berparameter pada arsitektur komputasi kuantum, diperlukan perhitungan gradien analitis yang cepat dan tepat untuk mengarahkan pembaruan matriks parameter menuju titik konfigurasi energi minimum. Pendekatan konvensional dengan menggunakan metode diferensiasi beda hingga (*finite difference*) sering kali sulit diimplementasikan karena algoritma tersebut sangat bergantung pada presisi pengukuran

selisih ruang parameter berskala sangat kecil. Ketergantungan terhadap interval pergeseran fasa infinitesimal pada perangkat keras sirkuit dunia nyata selalu memicu munculnya ketidakstabilan akibat derau statistik yang merusak akurasi perhitungan gradien. Untuk mengatasi hambatan teknis tersebut, metode *Parameter Shift Rule* diimplementasikan ke dalam simulasi karena mampu mengeksploitasi sifat periodik dari operator uniter pada gerbang rotasi kuantum secara eksak (Mitarai dkk., 2018). Penerapan algoritma perhitungan analitis ini diawali secara sistematis dengan merumuskan turunan parsial dari nilai ekspektasi energi terhadap variabel sudut representasi θ melalui hubungan:

$$\begin{aligned}\frac{\partial E(\theta)}{\partial \theta} &= \frac{\partial}{\partial \theta} \langle \psi(\theta) | \hat{\mathcal{H}} | \psi(\theta) \rangle \\ &= \left\langle \frac{\partial \psi(\theta)}{\partial \theta} \middle| \hat{\mathcal{H}} | \psi(\theta) \right\rangle + \langle \psi(\theta) | \hat{\mathcal{H}} \middle| \frac{\partial \psi(\theta)}{\partial \theta} \rangle.\end{aligned}\quad (2.79)$$

Untuk suatu gerbang rotasi yang bergantung pada parameter θ , keadaan kuantum dapat dituliskan sebagai

$$|\psi(\theta)\rangle = \hat{U}_B \hat{U}(\theta) \hat{U}_A |0\rangle, \quad \hat{U}(\theta) = e^{-i\frac{\theta}{2}\sigma}, \quad (2.80)$$

dengan $\hat{U}(\theta)$ merupakan generator gerbang rotasi yang berbasis pada generator pauli $\sigma \in \{I, X, Y, Z\}$ sebagaimana persamaan (2.62) dengan ukuran matriks sebesar $2^n \times 2^n$ dengan n merupakan jumlah qubit. Definisikan

$$|\phi\rangle = \hat{U}_A |0\rangle, \quad \hat{M} = \hat{U}_B^\dagger \hat{\mathcal{H}} \hat{U}_B \quad (2.81)$$

di mana $\hat{U}_A$ merupakan operator uniter sebelum gerbang rotasi dan $\hat{U}_B$ merupakan operator uniter sesudah gerbang rotasi. Nilai ekspektasi energi kemudian dapat ditulis ulang sebagai

$$\begin{aligned}E(\theta) &= \langle \psi(\theta) | \hat{\mathcal{H}} | \psi(\theta) \rangle \\ &= \langle \phi | \hat{U}^\dagger(\theta) \hat{M} \hat{U}(\theta) | \phi \rangle.\end{aligned}\quad (2.82)$$

Turunannya terhadap θ adalah

$$\frac{\partial E(\theta)}{\partial \theta} = \langle \phi | \frac{\partial \hat{U}^\dagger(\theta)}{\partial \theta} \hat{M} \hat{U}(\theta) | \phi \rangle + \langle \phi | \hat{U}^\dagger(\theta) \hat{M} \frac{\partial \hat{U}(\theta)}{\partial \theta} | \phi \rangle. \quad (2.83)$$

Karena

$$\frac{\partial \hat{U}(\theta)}{\partial \theta} = -\frac{i}{2} \sigma \hat{U}(\theta), \quad (2.84)$$

maka diperoleh

$$\begin{aligned}\frac{\partial E(\theta)}{\partial \theta} &= \frac{i}{2} \langle \psi(\theta) | \sigma \hat{M} | \psi(\theta) \rangle - \frac{i}{2} \langle \psi(\theta) | \hat{M} \sigma | \psi(\theta) \rangle \\ &= \frac{i}{2} \langle \psi(\theta) | [\sigma, \hat{M}] | \psi(\theta) \rangle.\end{aligned}\quad (2.85)$$

Bentuk komutator tersebut sulit diukur secara langsung pada perangkat NISQ. Oleh karena itu digunakan pergeseran parameter sebesar s :

$$\begin{aligned}\hat{U}(\theta \pm s) &= \exp \left[-i \frac{\theta \pm s}{2} \sigma \right] \\ &= \hat{U}(\theta) \hat{U}(\pm s).\end{aligned}\quad (2.86)$$

sebut disajikan pada Daftar Kode 4.1.

```
import yfinance as yf
import pandas as pd

def download_market_data(tickers, benchmark_ticker, start_date, end_date, start_bt_date):
    """
    Mengunduh data saham dan benchmark, melakukan pembersihan dasar,
    dan mengekspor data harga harian.
    """
    print("Mengunduh data saham...")
    data = yf.download(tickers, start=start_date, end=end_date, progress=False)['Close']
    data = data.dropna()
    data_clean = data.sort_index()

    print(f"Mengunduh data indeks pembanding ({benchmark_ticker})...")
    # Mengunduh benchmark mulai dari tanggal backtest agar rets bisa dihitung dengan benar
    benchmark_data = yf.download(benchmark_ticker, start=start_bt_date, end=end_date, progress
    =False)['Close']
    benchmark_data = benchmark_data.dropna()
    benchmark_rets = benchmark_data.pct_change().dropna()

    print(f"Data Berhasil Diunduh. Total hari observasi: {len(data_clean)}")

    return data_clean, benchmark_data, benchmark_rets
```

Kode 4.1: Skrip ekstraksi data harga historis saham menggunakan API *yfinance*

4.1.1 Imbal Hasil dan Kovariansi

Tahap prapemrosesan diawali dengan mentransformasikan data harga saham menjadi data imbal hasil sederhana (*simple return*) dan imbal hasil logaritmik (*log return*). Imbal hasil sederhana harian untuk aset ke- i pada waktu t didefinisikan sebagai rasio perubahan harga terhadap harga pada periode sebelumnya, sedangkan imbal hasil logaritmik diperoleh melalui logaritma natural dari rasio harga berturut-turut. Berdasarkan kedua bentuk imbal hasil tersebut, selanjutnya dihitung nilai ekspektasi imbal hasil (μ_i), varians (σ_i^2), dan kovariansi antar aset sebagai parameter utama dalam model optimasi portofolio. Nilai ekspektasi imbal hasil diperoleh melalui rata-rata data historis pada setiap jendela pelatihan yang terdiri atas $n = 126$ hari perdagangan. Sementara itu, ukuran risiko dihitung menggunakan varians karena menunjukkan tingkat penyebaran imbal hasil terhadap nilai rata-ratanya. Semakin besar nilai varians, semakin tinggi tingkat ketidakpastian pergerakan harga aset tersebut. Selain risiko individual, hubungan antar aset juga dievaluasi melalui kovariansi. Kovariansi bernilai positif menunjukkan kecenderungan dua aset bergerak searah, sedangkan kovariansi bernilai negatif menunjukkan kecenderungan pergerakan yang berlawanan.

Dalam tugas akhir ini, *simple return* digunakan untuk menghitung ekspektasi imbal hasil portofolio karena memiliki sifat linear terhadap bobot aset. Sebaliknya, penggunaan *log return* untuk menghitung ekspektasi imbal hasil portofolio dapat menghasilkan bias akibat sifat nonlinier fungsi logaritma. Bias tersebut dapat dianalisis melalui aproksimasi deret Taylor orde kedua sehingga hubungan antara imbal hasil logaritmik portofolio yang sebenarnya (r_{true}) dan aproksimasi logaritmik berbobot ($\hat{r}_p$) dapat dievaluasi secara kuantitatif. Proses derivasi diawali dengan mendefinisikan dua bentuk representasi *log return* portofolio. Bentuk pertama adalah aproksimasi berbobot yang diperoleh dari penjumlahan *log return* masing-masing aset, sedangkan bentuk kedua merupakan *log return* portofolio yang sebenarnya berdasarkan total *simple return* portofolio. Dengan

menggunakan aproksimasi deret Taylor orde kedua,

$$\ln(1+x) \approx x - \frac{x^2}{2}, \quad (4.1)$$

yang mana kedua bentuk tersebut dapat diekspansikan menjadi

$$\hat{r}_p = \sum_{i=1}^n w_i \ln(1+R_i) \approx \sum_{i=1}^n w_i R_i - \frac{1}{2} \sum_{i=1}^n w_i R_i^2, \quad (4.2)$$

$$r_{true} = \ln(1+R_p) \approx \left(\sum_{i=1}^n w_i R_i \right) - \frac{1}{2} \left(\sum_{i=1}^n w_i R_i \right)^2. \quad (4.3)$$

Selisih antara kedua ekspansi tersebut merepresentasikan galat aproksimasi yang muncul akibat penggunaan *log return* berbobot sebagai pengganti *log return* portofolio yang sebenarnya. Galat tersebut dapat dituliskan sebagai

$$\begin{aligned} \text{Error} &= r_{true} - \hat{r}_p, \\ \text{Error} &= \left(\sum_{i=1}^n w_i R_i - \frac{1}{2} \left(\sum_{i=1}^n w_i R_i \right)^2 \right) - \left(\sum_{i=1}^n w_i R_i - \frac{1}{2} \sum_{i=1}^n w_i R_i^2 \right), \\ \text{Error} &= \frac{1}{2} \left[\sum_{i=1}^n w_i R_i^2 - \left(\sum_{i=1}^n w_i R_i \right)^2 \right]. \end{aligned}$$

Bentuk dalam tanda kurung tersebut identik dengan varians portofolio sehingga diperoleh

$$\text{Error} = \frac{1}{2} \text{Var}(R). \quad (4.4)$$

Sehingga, hubungan antara *log return* portofolio yang sebenarnya dan aproksimasi berbobotnya dapat dinyatakan sebagai

$$r_{true} \approx \hat{r}_p + \frac{1}{2} \text{Var}(R), \quad (4.5)$$

yang menunjukkan bahwa aproksimasi berbasis *log return* mengandung koreksi sebesar setengah varians portofolio. Oleh karena itu, ekspektasi imbal hasil dalam model optimasi portofolio dihitung menggunakan *simple return* agar tetap konsisten dengan sifat linear formulasi Markowitz.

Meskipun ekspektasi imbal hasil dihitung menggunakan *simple return*, estimasi matriks kovariansi dalam penelitian ini dilakukan menggunakan *log return*. Pemilihan tersebut didasarkan pada sifat *log return* yang lebih stabil secara statistik serta memiliki distribusi yang cenderung lebih mendekati asumsi normalitas dibandingkan *simple return*. Selain itu, untuk nilai imbal hasil harian yang relatif kecil, perbedaan antara kedua representasi tersebut sangat kecil sehingga tidak menghasilkan perubahan yang signifikan terhadap estimasi risiko portofolio. Berdasarkan Persamaan 4.5, hubungan antara *simple return* dan *log return* dapat dituliskan secara aproksimatif sebagai

$$r_i \approx R_i - \frac{1}{2} R_i^2. \quad (4.6)$$

Karena nilai imbal hasil harian umumnya berada pada orde 10^{-2} , maka suku kuadrat R_i^2 hanya berada pada orde 10^{-4} dan jauh lebih kecil dibandingkan suku utamanya. Dengan demikian dapat digunakan pendekatan

$$r_i \approx R_i. \quad (4.7)$$

Akibatnya, kovariansi yang dihitung menggunakan *log return* akan mendekati kovariansi yang dihitung menggunakan *simple return*. Secara eksplisit,

$$\text{Cov}(r_A, r_B) \approx \text{Cov}\left(R_A - \frac{R_A^2}{2}, R_B - \frac{R_B^2}{2}\right).$$

Dengan memanfaatkan sifat bilinear kovariansi diperoleh

$$\text{Cov}(r_A, r_B) \approx \text{Cov}(R_A, R_B) - \frac{1}{2}\text{Cov}(R_A, R_B^2) - \frac{1}{2}\text{Cov}(R_A^2, R_B) + \frac{1}{4}\text{Cov}(R_A^2, R_B^2). \quad (4.8)$$

Suku utama pada persamaan tersebut adalah $\text{Cov}(R_A, R_B)$ yang berada pada orde 10^{-4} , sedangkan suku-suku yang melibatkan pangkat dua imbal hasil berada pada orde yang lebih kecil, yaitu sekitar 10^{-6} hingga 10^{-8} . Oleh karena itu kontribusi suku-suku koreksi dapat diabaikan sehingga diperoleh pendekatan

$$\text{Cov}(r_A, r_B) \approx \text{Cov}(R_A, R_B). \quad (4.9)$$

Hasil ini menunjukkan bahwa penggunaan *log return* untuk estimasi matriks kovariansi tetap memberikan representasi risiko yang konsisten terhadap penggunaan *simple return*, terutama pada data saham harian dengan tingkat volatilitas yang relatif rendah.

Transformasi data harga saham menjadi *simple return* dan *log return* diimplementasikan menggunakan pustaka *NumPy* dan *Pandas*. Fungsi yang dikembangkan menerima masukan berupa deret harga historis dan menghasilkan deret imbal hasil sesuai dengan formulasi matematis yang telah dijelaskan sebelumnya. Selain itu, dilakukan pula perhitungan nilai ekspektasi imbal hasil sederhana sebagai parameter pertumbuhan aset pada setiap jendela pelatihan. Untuk keperluan analisis risiko, *log return* dihitung menggunakan transformasi logaritma natural terhadap rasio harga berturut-turut. Sementara itu, estimasi ekspektasi *log return* diperoleh melalui pendekatan koreksi volatilitas (*volatility drag*) yang dinyatakan sebagai selisih antara rata-rata *simple return* dan setengah variansnya. Pendekatan ini digunakan untuk menjaga konsistensi antara estimasi pertumbuhan geometrik dan karakteristik fluktuasi harga yang diamati pada data historis. Implementasi lengkap dari proses transformasi imbal hasil dan perhitungan parameter statistik tersebut ditunjukkan pada Daftar Kode 4.2.

```
import numpy as np
import pandas as pd

def calculate_simple_return(prices):
    """
    Menghitung simple return secara manual berdasarkan rumus:
    R_t = (P_t - P_{t-1}) / P_{t-1}
    """
    prev_prices = prices.shift(1)
    simple_returns = (prices - prev_prices) / prev_prices
    return simple_returns.dropna()

def calculate_log_return(prices):
    """
```

```

    Menghitung log return secara manual berdasarkan rumus:
    r_t = ln(P_t / P_{t-1})
    """
    prev_prices = prices.shift(1)
    log_returns = np.log(prices / prev_prices)
    return log_returns.dropna()

def calculate_expected_simple_return(simple_rets):
    """
    Menghitung expected simple return: mu_R = sum(R_t) / T
    """
    return simple_rets.mean()

def calculate_expected_log_return_with_drag(mu_simple, var_simple):
    """
    Menghitung expected log return dengan volatility drag:
    mu_r = mu_R - 0.5 * sigma_R^2
    """
    return mu_simple - 0.5 * var_simple

```

Kode 4.2: Fungsi prapemrosesan imbal hasil dan ekspektasi pertumbuhan portofolio

4.1.2 Implementasi Numerik pada Sampel Data

Sebagai ilustrasi penerapan formulasi yang telah dijelaskan sebelumnya, dilakukan perhitungan numerik menggunakan data historis saham BBKA pada periode observasi pertama. Berdasarkan Tabel E.1, harga penutupan meningkat dari 4515,50 menjadi 4869,32 sehingga diperoleh *simple return* dan *log return* harian sebagai berikut

$$R_{BBKA,1} = \frac{4869,32 - 4515,50}{4515,50} \approx 0,0783,$$

$$r_{BBKA,1} = \ln(1 + 0,0783) \approx 0,0754.$$

Nilai varians dan kovariansi selanjutnya dihitung menggunakan data *log return* pada jendela pelatihan yang terdiri atas 126 hari perdagangan. Untuk dua aset i dan j , kovariansi sampel didefinisikan sebagai

$$\begin{aligned} \sigma_{ij} &= \frac{1}{n-1} \sum_{t=1}^n (r_{i,t} - \bar{r}_i)(r_{j,t} - \bar{r}_j) \\ &\approx \frac{1}{n-1} \sum_{t=1}^n (R_{i,t} - \bar{R}_i)(R_{j,t} - \bar{R}_j), \end{aligned} \quad (4.10)$$

dengan pendekatan pada baris kedua mengikuti hasil analisis pada Subsection 4.1.1 yang menunjukkan bahwa kovariansi *log return* dan *simple return* bernilai hampir identik untuk data imbal hasil harian.

Untuk konfigurasi sistem dengan $N = 2$ aset, matriks kovariansi pada jendela pelatihan pertama diperoleh dalam bentuk

$$\Sigma_{N2} = \begin{pmatrix} \text{Var}(r_1) & \text{Cov}(r_1, r_2) \\ \text{Cov}(r_2, r_1) & \text{Var}(r_2) \end{pmatrix}, \quad (4.11)$$

yang secara numerik menghasilkan

$$\Sigma_{N2} = \begin{pmatrix} 0,0915 & 0,0202 \\ 0,0202 & 0,0367 \end{pmatrix}. \quad (4.12)$$

Sementara itu, untuk konfigurasi sistem dengan $N = 4$ aset, matriks kovariansi pada jendela pelatihan pertama diperoleh sebagai

$$\Sigma_{N4} = \begin{pmatrix} 0,0171 & 0,0012 & 0,0009 & 0,0022 \\ 0,0012 & 0,0198 & 0,0015 & 0,0018 \\ 0,0009 & 0,0015 & 0,0241 & 0,0011 \\ 0,0022 & 0,0018 & 0,0011 & 0,0225 \end{pmatrix}. \quad (4.13)$$

Matriks kovariansi tersebut selanjutnya digunakan sebagai masukan utama dalam pembentukan fungsi objektif Markowitz dan konstruksi Hamiltonian portofolio pada tahap optimasi kuantum.

4.1.3 Parameter *Risk Aversion* Endogen

Dalam model optimasi portofolio Markowitz, parameter *risk aversion* (γ) umumnya diperlakukan sebagai konstanta eksogen yang nilainya ditentukan sebelum proses optimasi dilakukan. Berbeda dengan pendekatan tersebut, penelitian ini menggunakan parameter *risk aversion* yang dihitung secara endogen berdasarkan karakteristik data pasar pada setiap jendela pelatihan. Dengan demikian, tingkat penalti risiko dapat berubah secara adaptif mengikuti kondisi pasar yang diamati. Nilai γ ditentukan melalui fungsi sigmoid yang dibangun dari perbandingan antara rata-rata ekspektasi imbal hasil logaritmik ($\bar{\mu}_l$) dan rata-rata volatilitas logaritmik ($\bar{\sigma}_l$), yaitu

$$\gamma = \frac{1}{1 + \exp(\bar{\mu}_l / \bar{\sigma}_l)}. \quad (4.14)$$

Rasio $\bar{\mu}_l / \bar{\sigma}_l$ berperan sebagai indikator keseimbangan antara potensi keuntungan dan risiko pasar. Ketika ekspektasi imbal hasil relatif lebih tinggi dibandingkan volatilitas, nilai rasio akan meningkat sehingga menghasilkan nilai γ yang lebih kecil. Sebaliknya, ketika volatilitas mendominasi, nilai γ akan meningkat dan memberikan penalti risiko yang lebih besar pada fungsi objektif optimasi. Sebagai ilustrasi, pada jendela pelatihan pertama diperoleh parameter logaritmik BBCA sebesar $\mu_{l,1} = 0,3164$ dan $\sigma_{l,1} = 0,3024$, serta parameter ADRO sebesar $\mu_{l,2} = 0,1756$ dan $\sigma_{l,2} = 0,1916$. Dari data tersebut diperoleh rata-rata lintas aset

$$\bar{\mu}_l = \frac{0,3164 + 0,1756}{2} = 0,2460, \quad (4.15)$$

$$\bar{\sigma}_l = \frac{0,3024 + 0,1916}{2} = 0,2470. \quad (4.16)$$

Rasio keuntungan terhadap risiko kemudian dihitung sebagai

$$Z = \frac{\bar{\mu}_l}{\bar{\sigma}_l} = \frac{0,2460}{0,2470} \approx 0,9960. \quad (4.17)$$

Dengan mensubstitusikan nilai tersebut ke dalam fungsi sigmoid diperoleh

$$\gamma = \frac{1}{1 + e^{0,9960}} = 0,2697. \quad (4.18)$$

Nilai γ yang relatif rendah menunjukkan bahwa kondisi pasar pada periode tersebut memiliki rasio keuntungan terhadap risiko yang cukup baik. Akibatnya, penalti risiko yang

diberikan pada fungsi objektif menjadi lebih kecil sehingga algoritma optimasi cenderung memberikan toleransi yang lebih besar terhadap pengambilan risiko untuk memperoleh potensi imbal hasil yang lebih tinggi. Riwayat perubahan parameter γ untuk seluruh jendela pengamatan disajikan pada Tabel F.1 dan Tabel H.1.

Perhitungan parameter *risk aversion* endogen diimplementasikan menggunakan pustaka *NumPy*. Fungsi yang dikembangkan menerima masukan berupa vektor ekspektasi imbal hasil dan volatilitas dari seluruh aset pada suatu jendela pelatihan, kemudian menghitung rata-rata lintas aset untuk memperoleh rasio keuntungan terhadap risiko. Nilai rasio tersebut selanjutnya ditransformasikan menggunakan fungsi sigmoid sehingga menghasilkan parameter γ yang berada pada rentang 0 hingga 1. Untuk menjaga stabilitas numerik, fungsi juga melakukan pemeriksaan terhadap nilai yang hilang (*missing values*) maupun kondisi volatilitas bernilai nol. Apabila salah satu kondisi tersebut terdeteksi, fungsi akan mengembalikan nilai $\gamma = 0,5$ sebagai nilai netral. Implementasi lengkap prosedur perhitungan parameter *risk aversion* endogen ditunjukkan pada Daftar Kode 4.3.

```
import numpy as np

def compute_endogenous_lambda(mu_period, sigma_period):
    """
    Menghitung parameter risk-aversion (gamma) secara endogen berdasarkan
    Sharpe Ratio rata-rata lintas aset menggunakan variabel yang sudah
    dihitung (termasuk volatility drag).

    Z = mu_avg / sigma_avg (Sharpe Ratio agregat)
    gamma = 1 / (1 + exp(Z))
    """
    mu_avg = np.abs(mu_period).mean()
    sigma_avg = sigma_period.mean()

    if np.isnan(mu_avg) or np.isnan(sigma_avg) or sigma_avg == 0:
        return 0.5

    Z = mu_avg / sigma_avg # Sharpe Ratio agregat
    gamma = 1.0 / (1.0 + np.exp(Z))

    return gamma
```

Kode 4.3: Algoritma kalkulasi parameter *risk aversion* dinamis berbasis Rasio Sharpe agregat

4.2 Formulasi *Exact Potential Game* (EPG) dan Pencarian *Nash Equilibrium*

Tahap berikutnya adalah memetakan permasalahan optimasi portofolio Markowitz ke dalam kerangka *Exact Potential Game* (EPG). Formulasi yang digunakan mengacu pada model *Mean-Variance* biner sebagaimana telah dijabarkan pada Bab 2.3. Dalam representasi ini, setiap aset diperlakukan sebagai agen strategis yang memilih status biner ($x_i \in 0, 1$), dengan nilai ($x_i = 1$) menyatakan bahwa aset dipilih ke dalam portofolio dan ($x_i = 0$) menyatakan sebaliknya. Untuk memenuhi kendala anggaran, jumlah aset yang dipilih ditetapkan sebanyak ($k = N/2$). Dengan demikian, bobot portofolio dapat dituliskan sebagai ($\omega_i = x_i/k$), sehingga berlaku

$$\sum_i \frac{x_i}{k} = 1. \quad (4.19)$$

Konstruksi tersebut dilakukan agar total alokasi modal selalu memenuhi kendala investasi yang digunakan dalam formulasi optimasi portofolio dasar dari model Markowitz.

4.2.1 Formulasi EPG

Berdasarkan kerangka teorema Monderer dan Shapley yang mendasari teori permainan kooperatif, suatu permainan finansial dikategorikan secara formal sebagai *Exact Potential Game* (EPG) apabila terdapat sebuah fungsi potensial skalar $\Phi(\vec{x})$ sedemikian rupa sehingga ekuivalensi utilitas terpenuhi. Ekuivalensi tersebut mensyaratkan bahwa selisih utilitas individu Δu_i akibat dinamika perubahan strategi satu agen harus secara eksak sama nilainya dengan selisih evaluasi nilai fungsi potensial sistem global tersebut. Fungsi potensial skalar ini secara matematis diperoleh dari penjabaran ekspansi fungsi biaya portofolio kuadratik yang memetakan hubungan kovariansi multivariat antar instrumen. Untuk mengidentifikasi kontribusi marginal spesifik dari agen ke- i , proses derivasi melakukan dekomposisi jumlahan polinomial tersebut dengan mengisolasi suku-suku yang mengandung variabel x_i serta mendayagunakan sifat komutatif variabel biner $x_i^2 = x_i$. Derivasi fisis fungsi potensial tersebut pada akhirnya menghasilkan kuantitas selisih potensial $\Delta\Phi$ yang bertindak langsung sebagai insentif marginal ketika agen i memutuskan untuk mengubah status transisi dari kondisi 0 menjadi 1. Implementasi prosedur perhitungan tersebut menggunakan pustaka *Pandas* untuk melakukan pengelompokan data dan evaluasi probabilitas setiap *microstate*. Detail implementasinya ditunjukkan pada Daftar Kode 4.4.

```
import numpy as np

def compute_strategic_returns(log_rets, binary_st, tickers):
    """
    [Tugas 2: Perhitungan Imbal Hasil Strategis]
    Menghitung imbal hasil strategis (mu_tilde_i) sebagai jumlahan terbobot
    dari ekspektasi bersyarat pada 16 microstates sistem 4 aset.
    """
    n_assets = len(tickers)
    total_days = len(log_rets)
    mu_tilde = np.zeros(n_assets)

    # Kelompokkan return berdasarkan binary states (microstates) dari semua aset
    grouped = log_rets.groupby([binary_st[t] for t in tickers])

    for state, group in grouped:
        P_s = len(group) / total_days          # Probabilitas microstate P(s)
        R_bar_s = group[tickers].mean().values # Ekspektasi return bersyarat R_bar_i(s)
        mu_tilde += P_s * R_bar_s              # Jumlahan terbobot

    return mu_tilde
```

Kode 4.4: Algoritma kalkulasi ekspektasi imbal hasil strategis berbasis observasi probabilitas *microstates*

4.2.2 Pencarian *Nash Equilibrium*

Setelah fungsi potensial sistem diperoleh, langkah berikutnya adalah menentukan konfigurasi strategi yang memenuhi kondisi *Nash Equilibrium*. Pada kerangka *Exact Potential Game*, titik *Nash Equilibrium* ekuivalen dengan maksimum lokal dari fungsi potensial $\Phi(\mathbf{x})$. Oleh karena itu, pencarian kesetimbangan dapat dilakukan dengan mencari konfigurasi biner yang tidak lagi memberikan peningkatan nilai fungsi potensial ketika salah

satu agen mengubah strateginya. Pada tugas akhir ini, pencarian *Nash Equilibrium* dilakukan menggunakan metode *Stochastic Best Response* (SBR). Algoritma ini bekerja melalui mekanisme *swap move*, yaitu menukar satu aset yang sedang berada di dalam portofolio dengan satu aset yang berada di luar portofolio. Pada setiap iterasi, seluruh kandidat pertukaran dievaluasi dan dipilih perubahan yang memberikan kenaikan fungsi potensial terbesar. Proses iteratif tersebut dihentikan ketika tidak ditemukan lagi pertukaran yang mampu meningkatkan nilai fungsi potensial. Sebagai ilustrasi pada sistem dengan $N = 2$, interaksi strategis antar aset dapat direpresentasikan melalui matriks *payoff* pada Tabel 4.1. Setiap elemen matriks menunjukkan utilitas yang diperoleh kedua agen untuk kombinasi status biner tertentu. Berdasarkan evaluasi seluruh konfigurasi yang memungkinkan, keadaan $|10\rangle$ menghasilkan utilitas sistem tertinggi sehingga diidentifikasi sebagai titik *Nash Equilibrium* pada periode observasi tersebut.

Tabel 4.1: Matriks *Payoff* Strategis Sistem $N=2$ (Januari 2021).

		Aset ADRO (x_2)	
		Status 0	Status 1
Aset BBKA (x_1)	Status 0	(0, 0)	(0, 0,1890)
	Status 1	(0,3498, 0)	(0,3444, 0,1836)

Pada sistem dengan $N = 4$, jumlah konfigurasi yang memenuhi kendala $k = 2$ meningkat secara signifikan sehingga evaluasi seluruh kemungkinan strategi menjadi lebih kompleks. Oleh karena itu, pencarian kesetimbangan harus dilakukan menggunakan prosedur SBR yang menelusuri ruang solusi melalui serangkaian pertukaran aset sampai pada titik konfigurasi yang tidak mengalami peningkatan fungsi potensial lagi. Riwayat iterasi dan konfigurasi yang dihasilkan pada setiap periode pengamatan disajikan pada Lampiran H. Konfigurasi *Nash Equilibrium* yang diperoleh memiliki peran penting dalam penelitian ini karena digunakan sebagai titik inisialisasi bagi algoritma *Variational Quantum Eigensolver* (VQE). Pemanfaatan solusi hasil teori permainan sebagai tebakan awal memungkinkan proses optimasi kuantum dimulai dari wilayah solusi yang telah memiliki kualitas utilitas tinggi sehingga mempercepat konvergensi menuju keadaan energi minimum.

Implementasi pencarian *Nash Equilibrium* dilakukan menggunakan Python melalui algoritma *Stochastic Best Response*. Fungsi yang dikembangkan menerima masukan berupa vektor ekspektasi imbal hasil, matriks kovariansi, dan parameter *risk aversion*, kemudian mengevaluasi fungsi potensial untuk setiap kandidat pertukaran aset yang memenuhi kendala portofolio. Selama masih terdapat pertukaran yang meningkatkan fungsi potensial, konfigurasi portofolio akan diperbarui secara iteratif hingga tercapai kondisi konvergen. Selain menghasilkan konfigurasi kesetimbangan akhir, algoritma juga merekam seluruh riwayat iterasi sehingga proses konvergensi dapat dianalisis lebih lanjut. Implementasi lengkap prosedur tersebut ditunjukkan pada Daftar Kode 4.5.

```
import numpy as np
import pandas as pd
import os

def find_nash_sbr(mu, sigma_matrix, gamma, curr_date, N=4, K=2, max_iters=100, history_file='
    riwayat_nash_sbr.csv'):
    """
    Pencarian Nash Equilibrium menggunakan Sequential Best Response (SBR)
    dalam Exact Potential Game.
```

```

Potensial (Utilitas Portofolio):
Phi = (1/K) * mu^T * x - (gamma / (2 * K^2)) * x^T * sigma * x

Dalam Potential Game, Nash Equilibrium adalah local maximum dari fungsi potensial.
Karena ada batasan sum(x) = K, kita menggunakan swap moves untuk mencari NE.
"""
# Inisialisasi: pilih K aset pertama
current_selection = set(range(K))
history = []

def calculate_potential(selection):
    x = np.zeros(N)
    for idx in selection: x[idx] = 1

    # 1. Komponen Return: (1/K) * sum(mu_i * x_i)
    ret = np.sum(mu[list(selection)]) / K

    # 2. Komponen Risiko: (gamma / (2 * K^2)) * sum(sigma_ij * x_i * x_j)
    # x^T * Sigma * x untuk subset selection
    if len(selection) > 0:
        sub_sigma = sigma_matrix[np.ix_(list(selection), list(selection))]
        risk = np.sum(sub_sigma) * (gamma / (2.0 * K**2))
    else:
        risk = 0.0

    return ret - risk

current_phi = calculate_potential(current_selection)

history.append({
    'Date': curr_date.date(),
    'Iteration': 0,
    'Bitstring': "".join(str(int(i in current_selection)) for i in range(N)),
    'Utility': current_phi,
    'Swap': 'Initial'
})

for iteration in range(1, max_iters + 1):
    improved = False
    out_portfolio = set(range(N)) - current_selection

    best_swap = None
    max_phi = current_phi

    # SBR/Local Search: Coba tukar satu aset di dalam dengan satu aset di luar
    # Mencari swap yang meningkatkan fungsi potensial paling besar
    for i in current_selection:
        for j in out_portfolio:
            new_selection = (current_selection - {i}) | {j}
            new_phi = calculate_potential(new_selection)

            if new_phi > max_phi:
                max_phi = new_phi
                best_swap = (i, j)

    if best_swap:
        i, j = best_swap
        current_selection = (current_selection - {i}) | {j}
        current_phi = max_phi
        improved = True
        history.append({
            'Date': curr_date.date(),
            'Iteration': iteration,
            'Bitstring': "".join(str(int(idx in current_selection)) for idx in range(N)),
            'Utility': current_phi,
            'Swap': f"{i}<->{j}"
        })

    if not improved:
        break

# --- EKSPOR RIWAYAT NASH SBR ---

```

```

df_history = pd.DataFrame(history)
if not os.path.exists(history_file):
    df_history.to_csv(history_file, index=False)
else:
    df_history.to_csv(history_file, mode='a', header=False, index=False)

final_x = np.zeros(N, dtype=int)
for idx in current_selection: final_x[idx] = 1
# Kembalikan bitstring dan utilitas akhir (sebagai potensial)
return "".join(str(bit) for bit in final_x), current_phi

```

Kode 4.5: Skrip implementasi *Stochastic Best Response* untuk pencarian konvergensi *Nash Equilibrium*

4.3 Pemetaan Hamiltonian Ising

Transformasi permasalahan optimasi portofolio ke dalam domain komputasi kuantum dilakukan melalui formulasi *Quadratic Unconstrained Binary Optimization* (QUBO). Pada tahap ini, fungsi potensial pada teori permainan ditransformasikan menjadi fungsi energi sehingga sesuai dengan prinsip minimisasi energi yang digunakan oleh algoritma *Variational Quantum Eigensolver* (VQE). Hubungan tersebut dinyatakan sebagai

$$\begin{aligned}
 E(\vec{x}) &= -\Phi(\vec{x}) \\
 &= -\left(\frac{1}{k}\vec{\mu}^T\vec{x} - \frac{\gamma}{2k^2}\vec{x}^T\Sigma\vec{x}\right).
 \end{aligned} \tag{4.20}$$

Selanjutnya, fungsi energi dalam bentuk QUBO dipetakan ke Hamiltonian Ising menggunakan transformasi variabel biner ke variabel spin sebagaimana telah dijelaskan pada Bab 2. Proses ini menghasilkan operator Hamiltonian yang dapat direpresentasikan dalam bentuk

$$\hat{\mathcal{H}} = \sum_i h_i \hat{Z}_i + \sum_{i<j} J_{ij} \hat{Z}_i \hat{Z}_j + C, \tag{4.21}$$

dengan h_i menyatakan koefisien bias linear, J_{ij} menyatakan koefisien interaksi antar-spin, dan C merupakan konstanta pergeseran energi. Operator Hamiltonian inilah yang kemudian digunakan sebagai fungsi objektif kuantum pada proses optimasi VQE.

4.3.1 Implementasi Numerik Hamiltonian Ising

Setelah parameter QUBO dipetakan ke model Ising, Hamiltonian kuantum dapat dinyatakan dalam bentuk operator Pauli-(Z). Untuk sistem dengan dua aset ($N = 2$) dan kendala kardinalitas ($k = 1$), Hamiltonian memiliki bentuk

$$\hat{\mathcal{H}}_{N2} = h_1 \hat{Z}_1 + h_2 \hat{Z}_2 + J * 12 \hat{Z}_1 \hat{Z}_2 + C. \tag{4.22}$$

Pada sistem dengan empat aset ($N=4$) dan kendala kardinalitas ($k=2$), jumlah interaksi pasangan meningkat sehingga Hamiltonian berkembang menjadi

$$\begin{aligned}
 \hat{\mathcal{H}}_{N4} = & h_1 \hat{Z}_1 + h_2 \hat{Z}_2 + h_3 \hat{Z}_3 + h_4 \hat{Z}_4 \\
 & + J_{12} \hat{Z}_1 \hat{Z}_2 + J_{13} \hat{Z}_1 \hat{Z}_3 + J_{14} \hat{Z}_1 \hat{Z}_4 + J_{23} \hat{Z}_2 \hat{Z}_3 + J_{24} \hat{Z}_2 \hat{Z}_4 + J_{34} \hat{Z}_3 \hat{Z}_4 + C.
 \end{aligned} \tag{4.23}$$

Persamaan di atas menunjukkan bahwa setiap aset direpresentasikan oleh sebuah qubit, sedangkan hubungan risiko antar aset direpresentasikan melalui koefisien interaksi (J_{ij}). Dengan demikian, struktur korelasi yang semula tersimpan dalam matriks kovariansi ditransformasikan menjadi jaringan interaksi spin pada Hamiltonian Ising. Sebagai ilustrasi numerik pada jendela pelatihan pertama untuk sistem dua aset, proses pemetaan menghasilkan Hamiltonian Ising

$$\hat{\mathcal{H}}_{N2} = 0,1735 \hat{Z}_1 + 0,0931 \hat{Z}_2 + 0,5014 \hat{Z}_1 \hat{Z}_2 + 0,2320. \quad (4.24)$$

Koefisien linear $h_1 = 0,1735$ dan $h_2 = 0,0931$ merepresentasikan bias energi yang terkait dengan masing-masing aset, sedangkan koefisien interaksi $J_{12} = 0,5014$ menunjukkan adanya kopling yang relatif lebih dominan dibandingkan kontribusi linear individual. Sementara itu, konstanta $C = 0,2320$ berfungsi sebagai pergeseran energi global dan tidak memengaruhi posisi keadaan dasar (*ground state*) sistem. Besarnya nilai koefisien interaksi menunjukkan bahwa struktur Hamiltonian tidak hanya dipengaruhi oleh ekspektasi imbal hasil dan kovariansi aset, tetapi juga oleh komponen penalti yang digunakan untuk menegakkan kendala kardinalitas portofolio. Dengan demikian, konfigurasi yang melanggar kendala pemilihan aset akan memperoleh energi yang lebih tinggi dibandingkan konfigurasi yang memenuhi batasan investasi yang telah ditetapkan. Hamiltonian hasil pemetaan inilah yang selanjutnya digunakan sebagai fungsi objektif kuantum pada algoritma VQE. Hamiltonian inilah yang selanjutnya digunakan sebagai masukan utama pada algoritma *Variational Quantum Eigensolver* untuk mencari konfigurasi portofolio berenergi minimum.

Hamiltonian Ising diimplementasikan ke dalam simulator kuantum menggunakan pustaka *PennyLane*. Fungsi yang dikembangkan menerima masukan berupa himpunan koefisien bias linear (h_i), koefisien interaksi (J_{ij}), jumlah aset, serta konstanta pergeseran energi C . Selanjutnya, setiap koefisien dipetakan ke operator Pauli-Z yang sesuai sehingga terbentuk Hamiltonian Ising dalam format yang dapat dievaluasi secara langsung oleh algoritma VQE. Koefisien yang bernilai sangat kecil ($< 10^{-12}$) dapat diabaikan untuk menghindari penambahan operator yang tidak memberikan kontribusi signifikan terhadap energi sistem. Implementasi lengkap prosedur perakitan Hamiltonian tersebut ditunjukkan pada Daftar Kode 4.6.

```
import pennylane as qml

def build_hamiltonian_total(h_total, J_total, n_assets, offset=0.0):
    """
    Membangun operator Hamiltonian Ising dari parameter h_i dan J_ij,
    termasuk konstanta pergeseran energi C_Ising.
    H = sum(h_i * Z_i) + sum(J_ij * Z_i * Z_j) + offset * I
    """
    coeffs = []
    obs = []

    # Suku Bias (h_i * Z_i)
    for i in range(n_assets):
        if abs(h_total[i]) > 1e-12:
            coeffs.append(float(h_total[i]))
            obs.append(qml.PauliZ(i))

    # Suku Interaksi (J_ij * Z_i * Z_j)
    for i in range(n_assets):
        for j in range(i + 1, n_assets):
            if abs(J_total[i, j]) > 1e-12:
                coeffs.append(float(J_total[i, j]))
                obs.append(qml.PauliZ(i) @ qml.PauliZ(j))
```

```

# Suku Konstanta (offset * Identity)
if abs(offset) > 1e-12:
    coeffs.append(float(offset))
    obs.append(qml.Identity(0))

if len(coeffs) == 0:
    coeffs.append(0.0)
    obs.append(qml.Identity(0))

return qml.Hamiltonian(coeffs, obs)

```

Kode 4.6: Skrip perakitan operator Hamiltonian Ising dinamis menggunakan pustaka *PennyLane*

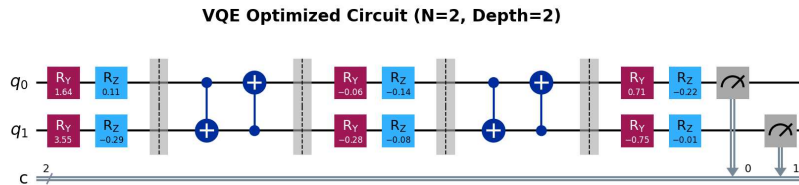
4.4 Arsitektur *Ansatz* dalam VQE

4.4.1 Konstruksi *Hardware-Efficient Ansatz*

Setelah Hamiltonian Ising berhasil dibangun, langkah berikutnya adalah menyiapkan keadaan kuantum variasional yang digunakan untuk menghitung energi ekspektasi sistem. Dalam algoritma VQE, keadaan kuantum tersebut dinyatakan sebagai fungsi dari parameter bebas (θ) yang dioptimasi secara iteratif untuk mendekati keadaan dasar (*ground state*) Hamiltonian. Ansatz yang digunakan pada tugas akhir ini adalah arsitektur *Hardware-Efficient Ansatz* (HEA), yaitu rangkaian kuantum yang tersusun atas lapisan gerbang rotasi satu-qubit dan lapisan keterikatan (*entanglement*) antar-qubit. Struktur ini dipilih karena memiliki jumlah parameter yang relatif sedikit, mudah diimplementasikan pada perangkat kuantum, dan mampu menghasilkan ruang keadaan yang cukup fleksibel untuk merepresentasikan solusi optimasi portofolio. Secara umum, keadaan variasional yang dibangun oleh HEA dapat dituliskan sebagai

$$|\psi(\theta)\rangle = (U_{\text{ent}}U_{\text{rot}}(\theta))^L |0\rangle^{\otimes N}, \quad (4.25)$$

dengan L menyatakan kedalaman (*depth*) sirkuit, U_{rot} - menyatakan lapisan rotasi parametrik, dan U_{ent} - menyatakan lapisan keterikatan antar-qubit. Pada penelitian ini digunakan konfigurasi sebagaimana ditunjukkan pada Gambar 4.1, yaitu HEA dengan satu lapisan keterikatan ($L = 1$). Melalui variasi parameter (θ), sirkuit menghasilkan himpunan keadaan kuantum kandidat yang berbeda-beda. Algoritma optimasi klasik kemudian memperbarui parameter tersebut secara iteratif untuk memperoleh keadaan dengan energi ekspektasi minimum terhadap Hamiltonian yang telah dibangun pada tahap sebelumnya.



Gambar 4.1: Rangkaian Sirkuit Kuantum *Hardware-Efficient Ansatz* dengan Kedalaman (L) = 1.

4.4.2 Implementasi Sirkuit Kuantum Variasional

Pada arsitektur *Hardware-Efficient Ansatz* yang digunakan, setiap lapisan rotasi dibangun dari kombinasi gerbang $\hat{R}_Y$ dan $\hat{R}_Z$ yang diterapkan pada seluruh qubit. Untuk lapisan ke- l , operator rotasi dapat dinyatakan sebagai

$$\hat{U}_{\text{rot}}^{(l)}(\theta) = \bigotimes_{j=0}^{N-1} \left[\hat{R}_Y(\theta_{l,j,y}) \hat{R}_Z(\theta_{l,j,z}) \right], \quad (4.26)$$

dengan N menyatakan jumlah qubit yang merepresentasikan aset portofolio, sedangkan $\theta_{l,j,y}$ dan $\theta_{l,j,z}$ merupakan parameter variasional yang dioptimasi selama proses VQE. Lapisan rotasi ini berfungsi untuk mengatur amplitudo dan fase keadaan kuantum sehingga ruang solusi dapat dieksplorasi secara fleksibel. Berdasarkan definisi gerbang rotasi $R_y(\theta)$ dan $R_z(\theta)$:

$$\begin{aligned} U_{1q}(\theta) &= R_z(\theta_{1z}) R_y(\theta_{1y}) \\ &= \begin{pmatrix} e^{-i\theta_{1z}/2} & 0 \\ 0 & e^{i\theta_{1z}/2} \end{pmatrix} \begin{pmatrix} \cos(\theta_{1y}/2) & -\sin(\theta_{1y}/2) \\ \sin(\theta_{1y}/2) & \cos(\theta_{1y}/2) \end{pmatrix} \\ &= \begin{pmatrix} e^{-i\theta_{1z}/2} \cos(\theta_{1y}/2) & -e^{-i\theta_{1z}/2} \sin(\theta_{1y}/2) \\ e^{i\theta_{1z}/2} \sin(\theta_{1y}/2) & e^{i\theta_{1z}/2} \cos(\theta_{1y}/2) \end{pmatrix} \end{aligned} \quad (4.27)$$

Maka untuk sistem 2 qubit bentuknya menjadi

$$\begin{aligned} U_q(\theta) &= [R_z(\theta_{1z}) R_y(\theta_{1y})] \otimes [R_z(\theta_{2z}) R_y(\theta_{2y})] \\ &= \begin{pmatrix} e^{-i\theta_{1z}/2} \cos(\theta_{1y}/2) & -e^{-i\theta_{1z}/2} \sin(\theta_{1y}/2) \\ e^{i\theta_{1z}/2} \sin(\theta_{1y}/2) & e^{i\theta_{1z}/2} \cos(\theta_{1y}/2) \end{pmatrix} \otimes \begin{pmatrix} e^{-i\theta_{2z}/2} \cos(\theta_{2y}/2) & -e^{-i\theta_{2z}/2} \sin(\theta_{2y}/2) \\ e^{i\theta_{2z}/2} \sin(\theta_{2y}/2) & e^{i\theta_{2z}/2} \cos(\theta_{2y}/2) \end{pmatrix} \\ &= \begin{pmatrix} e^{-i(\theta_{1z}+\theta_{2z})/2} c_1 c_2 & e^{-i(\theta_{1z}+\theta_{2z})/2} c_1 s_2 & e^{-i(\theta_{1z}+\theta_{2z})/2} s_1 c_2 & e^{-i(\theta_{1z}+\theta_{2z})/2} s_1 s_2 \\ e^{-i\theta_{1z}/2} e^{i\theta_{2z}/2} c_1 s_2 & e^{-i\theta_{1z}/2} e^{i\theta_{2z}/2} c_1 c_2 & e^{-i\theta_{1z}/2} e^{i\theta_{2z}/2} s_1 s_2 & e^{-i\theta_{1z}/2} e^{i\theta_{2z}/2} s_1 c_2 \\ e^{i\theta_{1z}/2} e^{-i\theta_{2z}/2} s_1 c_2 & e^{i\theta_{1z}/2} e^{-i\theta_{2z}/2} s_1 s_2 & e^{i\theta_{1z}/2} e^{-i\theta_{2z}/2} c_1 c_2 & e^{i\theta_{1z}/2} e^{-i\theta_{2z}/2} c_1 s_2 \\ e^{i(\theta_{1z}+\theta_{2z})/2} s_1 s_2 & e^{i(\theta_{1z}+\theta_{2z})/2} s_1 c_2 & e^{i(\theta_{1z}+\theta_{2z})/2} c_1 s_2 & e^{i(\theta_{1z}+\theta_{2z})/2} c_1 c_2 \end{pmatrix} \end{aligned} \quad (4.28)$$

dengan

$$c_i = \cos \frac{\theta_{iy}}{2}, \quad s_i = \sin \frac{\theta_{iy}}{2}.$$

Setelah seluruh qubit mengalami rotasi parametrik, diterapkan lapisan keterikatan (*entanglement*) menggunakan gerbang CNOT. Secara umum, lapisan keterbelitan didefinisikan sebagai

$$U_{\text{ent}} = \text{CNOT}_{(N-1,0)} \prod_{q=0}^{N-2} \text{CNOT}_{(q,q+1)} \quad (4.29)$$

menggunakan konvensi

$$\prod_{q=0}^{N-2} A_{(q)} = A_{N-2} A_{N-3} \cdots A_1 A_0 \quad (4.30)$$

karena operator yang paling kanan bekerja terlebih dulu pada keadaan kuantum. Untuk sistem dua aset ($N = 2$), keterikatan dibangun melalui hubungan langsung antara kedua qubit sehingga bentuknya menjadi

$$U_{\text{ent}} = \text{CNOT}_{(1,0)} \cdot \text{CNOT}_{(0,1)} \quad (4.31)$$

dengan matriks untuk masing-masing gerbang adalah:

$$CNOT_{(1,0)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \quad CNOT_{(0,1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \quad (4.32)$$

Sehingga matriks total keterbelitannya pada sistem 2 aset berbentuk

$$U_{ent} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \quad (4.33)$$

Sementara itu, pada sistem empat qubit ($N = 4$), operator keterikatan dibentuk oleh rangkaian gerbang CNOT yang disusun secara melingkar sehingga diperoleh

$$U_{ent} = CNOT_{(3,0)} CNOT_{(2,3)} CNOT_{(1,2)} CNOT_{(0,1)}. \quad (4.34)$$

Karena sistem terdiri atas empat qubit, maka ruang Hilbert yang dihasilkan memiliki dimensi 2^4 . Artinya, setiap operator $CNOT_{(i,j)}$ direpresentasikan oleh matriks uniter berukuran 16×16 , sehingga operator keseluruhan U_{ent} juga merupakan matriks uniter berukuran 16×16 . Kombinasi lapisan rotasi dan lapisan keterikatan menghasilkan keadaan variasional ($|\psi(\theta)\rangle$) yang selanjutnya digunakan untuk menghitung nilai energi ekspektasi Hamiltonian Ising. Melalui pembaruan parameter (θ) secara iteratif, algoritma VQE menelusuri ruang keadaan kuantum untuk menemukan konfigurasi dengan energi minimum yang merepresentasikan solusi optimasi portofolio.

Hardware-Efficient Ansatz juga diimplementasikan menggunakan pustaka *PennyLane*. Fungsi ansatz menerima vektor parameter variasional yang kemudian diorganisasikan ke dalam beberapa lapisan rotasi sesuai kedalaman sirkuit yang ditentukan. Pada setiap lapisan, seluruh qubit dikenai gerbang RY dan RZ, kemudian dihubungkan melalui rangkaian gerbang CNOT dengan pola keterikatan melingkar. Struktur ini menghasilkan rangkaian kuantum yang bersifat fleksibel namun tetap efisien dari sisi jumlah parameter. Selain mampu merepresentasikan berbagai konfigurasi keadaan kuantum kandidat, arsitektur tersebut juga memungkinkan pembentukan keterbelitan antar-qubit yang diperlukan untuk memodelkan korelasi antar aset pada proses optimasi portofolio. Implementasi lengkap konstruksi *ansatz* ditunjukkan pada Daftar Kode 4.7.

```
def ansatz(params):
    """
    Membangun lapisan-lapisan sirkuit kuantum HEA berdasarkan
    parameter rotasi (theta) yang diberikan.
    """
    w = params.reshape((depth + 1, n_qubits, 2))

    # Iterasi penyusunan lapisan (layer)
    for layer in range(depth + 1):
        # 1. Lapisan rotasi lokal (RY dan RZ)
        for q in range(n_qubits):
           qml.RY(w[layer, q, 0], wires=q)
           qml.RZ(w[layer, q, 1], wires=q)

        # 2. Lapisan keterikatan / entanglement (CNOT)
        if layer < depth:
            for q in range(n_qubits - 1):
                qml.CNOT(wires=[q, q + 1])
```

Berdasarkan fungsi biaya tersebut, prosedur SPSA dimulai dengan menentukan parameter awal, amplitudo gangguan, serta vektor perturbasi Rademacher yang akan digunakan untuk mengestimasi gradien. Misalkan parameter awal dipilih sebagai

$$\boldsymbol{\theta}^{(0)} = \begin{pmatrix} \pi/2 \\ \pi/2 \end{pmatrix} \approx \begin{pmatrix} 1,5708 \\ 1,5708 \end{pmatrix}, \quad (4.48)$$

dengan koefisien perturbasi $c_k = 0,1$ dan vektor gangguan

$$\Delta_k = \begin{pmatrix} 1 \\ -1 \end{pmatrix}. \quad (4.49)$$

Dari konfigurasi tersebut diperoleh dua titik evaluasi parameter

$$\boldsymbol{\theta}_+ = \boldsymbol{\theta}^{(0)} + c_k \Delta_k = \begin{pmatrix} 1,6708 \\ 1,4708 \end{pmatrix}, \quad (4.50)$$

dan

$$\boldsymbol{\theta}_- = \boldsymbol{\theta}^{(0)} - c_k \Delta_k = \begin{pmatrix} 1,4708 \\ 1,6708 \end{pmatrix}. \quad (4.51)$$

Nilai energi pada kedua titik tersebut kemudian dievaluasi menggunakan fungsi biaya yang sama sehingga diperoleh energi E_+ dan E_- . Selisih kedua energi ini digunakan untuk membangun estimator gradien SPSA

$$\hat{\mathbf{g}} = \frac{E_+ - E_-}{2c_k} \Delta_k^{-1}. \quad (4.52)$$

Sebagai ilustrasi numerik, misalkan hasil evaluasi menghasilkan $E_+ = 0,2066$ dan $E_- = 0,2227$, sehingga

$$E_+ - E_- = -0,0161. \quad (4.53)$$

Estimator gradien yang diperoleh menjadi

$$\hat{\mathbf{g}} = \begin{pmatrix} -0,0805 \\ 0,0805 \end{pmatrix}. \quad (4.54)$$

Dengan menggunakan laju pembelajaran $\eta = 0,1$, parameter kemudian diperbarui menurut aturan SPSA

$$\boldsymbol{\theta}^{(1)} = \boldsymbol{\theta}^{(0)} - \eta \hat{\mathbf{g}} = \begin{pmatrix} 1,5789 \\ 1,5627 \end{pmatrix}. \quad (4.55)$$

Siklus perturbasi, estimasi gradien, dan pembaruan parameter tersebut diulang secara terus-menerus hingga perubahan energi antar iterasi menjadi sangat kecil. Pada implementasi penelitian ini, proses optimasi dilakukan secara penuh terhadap energi ekspektasi yang dihitung langsung dari simulasi sirkuit kuantum VQE, sehingga seluruh parameter rotasi pada *hardware-efficient ansatz* diperbarui secara simultan menuju keadaan energi minimum. Konvergensi energi yang dicapai pada akhir proses optimasi kemudian diinterpretasikan sebagai pendekatan terhadap *ground state* Hamiltonian Ising yang merepresentasikan solusi portofolio optimal.

Untuk mengotomatisasi proses optimasi tersebut pada seluruh jendela data pengamatan, algoritma SPSA diimplementasikan menggunakan pustaka *NumPy*. Implementasi ini

membangkitkan vektor perturbasi Rademacher secara acak pada setiap iterasi, mengevaluasi energi ekspektasi pada dua titik perturbasi yang simetris, lalu memperbarui parameter sirkuit menggunakan estimator gradien SPSA. Selain itu, koefisien pembelajaran a_k dan amplitudo perturbasi c_k dibuat menurun secara adaptif mengikuti formulasi standar SPSA sehingga proses optimasi tetap stabil ketika mendekati titik minimum energi. Struktur algoritma lengkap yang digunakan pada penelitian ini ditunjukkan pada Daftar Kode 4.8.

```
import numpy as np

def run_spsa_test(cost_circuit, n_params, init_params=None, a_base=0.1, c_base=0.1, max_iters=30, batch_size=25):
    """Versi ringan SPSA untuk pengetesan Learning Rate."""
    rng = np.random.default_rng(42)
    if init_params is not None:
        params = init_params.copy()
    else:
        params = rng.uniform(0, 2 * np.pi, n_params)

    A_coeff = max_iters * 0.1
    alpha_exp = 0.602
    gamma_exp = 0.101

    total_iters = 0
    while total_iters < max_iters:
        for _ in range(batch_size):
            k = total_iters
            a_k = a_base / (A_coeff + k + 1) ** alpha_exp
            c_k = c_base / (k + 1) ** gamma_exp

            # Formulasi parameter perturbasi serentak (Rademacher)
            delta = 2 * rng.integers(0, 2, size=n_params) - 1

            # Estimasi fungsi biaya energi (cost circuit)
            cost_plus = float(cost_circuit(params + c_k * delta))
            cost_minus = float(cost_circuit(params - c_k * delta))

            # Kalkulasi gradien
            grad = (cost_plus - cost_minus) / (2 * c_k * delta)
            params = params - a_k * grad

            total_iters += 1

    return float(cost_circuit(params))
```

Kode 4.8: Implementasi algoritma dasar *Simultaneous Perturbation Stochastic Approximation* (SPSA)

4.6 Evolusi Entropi *Von Neumann* dan Distribusi Probabilitas

Dalam kasus simulasi pencarian alokasi portofolio optimum menggunakan VQE, probabilitas keputusan strategis yang didapatkan senantiasa mengalami perubahan pada setiap siklus evaluasi parameter. Pada tahap awal pembaruan iterasi, sirkuit kuantum pada umumnya menghasilkan distribusi probabilitas yang tersebar secara merata pada seluruh kemungkinan formasi kombinasi basis. Pola persebaran semacam ini mengindikasikan bahwa arsitektur algoritma kuantum masih berada dalam pengaruh keadaan superposisi maksimal dengan nilai entropi global yang sangat tinggi. Namun, seiring dengan peluruhan pembaruan parameter rotasi oleh *optimizer*, sistem kuantum pada akhirnya akan

mengalami peluruhan fungsi gelombang menuju satu basis keadaan. Fenomena tersebut dapat teramati pada hasil simulasi yang diperoleh. Pada Gambar I.5, nilai Entropi *Von Neumann* mengalami penurunan sepanjang proses optimasi SPSA hingga mencapai daerah konvergensi bernilai rendah. Penurunan entropi tersebut mengindikasikan bahwa parameter sirkuit mampu menghasilkan perubahan distribusi probabilitas signifikan selama proses optimasi berlangsung. Sebaliknya, pada Gambar I.2, nilai Entropi *Von Neumann* cenderung bertahan pada tingkat yang relatif tinggi meskipun proses optimasi telah berlangsung selama banyak iterasi. Kondisi tersebut mengindikasikan bahwa distribusi probabilitas sistem tetap tersebar pada banyak keadaan basis sehingga proses lokalisasi probabilitas menuju satu konfigurasi dominan tidak berlangsung secara efektif.

Untuk merealisasikan evaluasi tersebut pada lingkungan komputasi numerik, keadaan kuantum akhir yang dihasilkan oleh sirkuit VQE terlebih dahulu direpresentasikan sebagai *statevector* $|\psi\rangle$. Selanjutnya dilakukan proses *partial trace* terhadap seluruh qubit kecuali satu subsistem pengamatan guna memperoleh matriks densitas tereduksi ρ_A . Nilai eigen dari matriks ρ_A kemudian digunakan untuk menghitung Entropi *Von Neumann* sesuai Persamaan (??). Implementasi algoritmik dari prosedur tersebut ditunjukkan pada Daftar Kode 4.9.

```
import numpy as np

def calculate_entanglement_entropy(psi):
    """
    Menghitung Entanglement Entropy (Von Neumann) dari statevector N-qubit.
    Menggunakan partial trace terhadap semua qubit kecuali qubit 0.
    """
    # 1. Konversi ke numpy array complex
    psi = np.array(psi, dtype=complex)
    dim = len(psi)
    n_qubits = int(np.round(np.log2(dim)))

    # 2. Partial Trace untuk mendapatkan Reduced Density Matrix rho_A (Qubit 0)
    psi_resaped = psi.reshape(2, 2**(n_qubits - 1))
    rho_A = np.dot(psi_resaped, psi_resaped.conj().T)

    # 3. Hitung Eigenvalues dari rho_A (matriks 2x2)
    eigvals = np.linalg.eigvalsh(rho_A)

    # 4. Bersihkan nilai negatif sangat kecil akibat floating point error
    eigvals = eigvals[eigvals > 1e-12]

    # 5. Von Neumann Entropy: S = -sum(p * log2(p))
    if len(eigvals) == 0:
        return 0.0

    return -np.sum(eigvals * np.log2(eigvals))
```

Kode 4.9: Skrip kalkulasi fungsional matriks evaluasi observabel algoritma Entropi *Von Neumann* (*Entanglement Entropy*) komputasional menggunakan matriks *Partial Trace* arsitektur operasional pada ruang fase deterministik fungsi ekuilibrium

4.7 Validasi Solusi melalui Algoritma *Brute Force*

4.7.1 Kalkulasi Energi Kombinatorial

Validasi hasil optimasi yang diperoleh melalui algoritma VQE dilakukan menggunakan metode *brute force*, yaitu dengan mengevaluasi seluruh konfigurasi *bitstring* yang mungkin pada ruang solusi. Berbeda dengan VQE yang menelusuri lanskap energi secara varia-

sional melalui pembaruan parameter sirkuit kuantum, metode *brute force* menghitung energi setiap keadaan basis secara eksplisit sehingga solusi energi minimum global dapat diperoleh secara pasti. Oleh karena itu, metode ini digunakan sebagai acuan untuk menilai akurasi hasil optimasi kuantum yang diperoleh pada setiap jendela data pengamatan. Secara matematis, energi setiap konfigurasi portofolio dihitung menggunakan Hamiltonian Ising yang telah diperoleh pada bagian sebelumnya. Untuk suatu konfigurasi biner $\mathbf{x}$, energi sistem diberikan oleh

$$E(\mathbf{x}) = \sum_i h_i z_i + \sum_{i < j} J_{ij} z_i z_j + C, \quad (4.56)$$

dengan variabel spin didefinisikan sebagai

$$z_i = 1 - 2x_i, \quad (4.57)$$

sehingga setiap *bitstring* portofolio dapat langsung dipetakan ke keadaan spin yang bersesuaian. Pada sistem yang terdiri atas N aset, metode *brute force* harus mengevaluasi sebanyak 2^N konfigurasi yang berbeda untuk menemukan energi minimum global.

Sebagai ilustrasi numerik, untuk sistem dua aset ($N = 2$) dengan Hamiltonian

$$\hat{\mathcal{H}}_{N2} = 0,1735\hat{Z}_1 + 0,0931\hat{Z}_2 + 0,5014\hat{Z}_1\hat{Z}_2 + 0,2320, \quad (4.58)$$

terdapat empat konfigurasi basis yang harus dievaluasi, yaitu $|00\rangle$, $|01\rangle$, $|10\rangle$, dan $|11\rangle$. Dengan mensubstitusikan nilai spin yang bersesuaian ke dalam fungsi energi Ising, diperoleh

$$E(|00\rangle) \approx 1,0000, \quad (4.59)$$

$$E(|01\rangle) \approx -0,1890, \quad (4.60)$$

$$E(|10\rangle) \approx -0,3498, \quad (4.61)$$

$$E(|11\rangle) \approx 0,4667. \quad (4.62)$$

Hasil evaluasi tersebut menunjukkan bahwa konfigurasi $|10\rangle$ memiliki energi terendah sehingga ditetapkan sebagai *ground state* sistem. Solusi ini identik dengan hasil yang diperoleh melalui pencarian *Nash Equilibrium* maupun optimasi VQE pada jendela data yang sama. Kesesuaian ketiga pendekatan tersebut menunjukkan bahwa proses transformasi dari model portofolio klasik menuju Hamiltonian Ising serta prosedur optimasi kuantum yang diterapkan telah menghasilkan solusi yang konsisten.

4.7.2 Implementasi *Brute Force* Berbasis *Python*

Proses validasi solusi optimal dilakukan melalui eksplorasi menyeluruh terhadap seluruh konfigurasi *bitstring* yang mungkin pada Hamiltonian Ising hasil transformasi portofolio. Berbeda dengan algoritma VQE yang menelusuri lanskap energi secara variasional, metode *brute force* mengevaluasi energi setiap keadaan basis secara eksplisit sehingga solusi energi minimum global dapat diperoleh tanpa aproksimasi. Oleh karena itu, hasil yang diperoleh dari prosedur ini digunakan sebagai acuan pembandingan untuk mengukur tingkat akurasi solusi yang ditemukan oleh algoritma kuantum.

Implementasi numeriknya diawali dengan pendefinisian fungsi `calculate_ising_energy()`, yang bertugas menghitung energi suatu konfigurasi biner berdasarkan parameter Hamiltonian Ising (h_i, J_{ij}, C). Pada fungsi ini, setiap variabel biner portofolio x_i terlebih dahulu

dikonversi ke representasi spin Ising melalui transformasi $z_i = 1 - 2x_i$. Kemudian dihitung energi total yang terdiri dari penjumlahan kontribusi bias linear, interaksi pasangan spin, serta konstanta pergeseran energi. Selanjutnya, fungsi `brute_force_portfolio()` memanfaatkan pustaka `itertools.product` untuk membangkitkan seluruh konfigurasi *bitstring* sebanyak 2^N keadaan. Untuk setiap konfigurasi yang dihasilkan, program menghitung energi Ising yang bersesuaian dan membandingkannya dengan nilai minimum yang telah ditemukan sebelumnya. Mekanisme ini memungkinkan proses identifikasi *ground state* global dilakukan secara deterministik tanpa memerlukan prosedur optimasi iteratif. Selain mencari energi minimum global, algoritma juga mengevaluasi solusi yang memenuhi kendala kardinalitas portofolio $\sum_i x_i = K$. Hal tersebut berguna supaya hasil validasi juga memastikan solusi tetap konsisten dengan variabel batasan. Keseluruhan prosedur validasi numerik tersebut diimplementasikan menggunakan bahasa pemrograman *Python*. Struktur algoritma yang digunakan untuk menghitung energi setiap konfigurasi, mengeksplorasi seluruh ruang solusi, serta mengidentifikasi *ground state* portofolio ditunjukkan pada Daftar Kode 4.10.

```
import numpy as np
import pandas as pd
from itertools import product

def calculate_ising_energy(x, h, J, C_Ising):
    n_assets = len(x)
    Z = 1 - 2 * np.array(x)

    # Suku Bias: sum(h_i * Z_i)
    energy_bias = np.sum(h * Z)

    # Suku Interaksi: sum(J_ij * Z_i * Z_j)
    energy_interaction = 0
    for i in range(n_assets):
        for j in range(i + 1, n_assets):
            energy_interaction += J[i, j] * Z[i] * Z[j]

    return energy_bias + energy_interaction + C_Ising

def brute_force_portfolio(n_assets, h, J, C_Ising, K=2):
    best_global_e = float('inf')
    best_global_bs = None
    best_constrained_e = float('inf')
    best_constrained_bs = None

    # Eksplorasi 2^N
    for bit_tuple in product([0, 1], repeat=n_assets):
        x = np.array(bit_tuple)
        energy = calculate_ising_energy(x, h, J, C_Ising)
        bit_str = "".join(map(str, bit_tuple))

        if energy < best_global_e:
            best_global_e = energy
            best_global_bs = bit_str

        if sum(x) == K:
            if energy < best_constrained_e:
                best_constrained_e = energy
                best_constrained_bs = bit_str

    return best_constrained_bs, best_constrained_e
```

Kode 4.10: Skrip komputasi algoritma *brute force* untuk validasi lanskap energi portofolio pada arsitektur kuantum

4.8 Analisis Performa Portofolio dan Dinamika *Rebalancing*

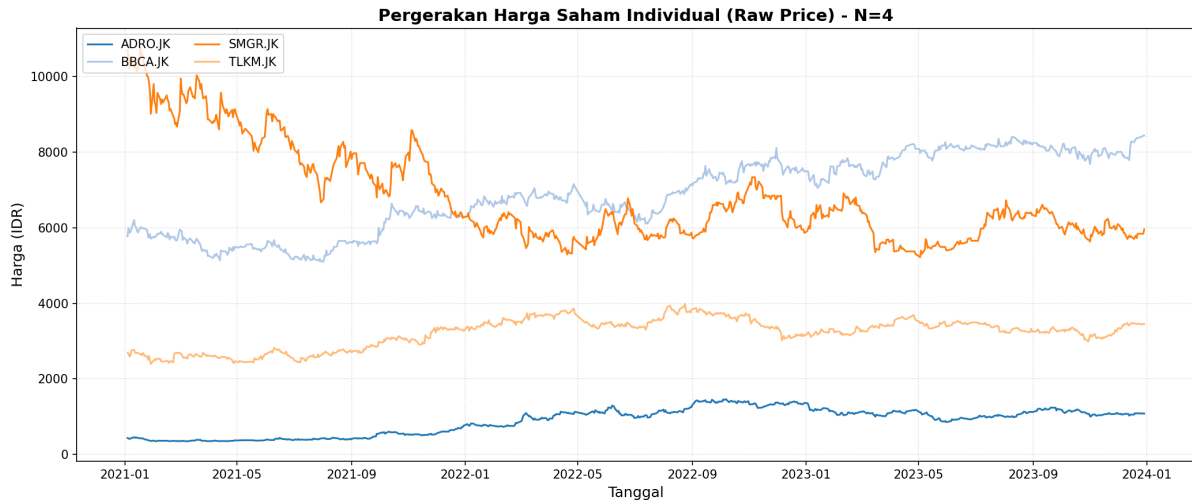
Evaluasi performa dilakukan melalui simulasi *backtesting* pada rentang waktu Januari 2021 hingga Desember 2023 dengan menerapkan mekanisme *rebalancing* bulanan. Pada setiap awal jendela observasi, parameter statistik pasar dihitung kembali menggunakan data historis pada periode *lookback*, kemudian dipetakan ke dalam fungsi Hamiltonian portofolio untuk menentukan konfigurasi aset optimal pada bulan berikutnya. Pendekatan ini memungkinkan sistem menyesuaikan komposisi portofolio secara dinamis terhadap perubahan karakteristik pasar yang terus berkembang. Untuk menilai efektivitas metode yang diusulkan, performa strategi GT-VQE dibandingkan dengan tiga pendekatan pembandingan, yaitu VQE murni tanpa integrasi teori permainan, optimasi klasik berbasis SLSQP berdasarkan model Markowitz, dan strategi pasif *Equal Weight* yang membeli dan menahan seluruh aset proporsi yang sama. Seluruh pengujian dilakukan menggunakan kombinasi saham BBKA, ADRO, TLKM, dan SMGR yang pergerakan harganya ditampilkan pada Gambar 4.2. Ringkasan hasil evaluasi kuantitatif untuk sistem $N = 2$ dan $N = 4$ kemudian disajikan pada Tabel 4.2. Berdasarkan hasil pengujian tersebut, strategi GT-VQE menunjukkan performa terbaik pada hampir seluruh metrik evaluasi yang digunakan. Pada sistem $N = 2$, strategi ini menghasilkan imbal hasil kumulatif sebesar 145,07%, melampaui VQE murni (121,30%), *Equal Weight* (97,88%), maupun optimasi klasik SLSQP (51,34%). Hal ini juga didukung dengan metrik *Sharpe Ratio* dengan rumus

$$\text{Sharpe Ratio} = \frac{R_p - R_f}{\sigma_p}, \quad (4.63)$$

dimana R_p adalah imbal hasil portofolio, R_f adalah imbal hasil bebas risiko (diasumsikan sebesar 4% per tahun), dan σ_p adalah deviasi standar imbal hasil portofolio. Nilai *Sharpe Ratio* yang dicapai strategi GT-VQE mencapai 1,0185, yang mengindikasikan bahwa setiap unit risiko yang ditanggung mampu dikonversi menjadi tingkat *return* yang lebih tinggi dibandingkan seluruh metode pembandingan. Walaupun nilai *Maximum Drawdown* yang dicatat mencapai 30,33%, tingkat *return* yang diperoleh tetap mampu mengompensasi risiko tersebut sehingga menghasilkan profil kinerja keseluruhan yang paling kompetitif.

Keunggulan metode hibrida semakin terlihat ketika dimensi masalah diperbesar menjadi sistem $N = 4$. Pada kondisi ini, performa optimasi klasik mengalami degradasi yang cukup signifikan dengan imbal hasil hanya sebesar 12,91%, sedangkan VQE murni menghasilkan *return* sebesar 16,89%. Sebaliknya, GT-VQE masih mampu mempertahankan imbal hasil sebesar 73,15% dengan *Sharpe Ratio* mencapai 1,0107. Hasil ini menunjukkan bahwa integrasi informasi strategis yang diperoleh dari teori permainan berperan penting dalam menjaga kualitas solusi ketika ukuran ruang pencarian meningkat secara eksponensial. Selain menghasilkan *return* yang lebih tinggi, strategi GT-VQE juga mampu mempertahankan nilai *Maximum Drawdown* sebesar 16,68%, lebih rendah dibandingkan VQE murni maupun strategi *Equal Weight*. Dengan demikian, peningkatan performa yang dicapai tidak hanya berasal dari peningkatan potensi keuntungan, tetapi juga dari kemampuan sistem dalam mengendalikan risiko selama proses investasi berlangsung. Jika dibandingkan dengan kinerja pasar secara keseluruhan yang direpresentasikan oleh IHSG dengan imbal hasil kumulatif sebesar 19,13% dan *Sharpe Ratio* 0,5557, seluruh pendekatan berbasis GT-VQE mampu menghasilkan keunggulan yang sangat signifikan. Temuan ini mengindikasikan bahwa kombinasi antara formulasi Hamiltonian portofolio, optimasi

kuantum variasional, serta inisialisasi berbasis *Nash Equilibrium* berhasil mengeksploitasi informasi struktural pasar yang tidak sepenuhnya dapat ditangkap oleh pendekatan optimasi konvensional. Oleh karena itu, hasil pada Tabel 4.2 menjadi indikasi awal bahwa integrasi teori permainan dan komputasi kuantum memberikan kontribusi nyata terhadap peningkatan kualitas keputusan alokasi aset pada berbagai skala kompleksitas sistem.



Gambar 4.2: Pergerakan Harga Saham Individual (*Raw Price*) pada Sistem $N=4$ (2021–2023).

Tabel 4.2: Ringkasan Performa Finansial Strategi Kuantum, SLSQP, dan *Equal Weight* ($N = 2, 4$).

Strategi	Sistem $N=2$			Sistem $N=4$		
	Return	Sharpe	MDD	Return	Sharpe	MDD
GT-VQE (Hibrida)	145,07%	1,0185	30,33%	73,15%	1,0107	16,68%
VQE (Tanpa GT)	121,30%	0,9331	30,33%	16,89%	0,3592	20,89%
SLSQP (Klasik)	51,34%	0,6736	15,91%	12,91%	0,2810	21,12%
<i>Equal Weight</i>	97,88%	0,9764	27,49%	35,57%	0,6790	18,42%

Keunggulan performa GT-VQE yang ditunjukkan pada Tabel 4.2 sesuai karakteristik dinamika kuantum yang telah diamati pada tahapan optimasi sebelumnya. Pada setiap jendela *rebalancing*, proses pencarian portofolio dilakukan dengan memetakan fungsi objektif investasi ke dalam Hamiltonian Ising, kemudian menavigasi lanskap energinya menggunakan algoritma VQE berbasis SPSA. Dalam kerangka tersebut, kualitas keputusan investasi sangat bergantung pada kemampuan sirkuit variasional untuk menemukan *ground state* Hamiltonian yang merepresentasikan konfigurasi aset dengan utilitas optimal. Semakin dekat energi ekspektasi yang dicapai terhadap energi minimum global hasil validasi *brute force*, semakin besar probabilitas bahwa portofolio yang dipilih merepresentasikan solusi optimum dari permasalahan optimasi yang sedang dihadapi. Hubungan antara kualitas solusi tersebut dan dinamika internal sirkuit kuantum dapat diamati melalui evolusi entropi *Von Neumann* yang telah dibahas pada Section 4.6. Pada jendela-jendela simulasi yang berhasil mencapai solusi optimal, distribusi probabilitas portofolio secara bertahap mengalami pemusatan pada satu *bitstring* dominan sehingga nilai entropi menurun menuju kondisi yang relatif rendah. Fenomena ini menunjukkan bahwa parameter

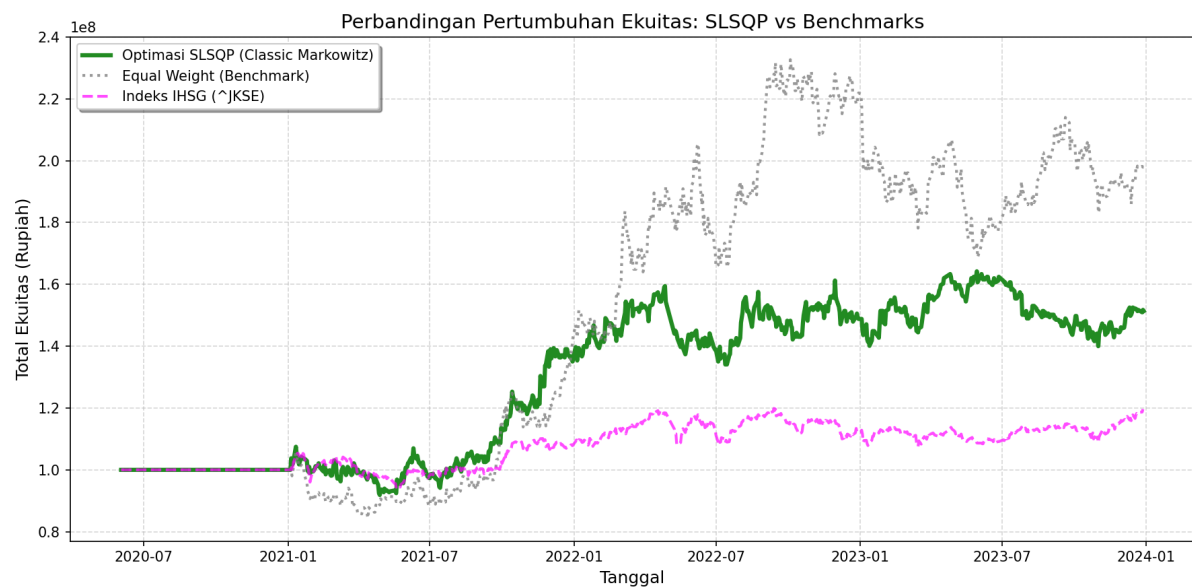
rotasi θ berhasil mengarahkan sistem keluar dari superposisi yang terlalu menyebar menuju keadaan kuantum yang lebih terlokalisasi. Sebaliknya, ketika distribusi probabilitas tetap tersebar pada banyak konfigurasi portofolio sekaligus, entropi cenderung bertahan pada nilai yang tinggi dan kualitas solusi yang diperoleh menjadi kurang optimal.

Interpretasi tersebut juga sejalan dengan analisis fenomena *barren plateau*. Pada beberapa jendela observasi ditemukan kondisi ketika nilai entropi gagal mengalami penurunan yang berarti dan bertahan pada tingkat yang relatif tinggi sepanjang proses optimasi. Hal ini mengindikasikan bahwa lanskap energi yang dieksplorasi memiliki gradien yang sangat kecil sehingga pembaruan parameter SPSA tidak mampu menghasilkan pergeseran amplitudo probabilitas yang signifikan. Akibatnya, sirkuit kesulitan membedakan konfigurasi portofolio yang benar-benar optimal dari konfigurasi lain yang memiliki energi serupa. Sebaliknya, jendela-jendela yang menunjukkan penurunan entropi secara konsisten umumnya juga menghasilkan konvergensi energi yang lebih cepat serta probabilitas solusi optimal yang lebih tinggi. Hubungan ini memperlihatkan bahwa entropi *Von Neumann* tidak hanya berfungsi sebagai ukuran keterbelitan dan ketidakpastian kuantum, tetapi juga dapat digunakan sebagai indikator tambahan untuk mendeteksi keberhasilan proses optimasi VQE dalam menghindari hambatan *barren plateau*. Dalam konteks tersebut, keberhasilan GT-VQE mempertahankan performa pada sistem $N = 4$ mengindikasikan bahwa mekanisme *warm-start* berbasis *Nash Equilibrium* mampu memberikan titik awal parameter yang lebih dekat terhadap wilayah solusi berkualitas tinggi. Dampaknya, ruang eksplorasi yang harus ditelusuri oleh SPSA menjadi lebih sempit sehingga probabilitas terjebak pada daerah gradien rendah dapat dikurangi. Efek ini menjelaskan mengapa GT-VQE tetap mampu menghasilkan *Sharpe Ratio* yang tinggi dan imbal hasil yang stabil ketika ukuran ruang solusi meningkat secara eksponensial, sedangkan VQE murni dan optimasi klasik mulai mengalami penurunan performa yang cukup signifikan. Dengan kata lain, peningkatan performa finansial yang diamati pada tahap *backtesting* merupakan konsekuensi langsung dari peningkatan kualitas proses optimasi kuantum yang terjadi pada tingkat fundamental sirkuit variasional.

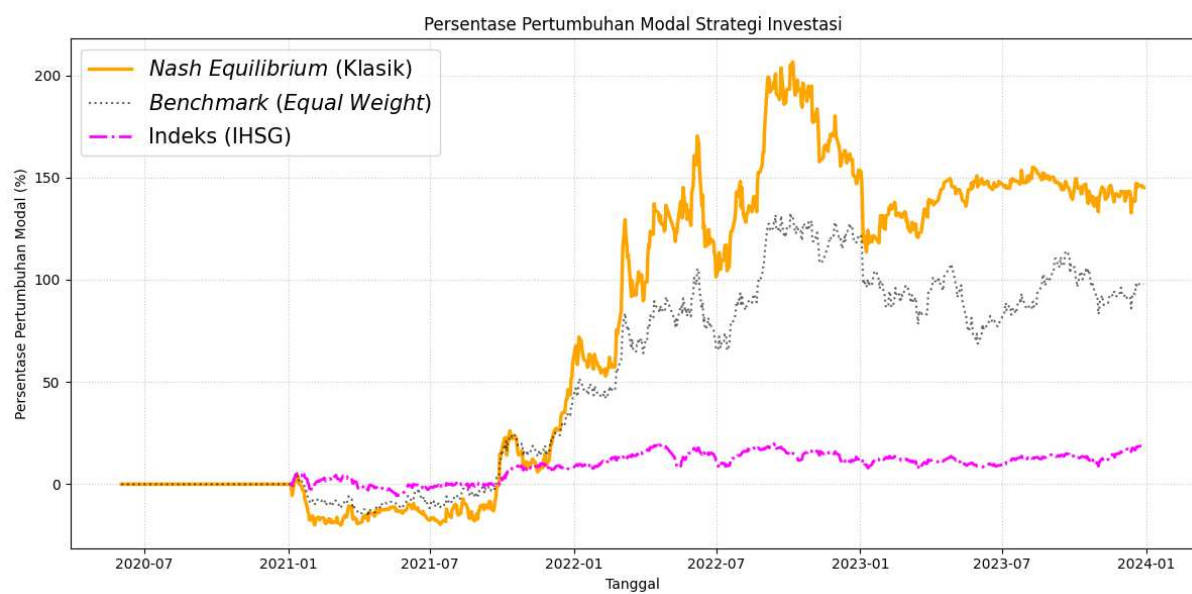
Perbandingan performa antara strategi kuantum hibrida, kuantum murni, dan klasik untuk sistem $N = 2$ dapat ditinjau pada Gambar 4.3 hingga 4.6, sedangkan untuk perbandingan ekstensif pada sistem $N = 4$ disajikan pada Gambar 4.7 hingga 4.10.

Analisis visual pada Gambar 4.6 dan Gambar 4.10 memperlihatkan bahwa strategi GT-VQE menghasilkan kurva pertumbuhan modal yang lebih stabil dibandingkan metode pembanding. Pada sistem $N = 2$, baik VQE murni maupun GT-VQE mampu menghasilkan pertumbuhan portofolio yang melampaui strategi klasik. Namun, ketika jumlah aset diperbesar menjadi $N = 4$, perbedaan karakteristik kedua pendekatan tersebut mulai terlihat secara jelas. VQE murni mengalami penurunan performa yang cukup drastis, sedangkan GT-VQE tetap mempertahankan laju pertumbuhan modal yang konsisten hingga akhir periode pengujian. Fenomena ini menunjukkan bahwa penambahan dimensi ruang solusi memberikan tantangan optimasi yang semakin besar bagi algoritma variasional, sehingga kualitas inisialisasi parameter menjadi faktor yang semakin menentukan keberhasilan pencarian solusi.

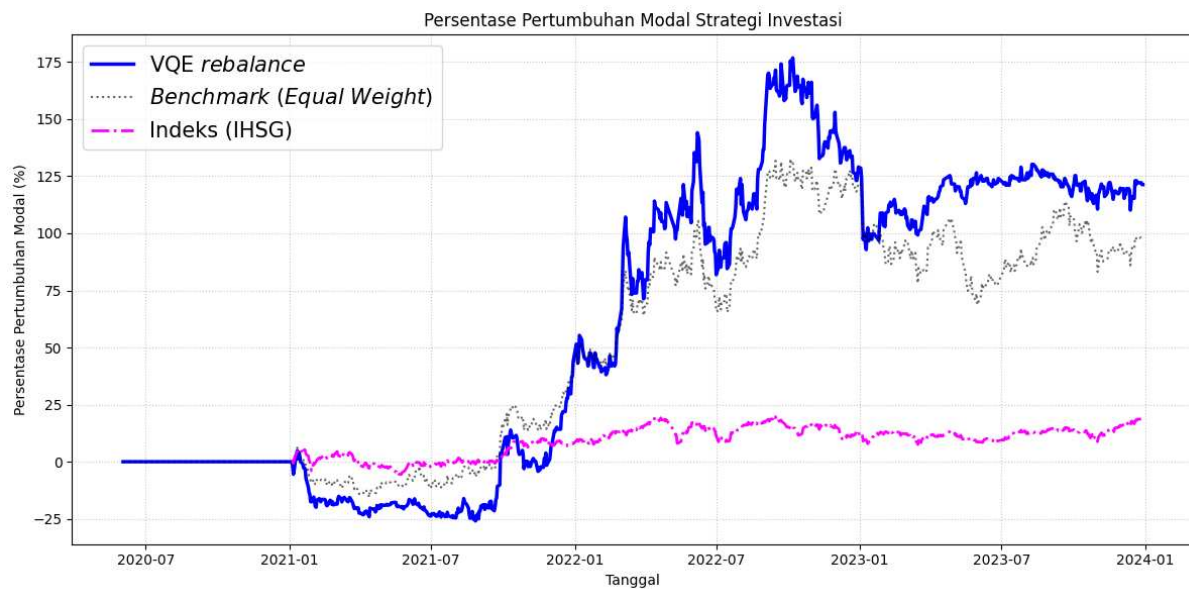
Efektivitas pendekatan *warm-start* juga dapat diamati melalui dinamika adaptasi kedalaman sirkuit pada Gambar 4.11. Untuk sistem tanpa integrasi teori permainan, algoritma VQE cenderung membutuhkan kedalaman sirkuit yang lebih tinggi dan lebih fluktuatif dari satu jendela observasi ke jendela berikutnya. Kondisi ini menunjukkan bahwa sistem harus mengeksplorasi ruang parameter yang lebih luas untuk mencapai



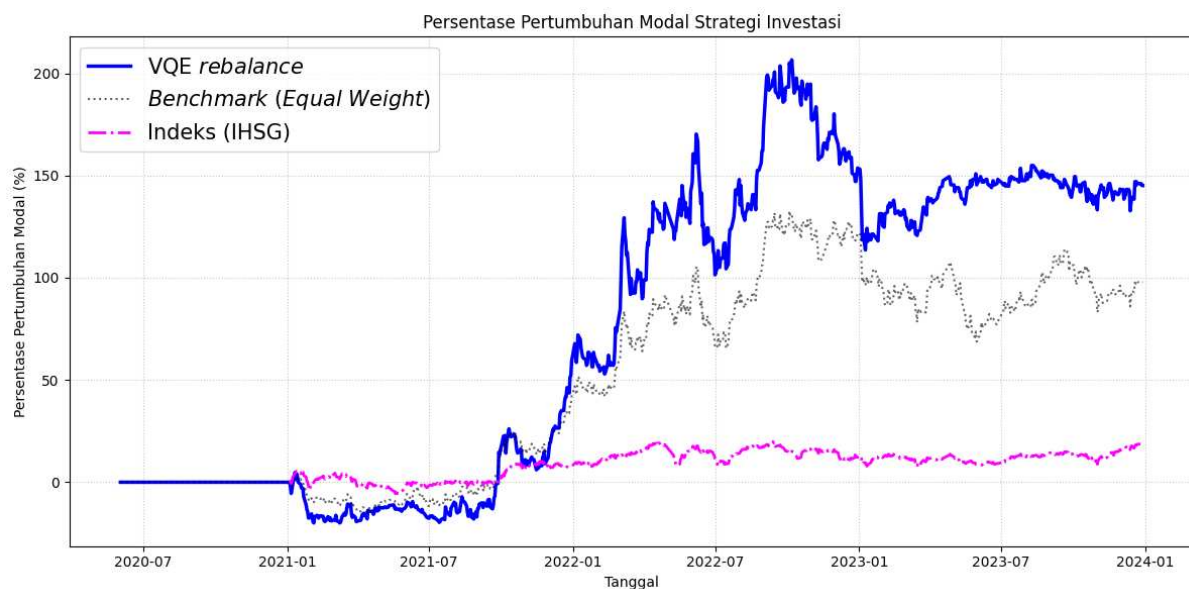
Gambar 4.3: Visualisasi Pertumbuhan Modal Optimasi Klasik SLSQP pada Sistem $N=2$.



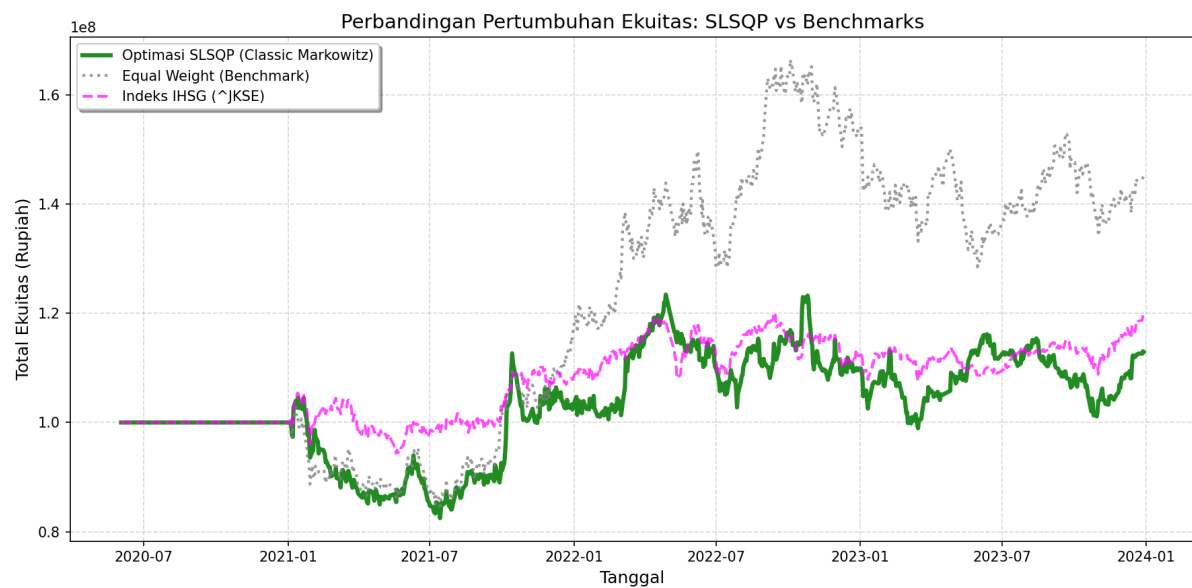
Gambar 4.4: Visualisasi Pertumbuhan Modal Strategi *Nash* (Klasik) pada Sistem $N=2$.



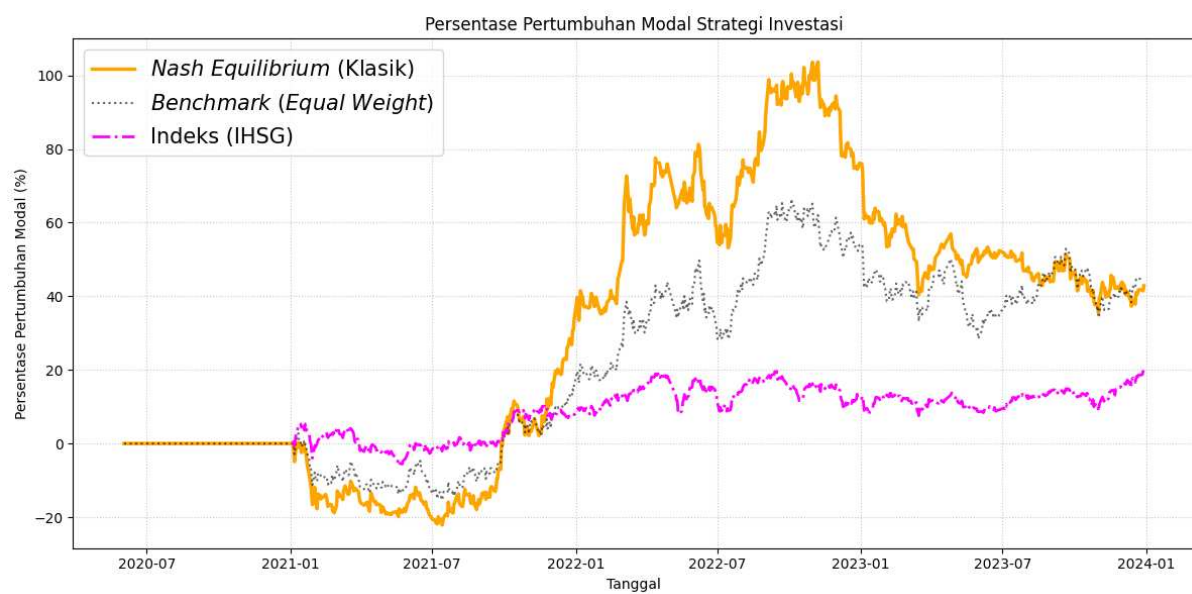
Gambar 4.5: Visualisasi Pertumbuhan Modal VQE Murni (Tanpa GT) pada Sistem $N=2$.



Gambar 4.6: Visualisasi Pertumbuhan Modal Strategi GT-VQE (Hibrida) pada Sistem $N=2$.



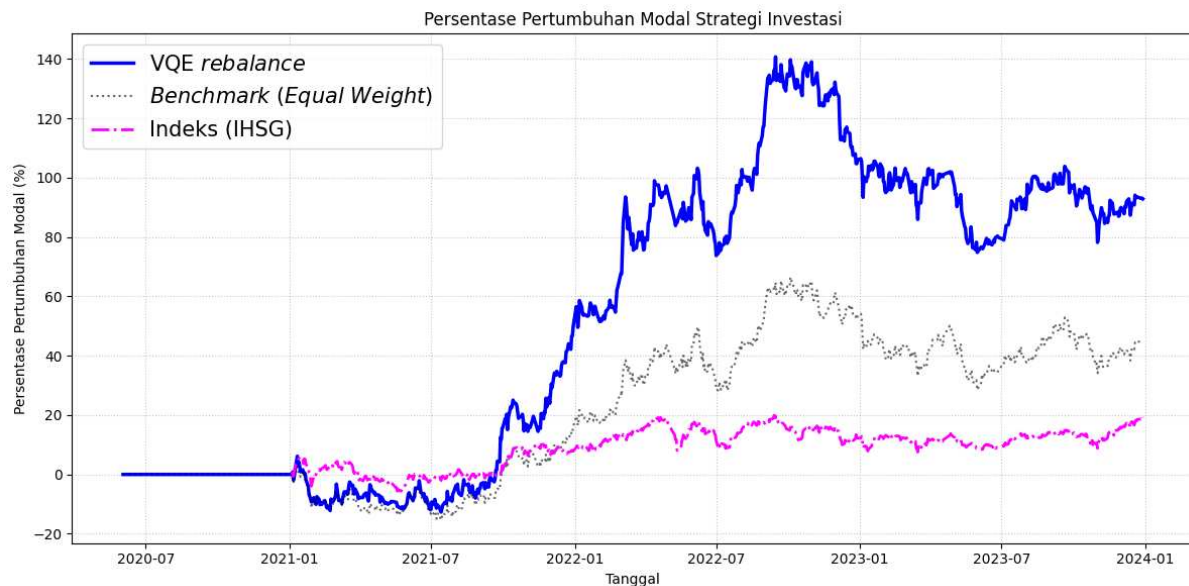
Gambar 4.7: Visualisasi Pertumbuhan Modal Optimasi Klasik SLSQP pada Sistem $N=4$.



Gambar 4.8: Visualisasi Pertumbuhan Modal Strategi *Nash* (Klasik) pada Sistem $N=4$.

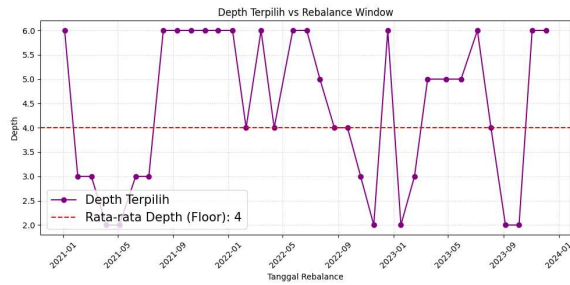


Gambar 4.9: Visualisasi Pertumbuhan Modal VQE Murni (Tanpa GT) pada Sistem $N=4$.

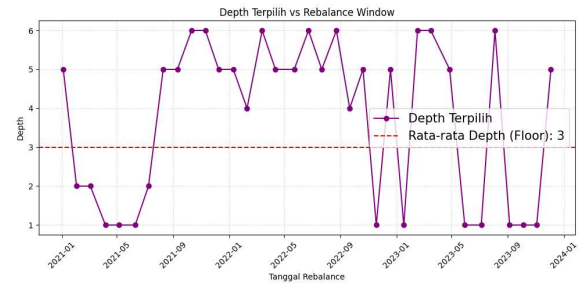


Gambar 4.10: Visualisasi Pertumbuhan Modal Strategi GT-VQE (Hibrida) pada Sistem $N=4$.

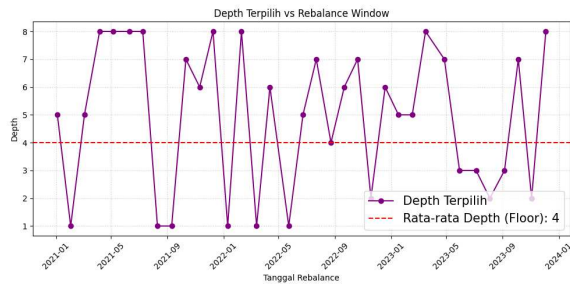
energi minimum yang memadai. Sebaliknya, GT-VQE secara dominan mampu beroperasi pada kedalaman rendah, terutama pada *Depth* 2, baik untuk sistem $N = 2$ maupun $N = 4$. Temuan ini memiliki implikasi penting karena peningkatan kedalaman sirkuit umumnya memperbesar risiko munculnya *barren plateau*, meningkatkan akumulasi galat numerik, serta memperpanjang waktu komputasi. Oleh sebab itu, kemampuan GT-VQE mencapai solusi berkualitas tinggi pada kedalaman yang relatif dangkal menunjukkan adanya peningkatan efisiensi optimasi yang signifikan.



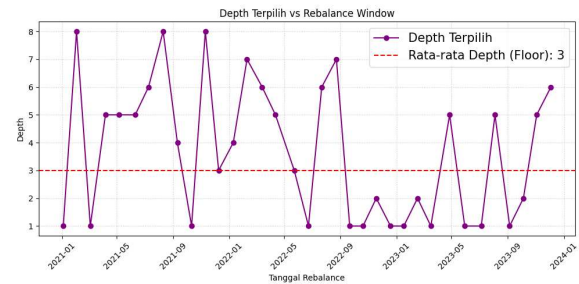
(a) VQE Murni ($N = 2$)



(b) GT-VQE Hibrida ($N = 2$)



(c) VQE Murni ($N = 4$)



(d) GT-VQE Hibrida ($N = 4$)

Gambar 4.11: Perbandingan Dinamika Adaptasi Kedalaman Sirkuit (*Depth*) per Jendela Simulasi antara VQE Murni dan GT-VQE untuk Berbagai Konfigurasi Aset.

Secara keseluruhan, hasil *backtesting* memperlihatkan bahwa integrasi teori permainan, optimasi variasional kuantum, dan mekanisme *rebalancing* adaptif membentuk suatu kerangka pengambilan keputusan investasi yang saling melengkapi. Informasi strategis yang diperoleh dari *Nash Equilibrium* berperan mempersempit ruang pencarian awal, SPSA bertugas mengarahkan parameter menuju energi minimum secara efisien, sedangkan evaluasi entropi memberikan indikator tambahan terhadap kualitas konvergensi yang dicapai. Kombinasi ketiga komponen tersebut menjelaskan mengapa GT-VQE mampu mempertahankan performa yang lebih baik dibandingkan pendekatan kuantum murni maupun optimasi klasik ketika kompleksitas sistem terus meningkat. Proses evaluasi performa dan simulasi *backtesting* ini dikendalikan oleh skrip utama yang melakukan iterasi jendela waktu dan mengeksekusi penyesuaian bobot portofolio. Skrip operasional *backtesting* dapat dilihat pada Daftar Kode 4.11, sedangkan mekanisme penyesuaian bobot (*rebalancing*) diimplementasikan pada Daftar Kode 4.12.

```
import numpy as np
import pandas as pd
from run_strategy_step import run_strategy_step
from rebalance_portfolio import rebalance_portfolio

def run_backtest(data_clean, tickers, config):
```

```

"""
Menjalankan simulasi backtest berdasarkan data historis dan konfigurasi.
"""
N = len(tickers)
K = config['K']
penalty_A = config['penalty_A']
max_depth = config['max_depth']
maxiter = config['maxiter']
max_total_iter = config.get('max_total_iter', 500)
batch_size = config.get('batch_size', 10)
conv_window = config.get('conv_window', 4)
conv_tol = config.get('conv_tol', 1e-3)
use_warm_start = config.get('use_warm_start', True)
initial_capital = config['initial_capital']
lookback_days = config['lookback_days']
rebalance_days = config['rebalance_days']
start_bt_date = pd.to_datetime(config['start_bt_date'])

start_idx = np.searchsorted(data_clean.index, start_bt_date)
rebalance_indices = range(start_idx, len(data_clean), rebalance_days)

value_vqe = [initial_capital] * start_idx
value_bench = [initial_capital] * start_idx
value_nash = [initial_capital] * start_idx
value_assets = {t: [initial_capital] * start_idx for t in tickers}

holdings_vqe, holdings_bench, holdings_nash = np.zeros(N), np.zeros(N), np.zeros(N)
cash_vqe, cash_bench, cash_nash = initial_capital, initial_capital, initial_capital
cash_assets = {t: initial_capital for t in tickers}
holdings_assets = {t: 0.0 for t in tickers}

detail_logs = []
depths_history = []
rebalance_dates = []
window_analysis_history = [] # Menyimpan semua data untuk plotting per window

print(f"\n--- Memulai Backtest dari {data_clean.index[start_idx].date()} hingga {
data_clean.index[-1].date()} ---")

for i, curr_idx in enumerate(rebalance_indices):
    curr_date = data_clean.index[curr_idx]
    train_start = max(0, curr_idx - lookback_days)
    train_data = data_clean.iloc[train_start:curr_idx]
    next_idx = (rebalance_indices[i + 1] if i + 1 < len(rebalance_indices) else len(
data_clean))

    # Strategi Step
    selected_indices, depth_used, energy_final, best_history, best_ent_hist,
    best_gvar_hist, depth_energies, lr_data, ne_bs, ne_utility, winning_probs, best_params =
    run_strategy_step(
        train_data, tickers, curr_date, K=K, penalty_A=penalty_A, max_depth=max_depth,
        maxiter=maxiter,
        max_total_iter=max_total_iter, batch_size=batch_size,
        conv_window=conv_window, conv_tol=conv_tol,
        use_warm_start=use_warm_start,
        file_config=config.get('files')
    )

    depths_history.append(depth_used)
    rebalance_dates.append(curr_date.date())

# Simpan semua detail untuk plotting nanti
window_analysis_history.append({
    'date': curr_date.date(),
    'best_history': best_history,
    'best_ent_hist': best_ent_hist,
    'best_gvar_hist': best_gvar_hist,
    'depth_energies': depth_energies,
    'lr_data': lr_data,
    'winning_probs': winning_probs,
    'ne_bs': ne_bs,
    'best_params': best_params,

```



```

        'best_depth': depth_used,
        'K': K,
        'use_warm_start': use_warm_start
    })

    selected_names = [tickers[idx] for idx in selected_indices]
    vqe_details = "".join([f"      [Depth {d}] Konvergen dalam {iters} iterasi | E = {en:.6f}
\n" for d, en, iters in depth_energies])

    print(f"[{curr_date.date()}] VQE Terpilih: {selected_names} | Depth: {depth_used} |
E_min: {energy_final:.6f}")

    log_entry = (f"[{curr_date.date()}]\n"
                 f"  - Nash Eq: {ne_bs} | Utility: {ne_utility:.6f}\n"
                 f"  - LR: {lr_data[2]:.4f}\n"
                 f"  - Detail:\n{vqe_details}"
                 f"  - Terpilih: {selected_names} | Depth: {depth_used} | E_min: {
energy_final:.6f}\n" + "-"*40)
    detail_logs.append(log_entry)

    # Rebalancing VQE
    target_w_vqe = np.zeros(N)
    if len(selected_indices) > 0:
        w = 1.0 / len(selected_indices)
        for idx in selected_indices: target_w_vqe[idx] = w

    # Rebalancing Nash
    target_w_nash = np.zeros(N)
    nash_indices = [j for j, bit in enumerate(ne_bs) if bit == '1']
    if len(nash_indices) > 0:
        w_n = 1.0 / len(nash_indices)
        for idx in nash_indices: target_w_nash[idx] = w_n

    target_w_bench = np.full(N, 1.0 / N)
    current_prices = data_clean.iloc[curr_idx].values

    cash_vqe, holdings_vqe = rebalance_portfolio(cash_vqe, holdings_vqe, target_w_vqe,
current_prices, N)
    cash_nash, holdings_nash = rebalance_portfolio(cash_nash, holdings_nash, target_w_nash
, current_prices, N)

    if i == 0:
        cash_bench, holdings_bench = rebalance_portfolio(cash_bench, holdings_bench,
target_w_bench, current_prices, N)
        for j, t in enumerate(tickers):
            target_w_indiv = np.zeros(N)
            target_w_indiv[j] = 1.0
            c_t, h_t = rebalance_portfolio(cash_assets[t], np.zeros(N), target_w_indiv,
current_prices, N)
            cash_assets[t], holdings_assets[t] = c_t, h_t[j]

        for d in range(curr_idx, next_idx):
            prices = data_clean.iloc[d].values
            value_vqe.append(cash_vqe + np.sum(holdings_vqe * prices))
            value_nash.append(cash_nash + np.sum(holdings_nash * prices))
            value_bench.append(cash_bench + np.sum(holdings_bench * prices))
            for j, t in enumerate(tickers):
                value_assets[t].append(cash_assets[t] + holdings_assets[t] * prices[j])

    # --- EKSPOR HASIL EKUITAS KE CSV (Untuk Perbandingan) ---
    suffix = f"_N{N}"
    equity_df = pd.DataFrame({
        'Date': data_clean.index[:len(value_vqe)],
        'VQE': value_vqe,
        'Nash': value_nash,
        'Benchmark': value_bench
    })

    # Gunakan folder files config jika ada, atau default suffix
    equity_csv_path = config.get('files', {}).get('equity_history', f'hasil_ekuitas_backtest{
suffix}.csv')
    equity_df.to_csv(equity_csv_path, index=False)

```

```

return {
    'value_vqe': value_vqe,
    'value_nash': value_nash,
    'value_bench': value_bench,
    'value_assets': value_assets,
    'depths_history': depths_history,
    'rebalance_dates': rebalance_dates,
    'detail_logs': detail_logs,
    'window_analysis_history': window_analysis_history,
    'start_idx': start_idx
}

```

Kode 4.11: Skrip komputasional untuk eksekusi simulasi *backtesting* dengan iterasi jendela bergulir (`backtest_runner.py`)

```

import numpy as np

def rebalance_portfolio(current_cash, current_holdings, target_weights, prices, N):
    """
    Menyeimbangkan portofolio berdasarkan bobot target.
    N: Jumlah aset
    """
    total_value = current_cash + np.sum(current_holdings * prices)
    target_values = total_value * target_weights
    new_holdings = current_holdings.copy()
    new_cash = current_cash

    for j in range(N):
        c_val = new_holdings[j] * prices[j]
        if c_val > target_values[j]:
            sell_val = c_val - target_values[j]
            new_cash += sell_val
            new_holdings[j] -= sell_val / prices[j]

    for j in range(N):
        c_val = new_holdings[j] * prices[j]
        if c_val < target_values[j]:
            buy_val = min(target_values[j] - c_val, new_cash)
            new_cash -= buy_val
            new_holdings[j] += buy_val / prices[j]

    return new_cash, new_holdings

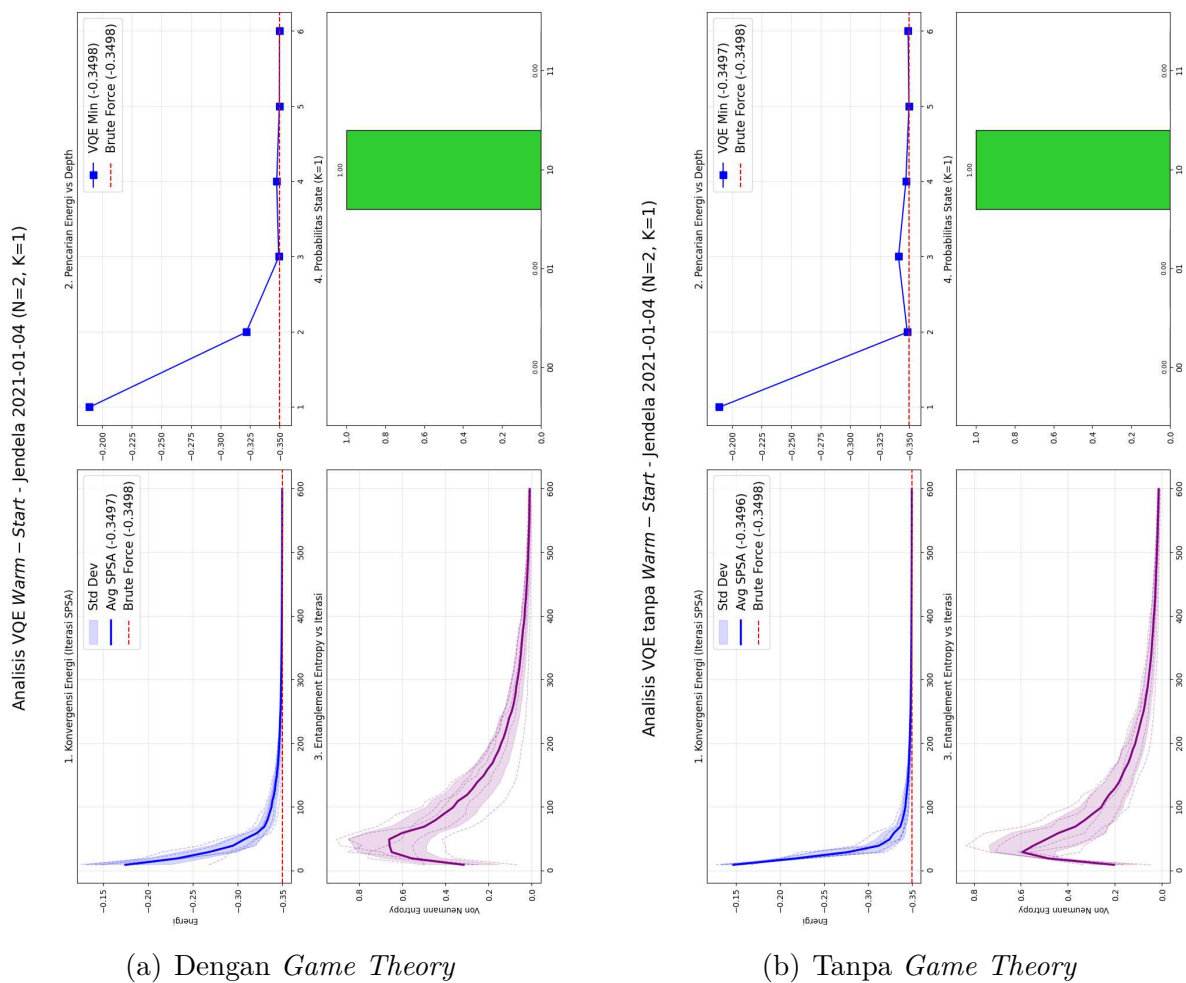
```

Kode 4.12: Fungsi operasional untuk penyesuaian bobot portofolio pada momen evaluasi (`rebalance_portfolio.py`)

Lampiran G

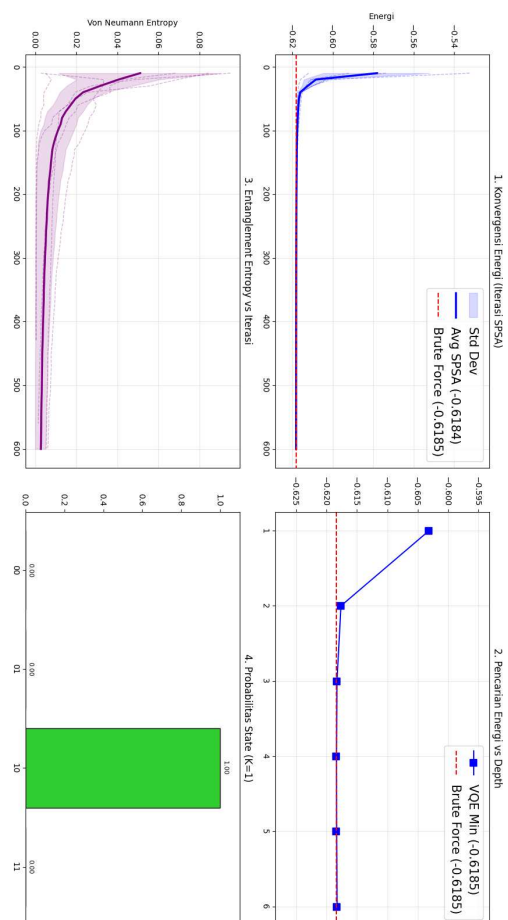
Visualisasi Perbandingan Alokasi Portofolio N=2 (GT vs No-GT)

Lampiran ini menampilkan grafik alokasi aset pada setiap jendela waktu untuk sistem dengan $N = 2$ aset, membandingkan pengaruh model *Game Theory* terhadap keputusan investasi.



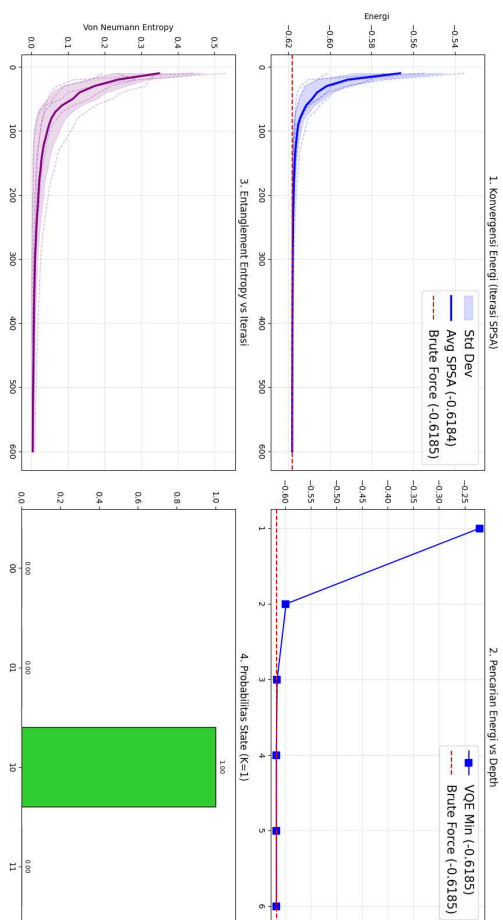
Gambar G.1: Perbandingan Alokasi Portofolio N=2 Jendela Waktu 2021-01-04

Analisis VQE tanpa Warm – Start - Jendela 2022-02-09 (N=2, K=1)



(b) Tanpa *Game Theory*

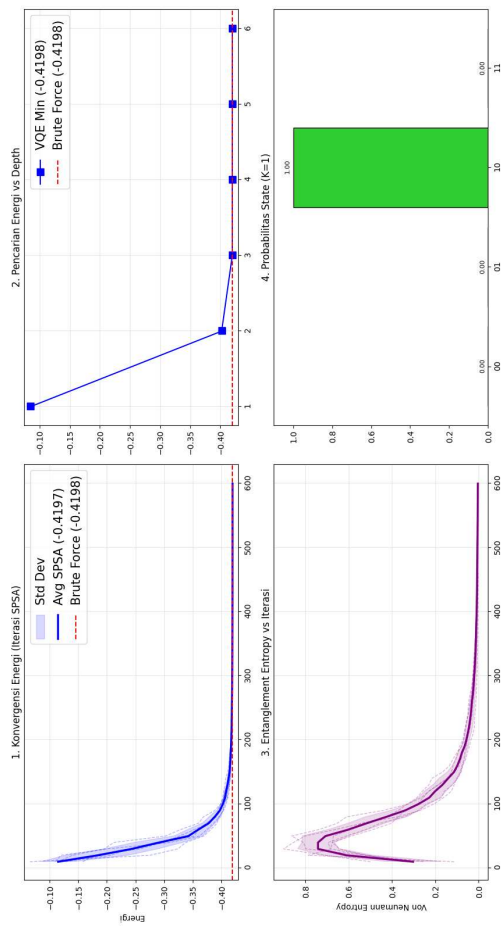
Analisis VQE Warm – Start - Jendela 2022-02-09 (N=2, K=1)



(a) Dengan *Game Theory*

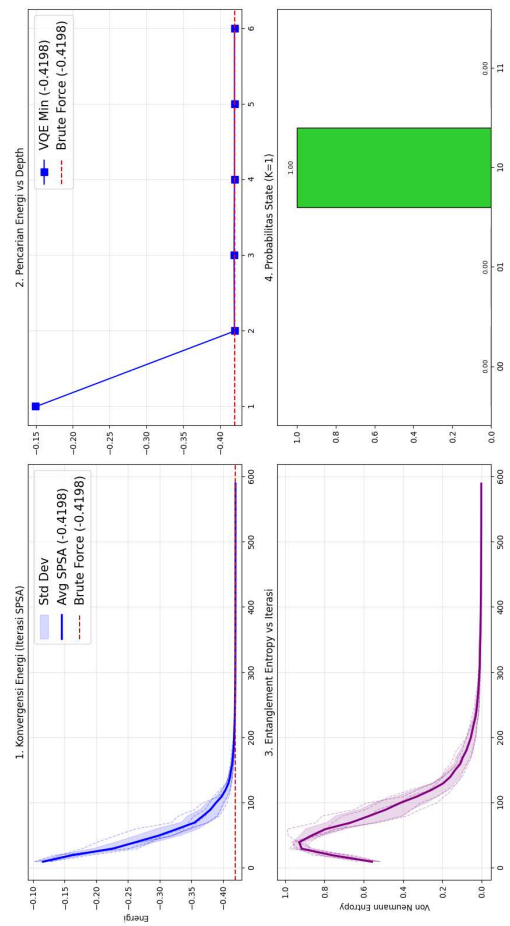
Gambar G.2: Perbandingan Alokasi Portofolio N=2 Jendela Waktu 2022-02-09

Analisis VQE Warm – Start - Jendela 2022-09-21 (N=2, K=1)



(a) Dengan *Game Theory*

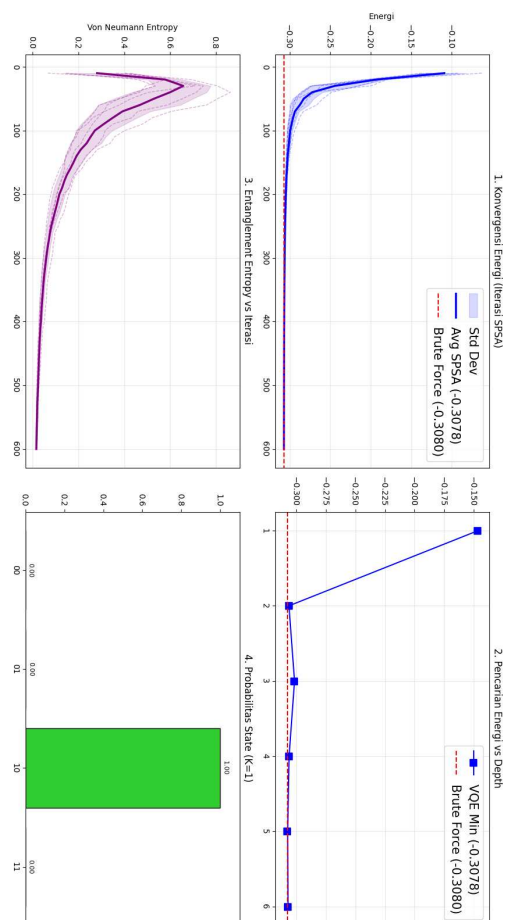
Analisis VQE tanpa Warm – Start - Jendela 2022-09-21 (N=2, K=1)



(b) Tanpa *Game Theory*

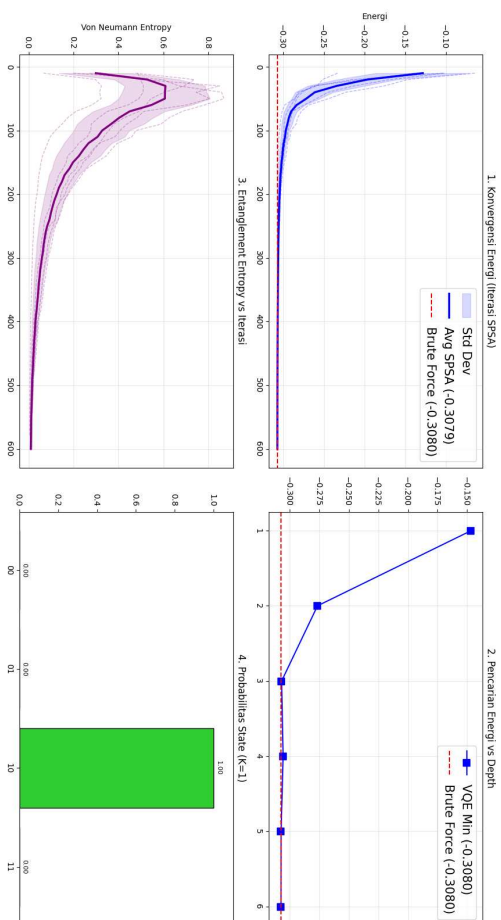
Gambar G.3: Perbandingan Alokasi Portofolio N=2 Jendela Waktu 2022-09-21

Analisis VQE tanpa Warm – Start - Jendela 2022-12-19 (N=2, K=1)



(b) Tanpa *Game Theory*

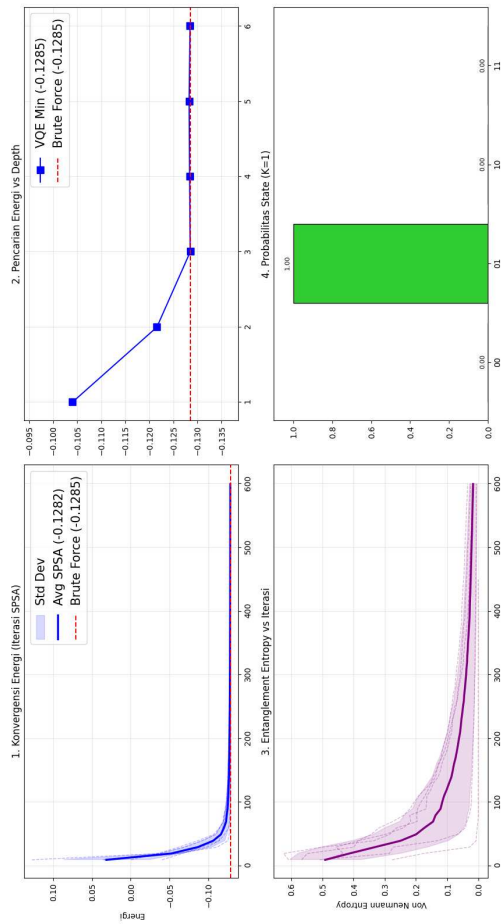
Analisis VQE Warm – Start - Jendela 2022-12-19 (N=2, K=1)



(a) Dengan *Game Theory*

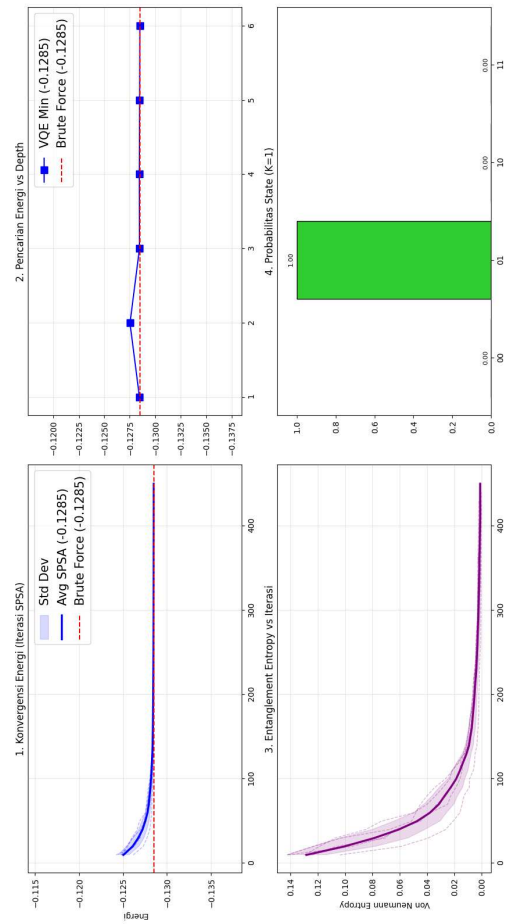
Gambar G.4: Perbandingan Alokasi Portofolio N=2 Jendela Waktu 2022-12-19

Analisis VQE Warm – Start - Jendela 2023-02-16 (N=2, K=1)



(a) Dengan *Game Theory*

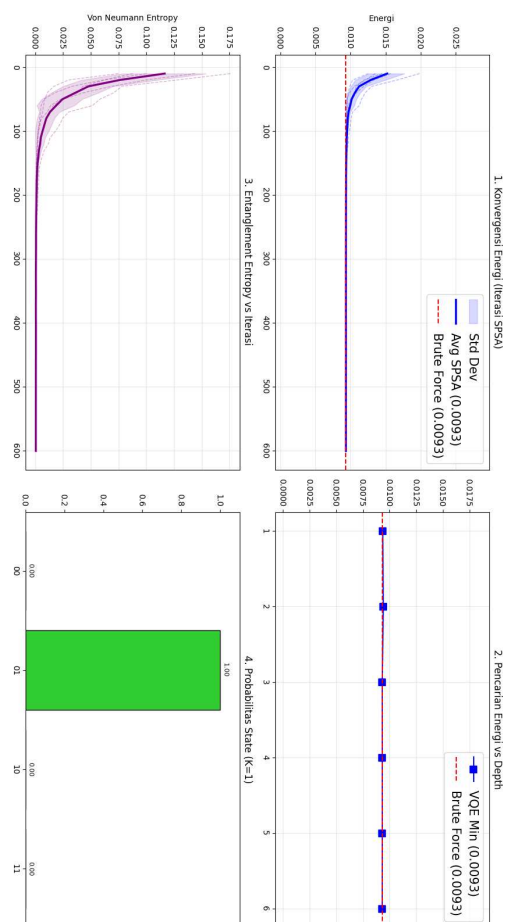
Analisis VQE tanpa Warm – Start - Jendela 2023-02-16 (N=2, K=1)



(b) Tanpa *Game Theory*

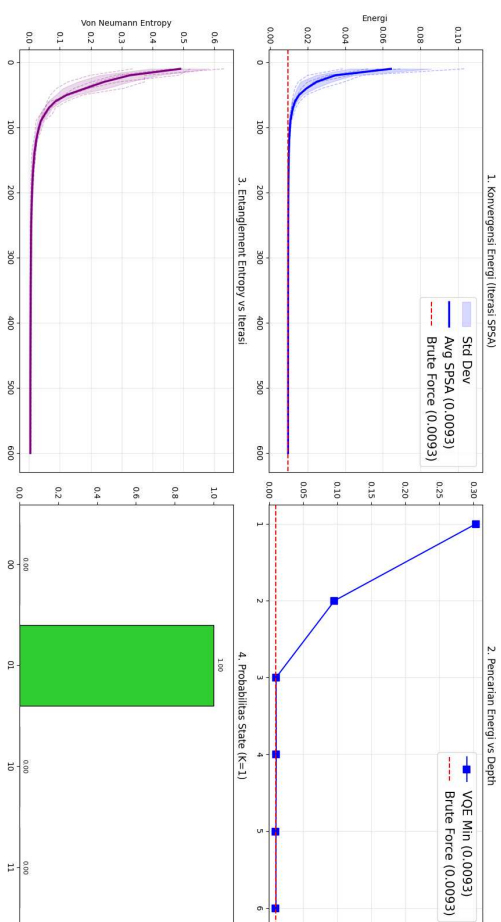
Gambar G.5: Perbandingan Alokasi Portofolio N=2 Jendela Waktu 2023-02-16

Analisis VQE tanpa Warm – Start - Jendela 2023-03-17 (N=2, K=1)



(b) Tanpa *Game Theory*

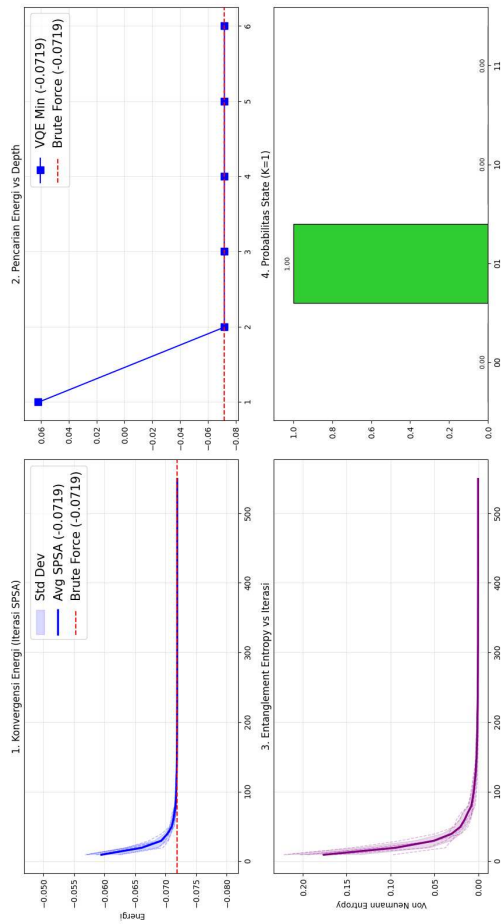
Analisis VQE Warm – Start - Jendela 2023-03-17 (N=2, K=1)



(a) Dengan *Game Theory*

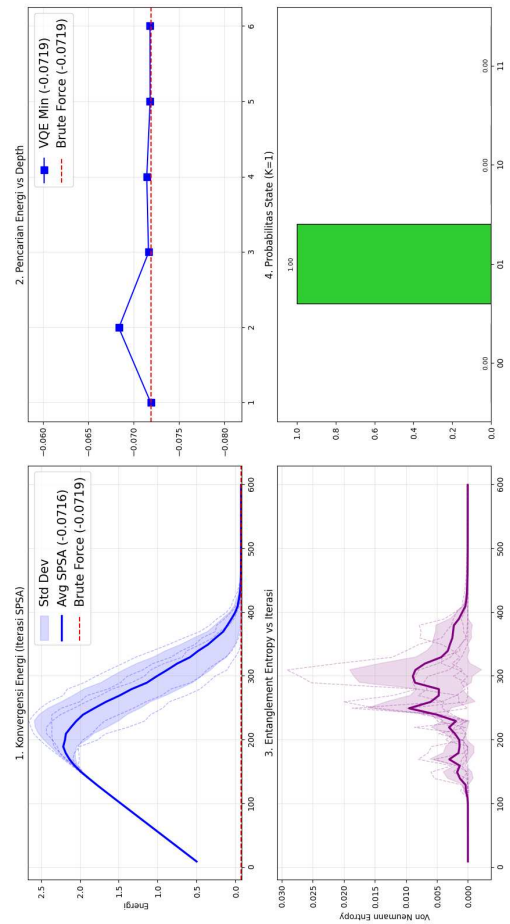
Gambar G.6: Perbandingan Alokasi Portofolio N=2 Jendela Waktu 2023-03-17

Analisis VQE Warm – Start - Jendela 2023-07-05 (N=2, K=1)



(a) Dengan *Game Theory*

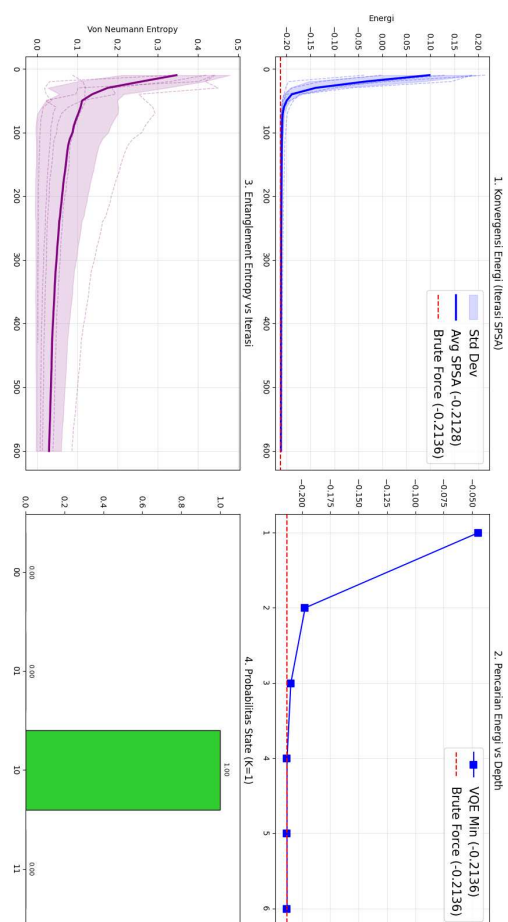
Analisis VQE tanpa Warm – Start - Jendela 2023-07-05 (N=2, K=1)



(b) Tanpa *Game Theory*

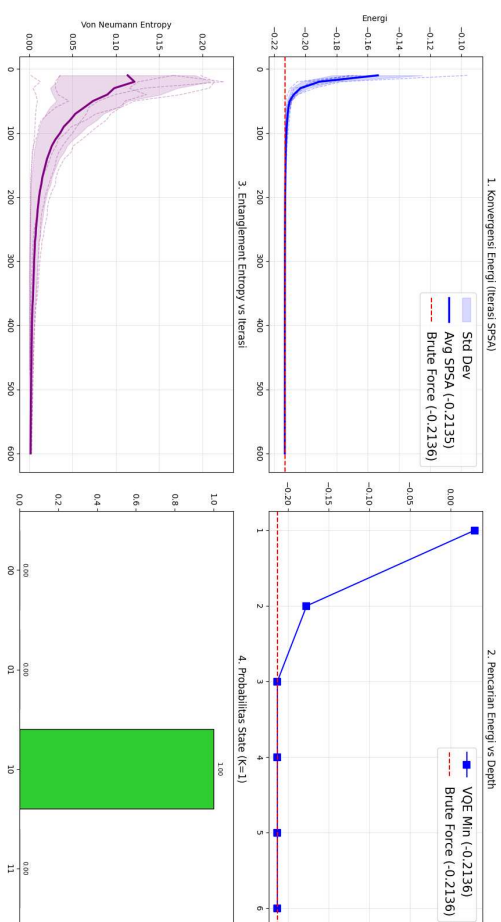
Gambar G.7: Perbandingan Alokasi Portofolio N=2 Jendela Waktu 2023-07-05

Analisis VQE tanpa Warm – Start - Jendela 2023-12-04 (N=2, K=1)



(b) Tanpa *Game Theory*

Analisis VQE Warm – Start - Jendela 2023-12-04 (N=2, K=1)



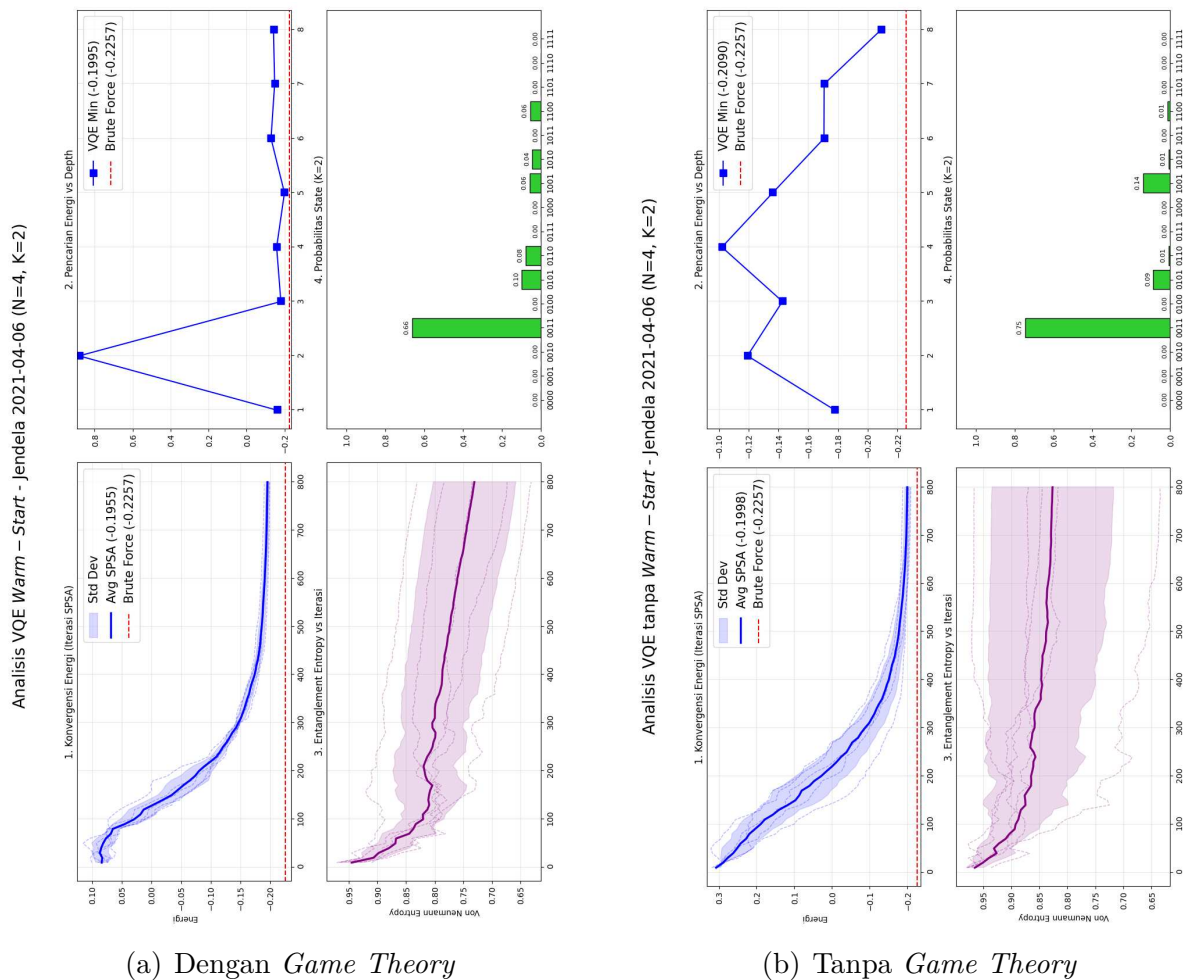
(a) Dengan *Game Theory*

Gambar G.8: Perbandingan Alokasi Portofolio N=2 Jendela Waktu 2023-12-04

Lampiran I

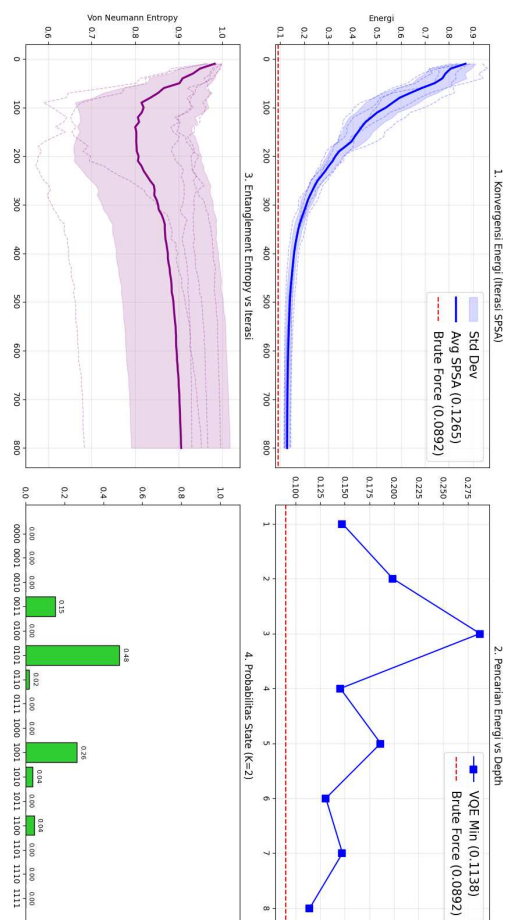
Visualisasi Perbandingan Alokasi Portofolio N=4 (GT vs No-GT)

Lampiran ini menampilkan grafik alokasi aset pada setiap jendela waktu untuk sistem dengan $N = 4$ aset, membandingkan pengaruh model *Game Theory* terhadap keputusan investasi.



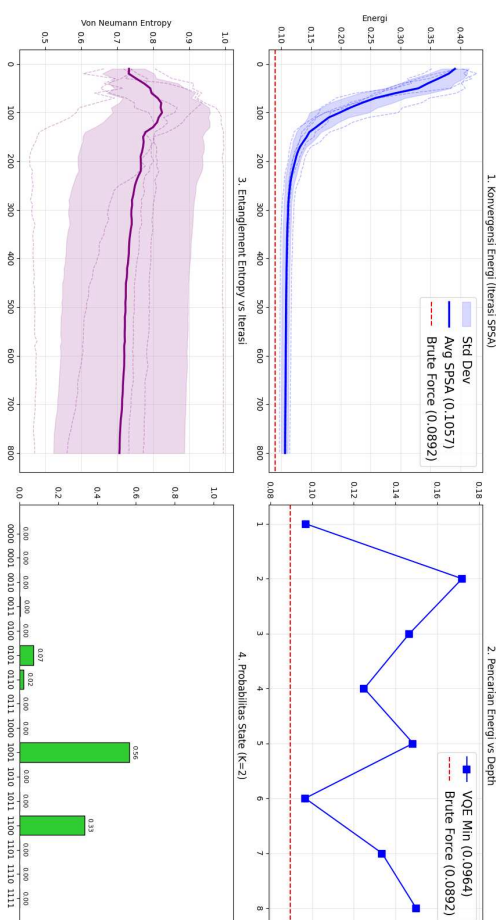
Gambar I.1: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2021-04-06

Analisis VQE tanpa Warm – Start - Jendela 2021-07-09 (N=4, K=2)



(b) Tanpa *Game Theory*

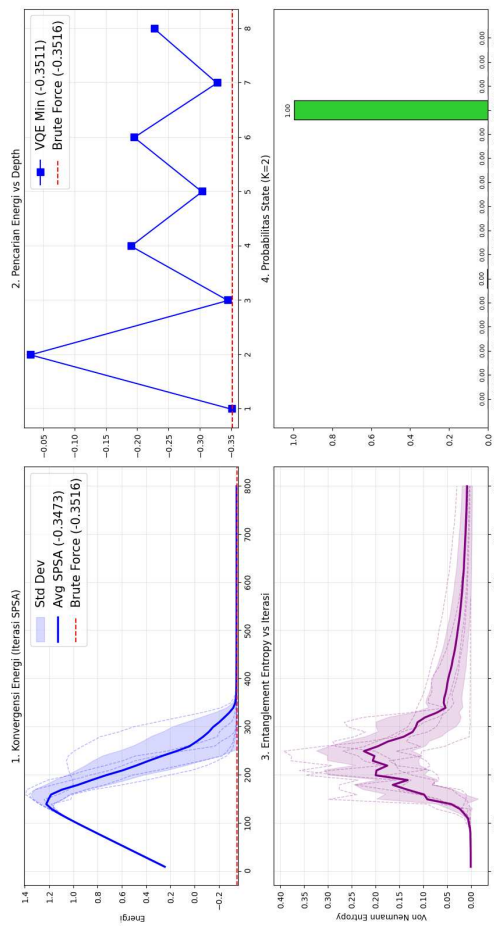
Analisis VQE Warm – Start - Jendela 2021-07-09 (N=4, K=2)



(a) Dengan *Game Theory*

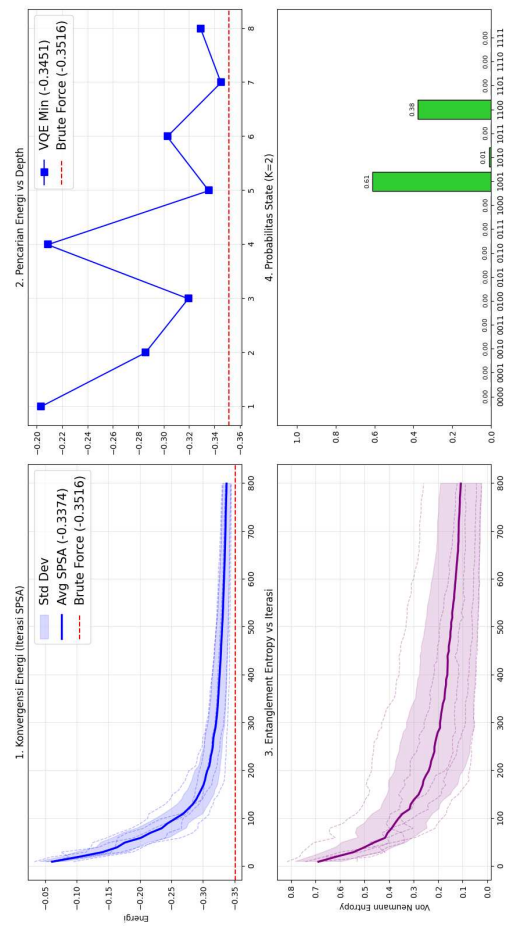
Gambar I.2: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2021-07-09

Analisis VQE Warm – Start - Jendela 2021-10-11 (N=4, K=2)



(a) Dengan *Game Theory*

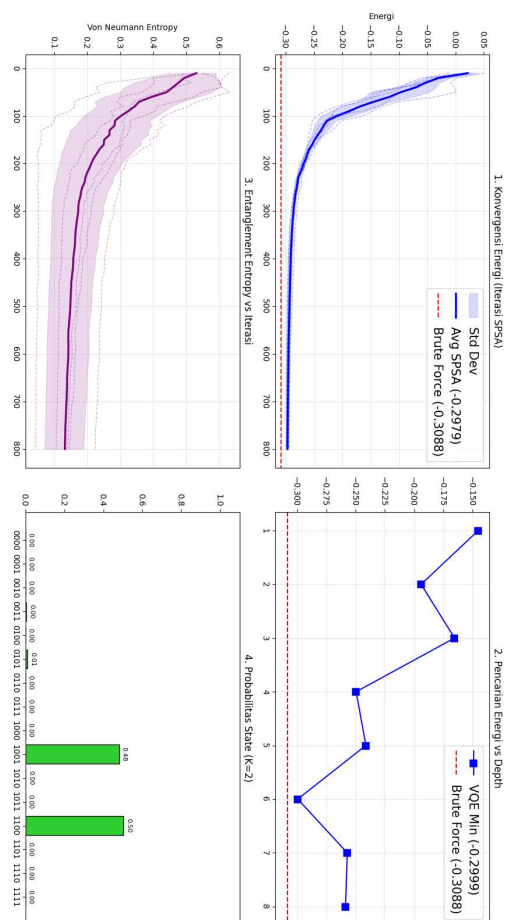
Analisis VQE tanpa Warm – Start - Jendela 2021-10-11 (N=4, K=2)



(b) Tanpa *Game Theory*

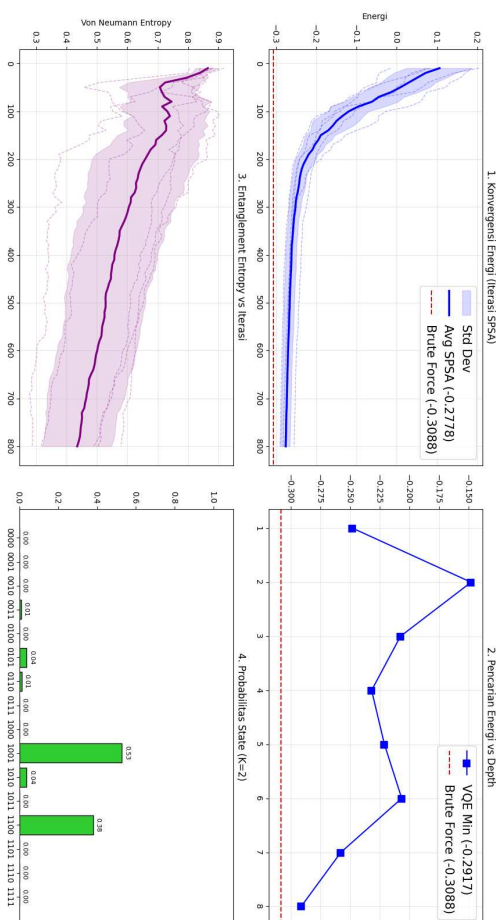
Gambar I.3: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2021-10-11

Analisis VQE tanpa Warm – Start - Jendela 2021-11-10 (N=4, K=2)



(b) Tanpa *Game Theory*

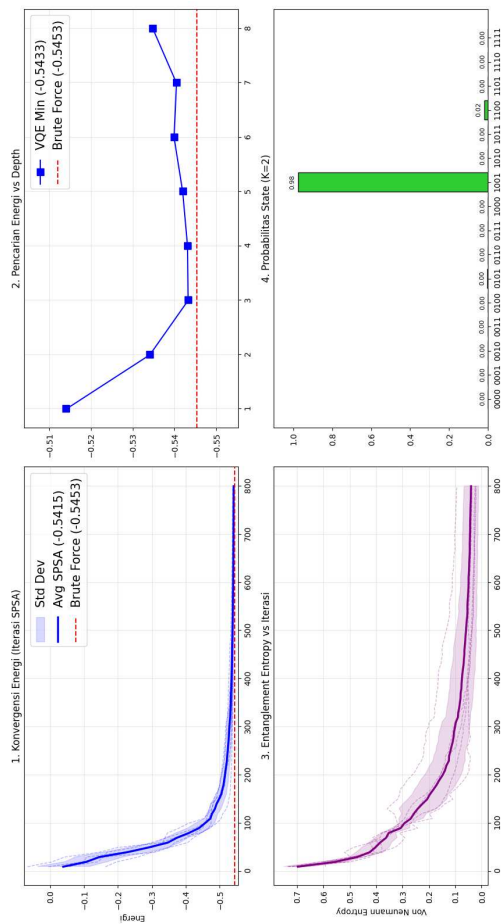
Analisis VQE Warm – Start - Jendela 2021-11-10 (N=4, K=2)



(a) Dengan *Game Theory*

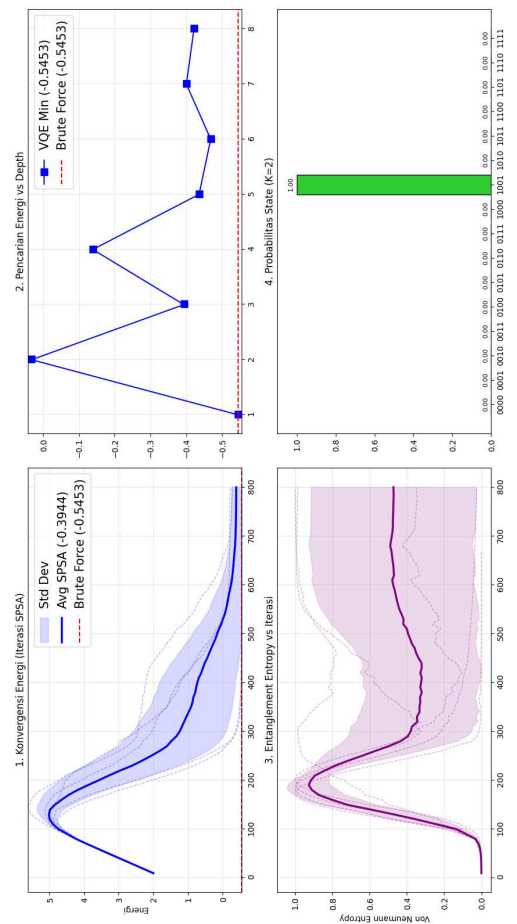
Gambar I.4: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2021-11-10

Analisis VQE Warm – Start - Jendela 2022-01-10 (N=4, K=2)



(a) Dengan *Game Theory*

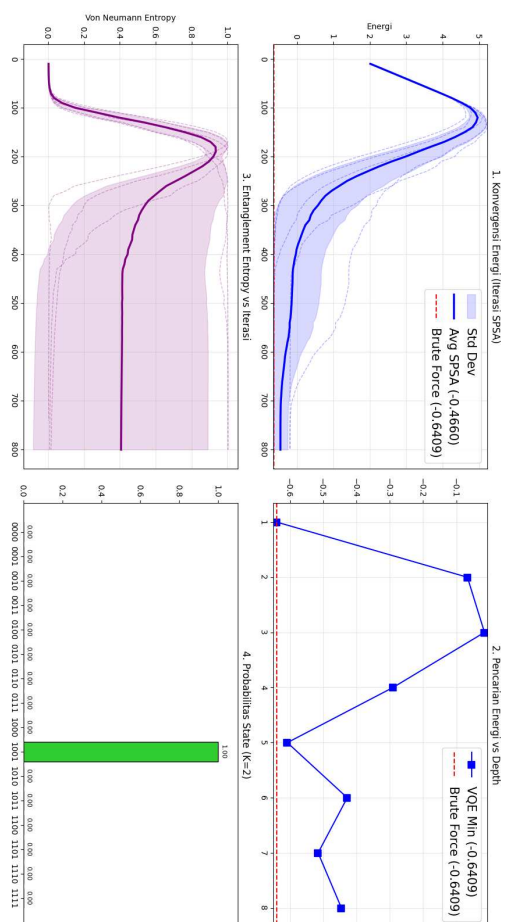
Analisis VQE tanpa Warm – Start - Jendela 2022-01-10 (N=4, K=2)



(b) Tanpa *Game Theory*

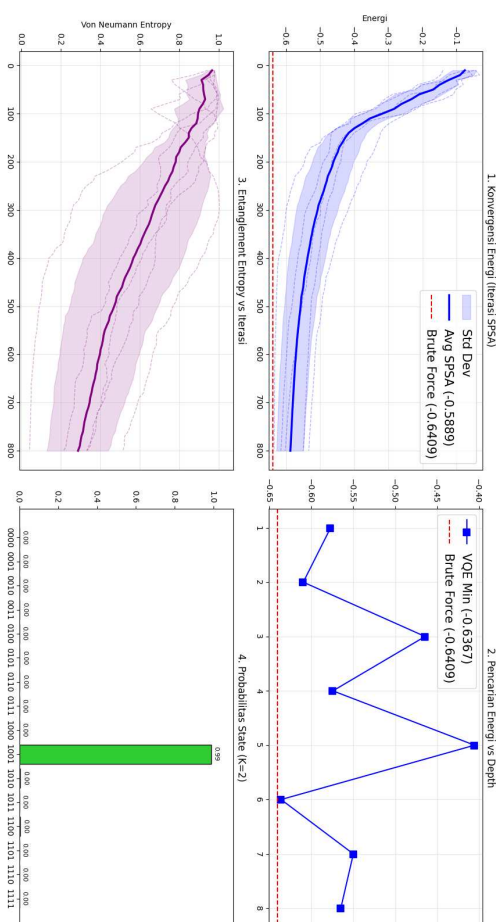
Gambar I.5: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2022-01-10

Analisis VQE tanpa Warm – Start - Jendela 2022-03-14 (N=4, K=2)



(b) Tanpa *Game Theory*

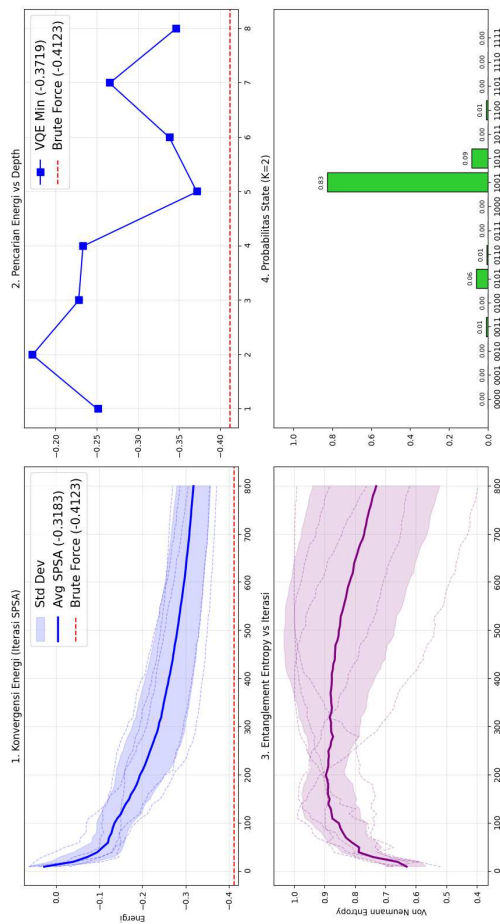
Analisis VQE Warm – Start - Jendela 2022-03-14 (N=4, K=2)



(a) Dengan *Game Theory*

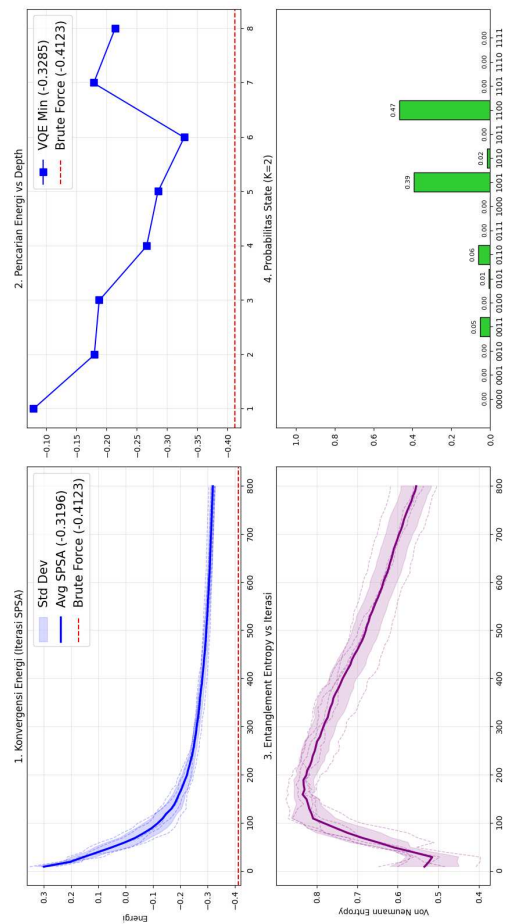
Gambar I.6: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2022-03-14

Analisis VQE Warm – Start - Jendela 2022-04-12 (N=4, K=2)



(a) Dengan *Game Theory*

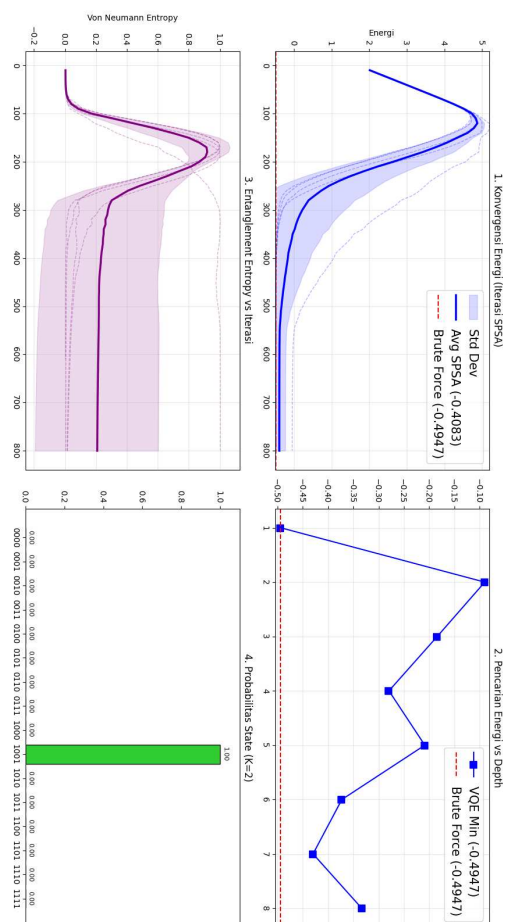
Analisis VQE tanpa Warm – Start - Jendela 2022-04-12 (N=4, K=2)



(b) Tanpa *Game Theory*

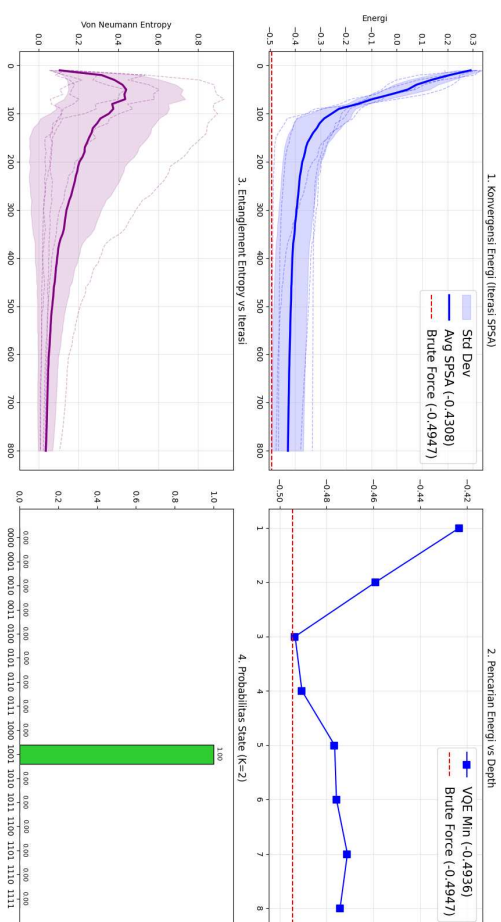
Gambar I.7: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2022-04-12

Analisis VQE tanpa Warm – Start - Jendela 2022-05-23 (N=4, K=2)



(b) Tanpa *Game Theory*

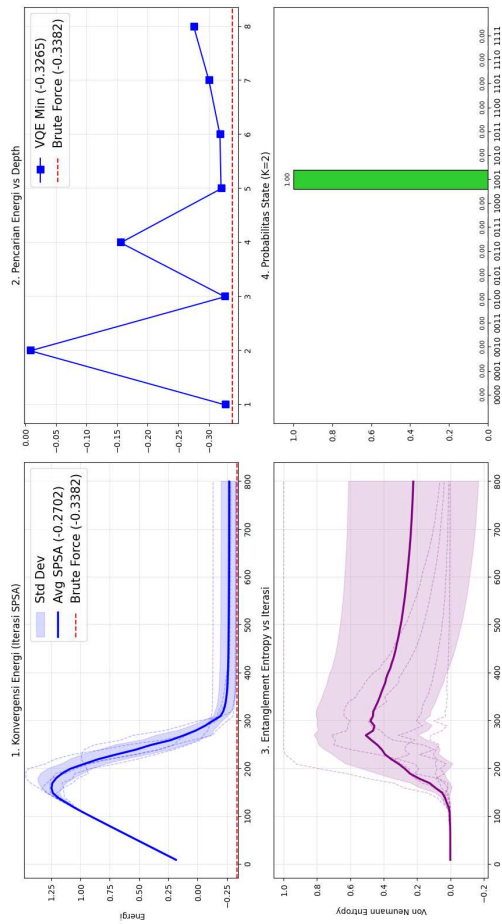
Analisis VQE Warm – Start - Jendela 2022-05-23 (N=4, K=2)



(a) Dengan *Game Theory*

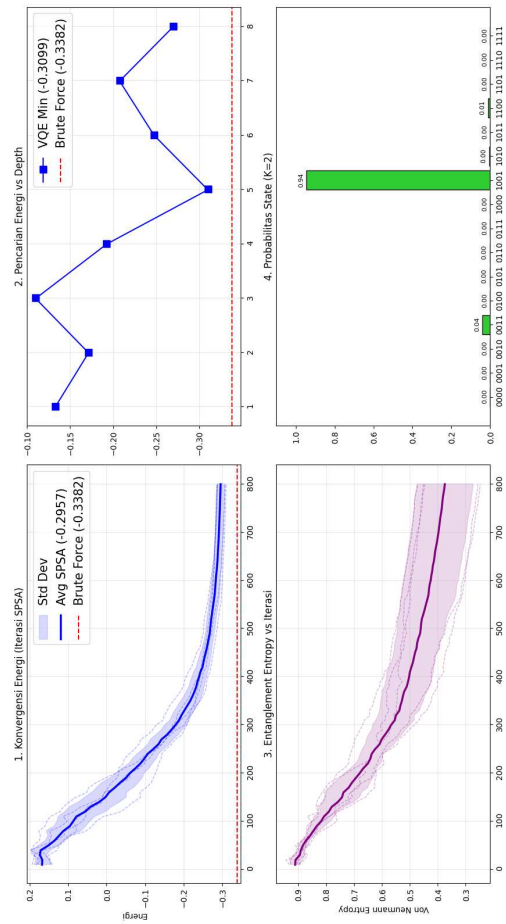
Gambar I.8: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2022-05-23

Analisis VQE Warm – Start - Jendela 2022-06-23 (N=4, K=2)



(a) Dengan *Game Theory*

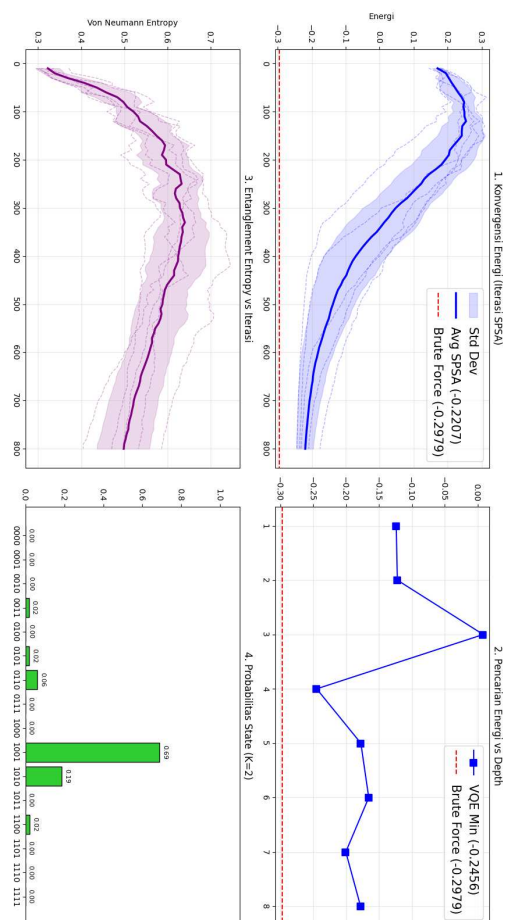
Analisis VQE tanpa Warm – Start - Jendela 2022-06-23 (N=4, K=2)



(b) Tanpa *Game Theory*

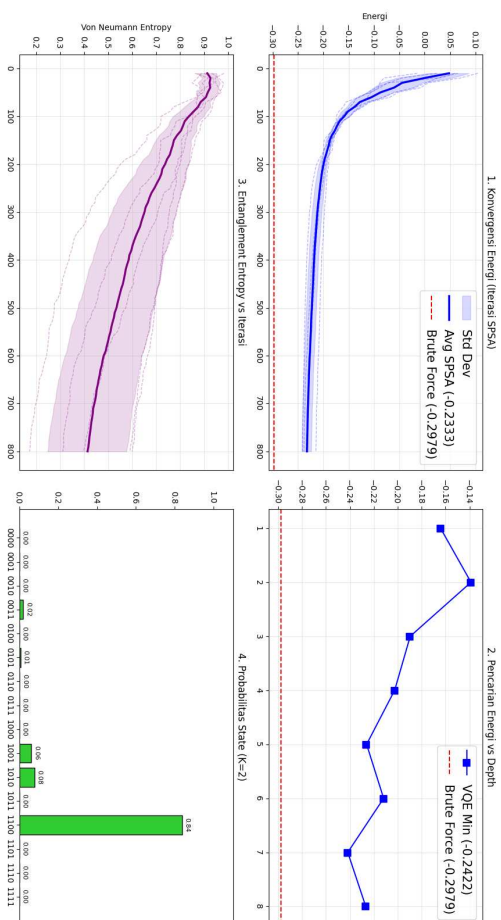
Gambar I.9: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2022-06-23

Analisis VQE tanpa Warm – Start - Jendela 2022-08-23 (N=4, K=2)



(b) Tanpa *Game Theory*

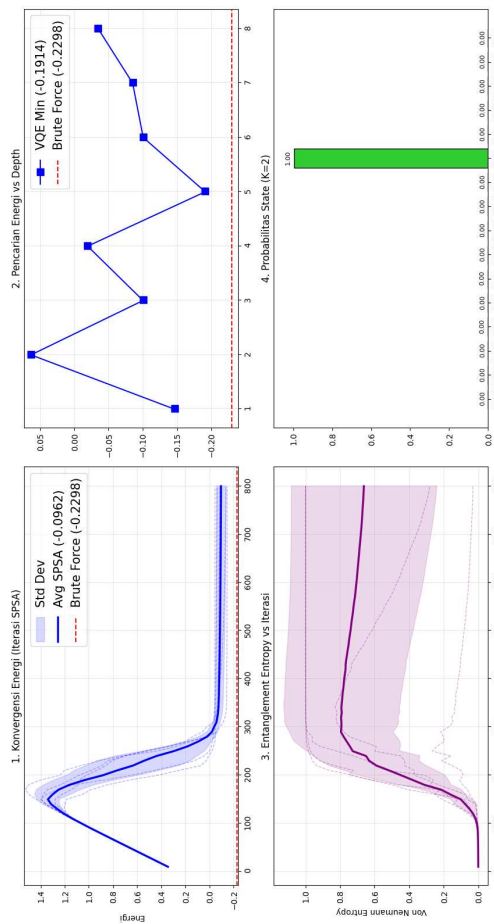
Analisis VQE Warm – Start - Jendela 2022-08-23 (N=4, K=2)



(a) Dengan *Game Theory*

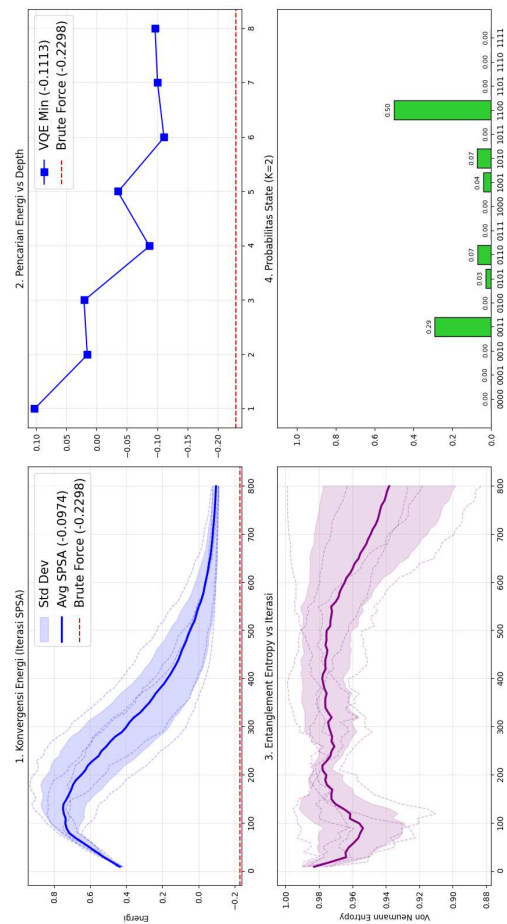
Gambar I.10: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2022-08-23

Analisis VQE Warm – Start - Jendela 2022-12-19 (N=4, K=2)



(a) Dengan *Game Theory*

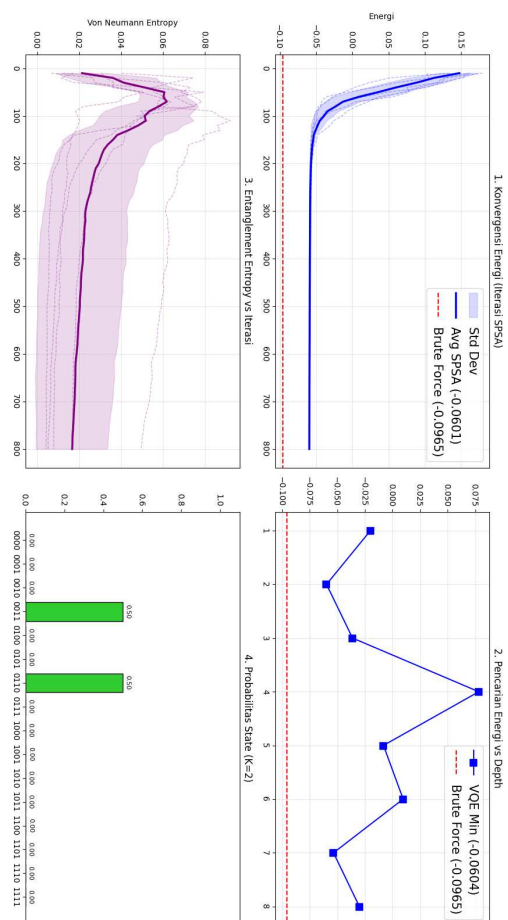
Analisis VQE tanpa Warm – Start - Jendela 2022-12-19 (N=4, K=2)



(b) Tanpa *Game Theory*

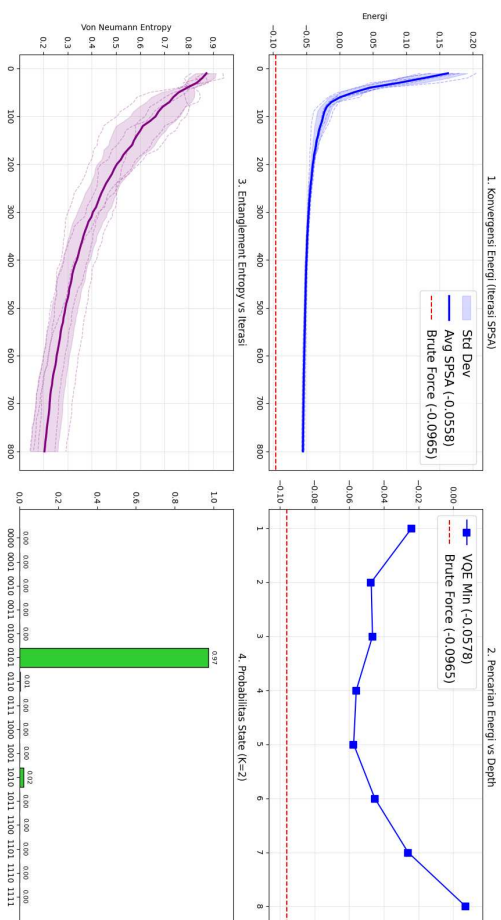
Gambar I.11: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2022-12-19

Analisis VQE tanpa Warm – Start - Jendela 2023-08-04 (N=4, K=2)



(b) Tanpa *Game Theory*

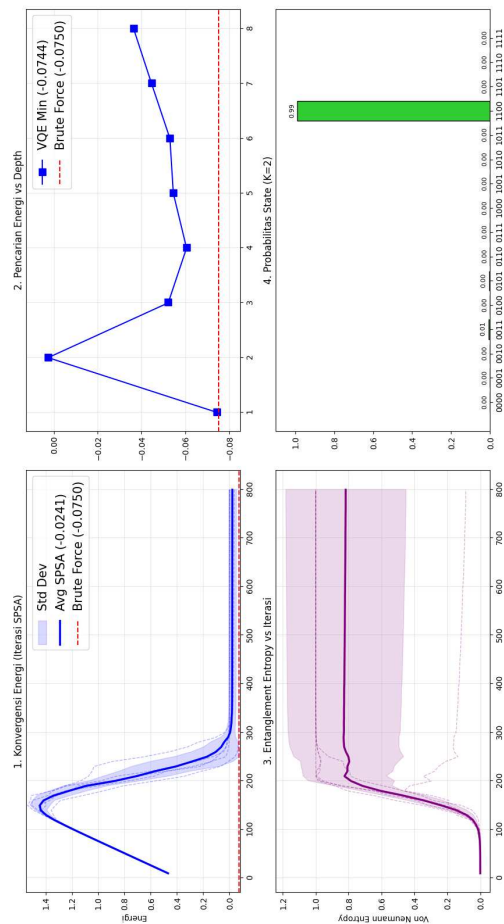
Analisis VQE Warm – Start - Jendela 2023-08-04 (N=4, K=2)



(a) Dengan *Game Theory*

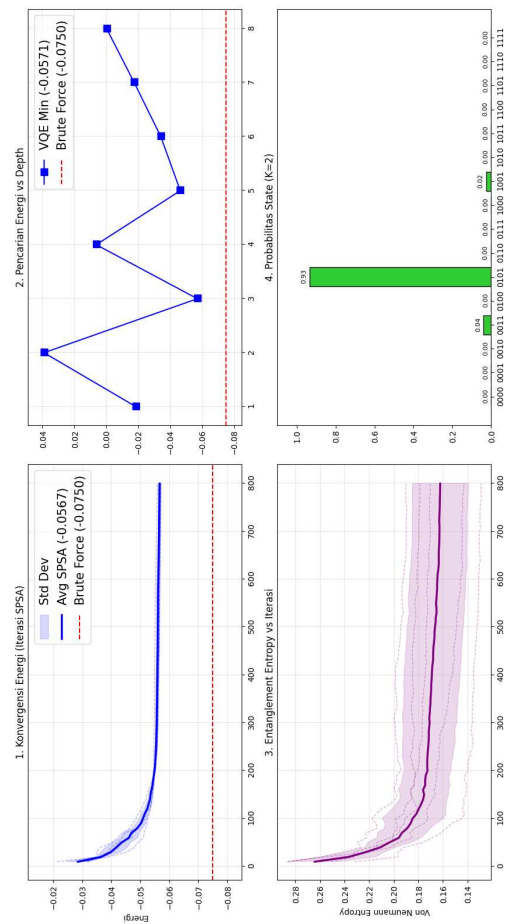
Gambar I.12: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2023-08-04

Analisis VQE Warm – Start - Jendela 2023-09-05 (N=4, K=2)



(a) Dengan *Game Theory*

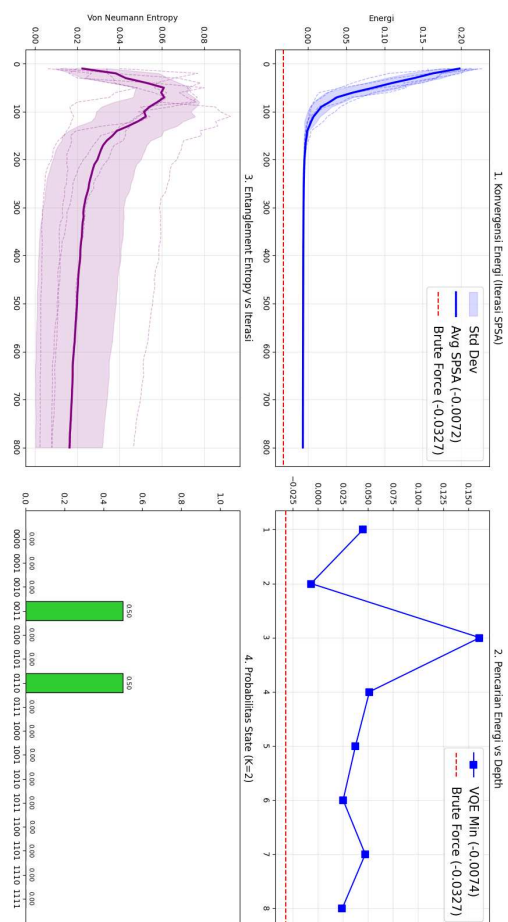
Analisis VQE tanpa Warm – Start - Jendela 2023-09-05 (N=4, K=2)



(b) Tanpa *Game Theory*

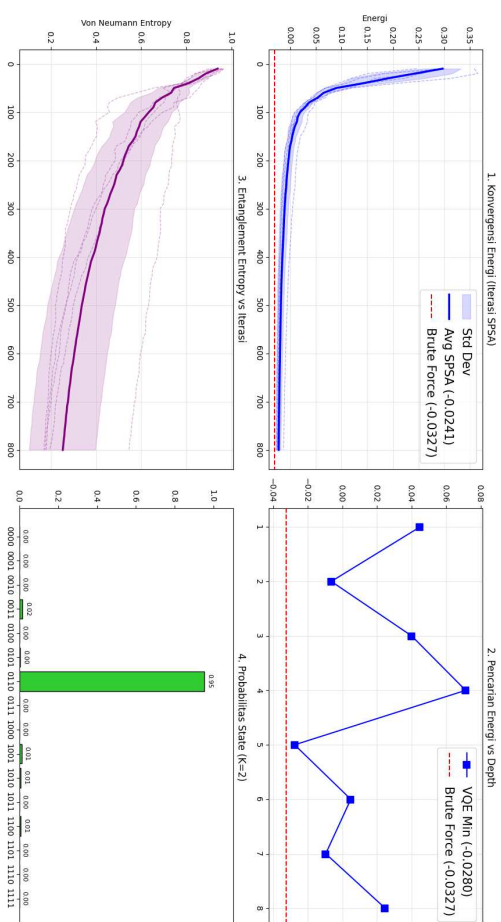
Gambar I.13: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2023-09-05

Analisis VQE tanpa Warm – Start - Jendela 2023-11-03 (N=4, K=2)



(b) Tanpa *Game Theory*

Analisis VQE Warm – Start - Jendela 2023-11-03 (N=4, K=2)



(a) Dengan *Game Theory*

Gambar I.14: Perbandingan Alokasi Portofolio N=4 Jendela Waktu 2023-11-03

Lampiran J

Skrip Visualisasi dan Pembangkitan Grafik Hasil Simulasi

Pada bagian ini, dilampirkan kumpulan skrip pemrograman *Python* yang secara khusus dikembangkan untuk mengeksekusi operasi pembuatan visualisasi data kuantitatif dari hasil simulasi *Variational Quantum Eigensolver* (VQE) dan ekuilibrium teori permanan. Skrip-skrip ini berfungsi sebagai instrumen konversi dari representasi data komputasional mentah menjadi bentuk diagram grafis yang komprehensif, mencakup pembentukan kurva konvergensi energi historis, histogram sebaran probabilitas status kuantum, grafik perbandingan imbal hasil portofolio, hingga komposisi metrik rasio absolut Sharpe. Rangkaian rutinitas komputasional visual ini merupakan tulang punggung utama dari analisis observabel data hasil evaluasi performa model yang memfasilitasi pelacakan konvergensi lanskap energi portofolio lintas dimensi aset.

```
import matplotlib
matplotlib.use('Agg') # Menggunakan backend non-interaktif
import matplotlib.pyplot as plt
import numpy as np
import pennylane as qml
import pandas as pd
import os

# Import Qiskit untuk perenderan rangkaian yang lebih baik
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

def plot_all(results, data_clean, value_benchmark_idx, config, filenames):
    """
    Menghasilkan semua grafik yang diperlukan untuk analisis.
    """
    tickers = config['tickers']
    N = len(tickers)
    benchmark_ticker = config['benchmark_ticker']

    # 1. Folder Khusus Analisis Jendela
    output_dir = f"Analisis_Window_N{N}"
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    print(f"\n--- Menghasilkan Grafik Analisis Per Jendela di '{output_dir}' ---")
    for i, win_data in enumerate(results['window_analysis_history']):
        win_filename = os.path.join(output_dir, f"{win_data['date']}_window.png")
        plot_window_analysis(win_data, n_qubits=N, filename=win_filename)

    # Render sirkuit per jendela
    circ_filename = os.path.join(output_dir, f"{win_data['date']}_circuit.png")
    plot_quantum_circuit(N, win_data['best_depth'], circ_filename, params=win_data['best_params'])
```

```

# 2. Update nama file rangkaian untuk depth terakhir (untuk folder Hasil_NX)
final_depth = results['depths_history'][-1] if len(results['depths_history']) > 0 else 1
suffix = f"_N{N}"
filenames['circuit_png'] = f'rangkai_kuantum_depth{final_depth}{suffix}.png'

# Rangkaian Kuantum Window Terakhir dengan Parameter
final_win = results['window_analysis_history'][-1]
plot_quantum_circuit(N, final_depth, filenames['circuit_png'], params=final_win['best_params'])

# 4. Hasil Backtest VQE vs Benchmarks
# Cari data classic jika ada
classic_file = f"classic_compare/Hasil_Classic_Compare/equity_history_classic_N{N}.csv"
value_classic = None
if os.path.exists(classic_file):
    try:
        df_classic = pd.read_csv(classic_file)
        value_classic = df_classic['Equity'].values
    except:
        pass

plot_equity_growth(data_clean, results['value_vqe'], results['value_bench'],
                    value_benchmark_idx, benchmark_ticker, 'Quantum VQE',
                    'blue', filenames['backtest_vqe_png'], value_classic=value_classic)

# 5. Hasil Backtest Nash vs Benchmarks
plot_equity_growth(data_clean, results['value_nash'], results['value_bench'],
                    value_benchmark_idx, benchmark_ticker, 'Nash Equilibrium (Klasik)',
                    'orange', filenames['backtest_nash_png'], value_classic=value_classic)

# 6. Depth per Window
plot_depth_per_window(results['rebalance_dates'], results['depths_history'], filenames['depth_png'])

def plot_window_analysis(win_data, n_qubits, filename):
    """
    Merender 6 panel analisis untuk satu jendela waktu tertentu.
    """
    date = win_data['date']
    best_history = win_data['best_history']
    best_ent_hist = win_data['best_ent_hist']
    depth_energies = win_data['depth_energies']
    lr_data = win_data['lr_data']
    probs = win_data['winning_probs']
    K = win_data['K']
    use_warm_start = win_data.get('use_warm_start', True)

    # Hitung Brute Force khusus untuk jendela ini (dengan suffix N)
    bf_energy = calculate_bf_for_window(date, n_qubits)

    fig, axes = plt.subplots(3, 2, figsize=(16, 15))

    # Judul Dinamis berdasarkan Warm-Start
    ws_text = "Warm-Start" if use_warm_start else "tanpa Warm-Start"
    fig.suptitle(f"Analisis VQE {ws_text} - Jendela {date} (N={n_qubits}, K={K})", fontsize=20, fontweight='bold')
    axes = axes.flatten()

    # 1. Konvergensi Energi
    iters = np.arange(1, len(best_history) + 1) * 10 # batch_size=10
    final_e_spsa = best_history[-1]
    axes[0].plot(iters, best_history, marker='o', markersize=3, color='blue', label=f'SPSA Progress ({final_e_spsa:.4f})')
    if bf_energy is not None:
        axes[0].axhline(y=bf_energy, color='red', linestyle='--', label=f'Brute Force ({bf_energy:.4f})')
        all_e = list(best_history) + [bf_energy]
        axes[0].set_ylim(min(all_e)-0.01, max(all_e)+0.01)
    axes[0].set_title('1. Konvergensi Energi (Iterasi SPSA)')
    axes[0].set_ylabel('Energi')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

```

```

# 2. Energi vs Depth
d_list = [d for d, e, it in depth_energies]
e_list = [e for d, e, it in depth_energies]
vqe_min_depth = min(e_list)
axes[1].plot(d_list, e_list, marker='s', markersize=8, color='blue', label=f'VQE Min ({vqe_min_depth:.4f})')
if bf_energy is not None:
    axes[1].axhline(y=bf_energy, color='red', linestyle='--', label=f'Brute Force ({bf_energy:.4f})')
    all_e = e_list + [bf_energy]
    axes[1].set_ylim(min(all_e)-0.01, max(all_e)+0.01)
axes[1].set_title('2. Pencarian Energi vs Depth')
axes[1].set_xticks(d_list)
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# 3. Entanglement Entropy
if len(best_ent_hist) > 0:
    axes[2].plot(iters, best_ent_hist, color='purple', marker='^', markersize=3)
    axes[2].set_title('3. Entanglement Entropy vs Iterasi')
    axes[2].set_ylabel('Von Neumann Entropy')
    axes[2].grid(True, alpha=0.3)

# 4. Learning Rate Finder
test_a, test_e, best_a = lr_data
axes[3].plot(test_a, test_e, marker='o', color='green')
axes[3].axvline(best_a, color='red', linestyle='--', label=f'Best a: {best_a:.4f}')
axes[3].set_xscale('log')
axes[3].set_title('4. LR Finder (Calibration)')
axes[3].legend()
axes[3].grid(True, alpha=0.3)

# 5. Distribusi Probabilitas (Bar Chart)
bitstrings = [format(i, f'0{n_qubits}b') for i in range(2**n_qubits)]
colors = ['limegreen' if b.count('1') == K else 'salmon' for b in bitstrings]
axes[4].bar(bitstrings, probs, color=colors, edgecolor='black')
axes[4].set_title(f'5. Probabilitas State (K={K})')
axes[4].set_ylim(0, 1.1)
# Tambahkan label teks di atas batang
for idx, p in enumerate(probs):
    axes[4].text(idx, p + 0.02, f'{p:.2f}', ha='center', fontsize=8)

# 6. Variansi Gradien (Diagnostic untuk Barren Plateaus)
best_gvar_hist = win_data.get('best_gvar_hist', [])
if len(best_gvar_hist) > 0:
    axes[5].plot(iters, best_gvar_hist, color='orange', marker='x', markersize=3)
    axes[5].set_yscale('log') # Gunakan skala log karena variansi bisa sangat kecil
    axes[5].set_title('6. Variansi Gradien vs Iterasi (Log)')
    axes[5].set_ylabel('Var(Grad)')
    axes[5].grid(True, alpha=0.3)
else:
    axes[5].text(0.5, 0.5, 'Data Gradien Tidak Tersedia', ha='center')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(filename)
plt.close()

def plot_quantum_circuit(N, depth, filename, params=None, title=None):
    """
    Merender rangkaian kuantum menggunakan Qiskit untuk hasil yang lebih berwarna,
    informatif (label parameter), dan otomatis dibagi baris (folding).
    """
    qr = QuantumRegister(N, 'q')
    cr = ClassicalRegister(N, 'c')
    qc = QuantumCircuit(qr, cr)

    if params is None:
        params = np.zeros(N * 2 * (depth + 1))

    # Reshape parameter sesuai struktur ansatz (layer, qubit, gate_type)
    w = params.reshape((depth + 1, N, 2))

```

```

for layer in range(depth + 1):
    for q_idx in range(N):
        # Ambil nilai theta dan bulatkan agar tidak terlalu panjang di gambar
        theta_ry = np.round(float(w[layer, q_idx, 0]), 2)
        theta_rz = np.round(float(w[layer, q_idx, 1]), 2)

        # Tambahkan gate RY dan RZ dengan label parameter numerik
        qc.ry(theta_ry, qr[q_idx])
        qc.rz(theta_rz, qr[q_idx])

    if layer < depth:
        # Tambahkan Barrier antar layer untuk kejelasan visual
        qc.barrier()
        # Chain CNOT (Entanglement)
        for q_idx in range(N - 1):
            qc.cx(qr[q_idx], qr[q_idx + 1])
        qc.cx(qr[N - 1], qr[0])
        qc.barrier()

# Tambahkan Pengukuran di akhir
qc.measure(qr, cr)

# Pengaturan Visual Qiskit
# style='iqp' memberikan skema warna modern/IBM standard
# fold=20 disetel agar muat sekitar 5 depth per baris (mencegah terlalu panjang ke samping
# )
fig = qc.draw(output='mpl', style='iqp', plot_barriers=True, justify='left', fold=20)

if title:
    fig.suptitle(title, fontsize=16, fontweight='bold', y=1.05)
else:
    fig.suptitle(f"VQE Optimized Circuit (N={N}, Depth={depth})", fontsize=16, fontweight=
'bold', y=1.05)

fig.savefig(filename, bbox_inches='tight', dpi=150)
plt.close(fig)
print(f"Gambar rangkaian kuantum (Qiskit) disimpan sebagai '{filename}'.")

def calculate_bf_for_window(date, n_qubits):
    """Fungsi pembantu untuk hitung BF on-the-fly dengan Penalty Annealing Target."""
    suffix = f"_N{n_qubits}"
    try:
        # 1. Ambil Bias (h)
        h_df = pd.read_csv(f'bias_h_total{suffix}.csv')
        h_win = h_df[h_df['Date'] == str(date)]
        h_obj = h_win['Bias_h_Obj'].values
        h_pen = h_win['Bias_h_Pen'].values
        n = len(h_obj)

        # 2. Ambil Interaksi (J)
        J_df = pd.read_csv(f'interaksi_J_total{suffix}.csv')
        J_win = J_df[J_df['Date'] == str(date)]
        J_mat_obj = np.zeros((n, n))
        J_mat_pen = np.zeros((n, n))
        tickers = h_win['Ticker'].values
        t2i = {t: i for i, t in enumerate(tickers)}
        for _, r in J_win.iterrows():
            i, j = t2i[r['Ticker_i']], t2i[r['Ticker_j']]
            J_mat_obj[i, j] = J_mat_obj[j, i] = r['Interaction_J_Obj']
            J_mat_pen[i, j] = J_mat_pen[j, i] = r['Interaction_J_Pen']

        # 3. Ambil Konstanta C dan Target Penalty
        c_df = pd.read_csv(f'parameter_pendamping{suffix}.csv')
        c_win = c_df[c_df['Date'] == str(date)].iloc[0]
        c_obj = c_win['C_Obj']
        c_pen = c_win['C_Pen']
        penalty_A = c_win['Penalty_A_Target']

        # 4. Kombinasikan menjadi Hamiltonian Total Akhir
        h_total = h_obj + penalty_A * h_pen
        J_total = J_mat_obj + penalty_A * J_mat_pen

```

```

c_total = c_obj + penalty_A * c_pen

from itertools import product
min_e = float('inf')
for b in product([0, 1], repeat=n):
    Z = 1 - 2 * np.array(b)
    # E = sum(h_i * Z_i) + sum(J_ij * Z_i * Z_j) + C
    e = np.sum(h_total * Z) + sum(J_total[i,j]*Z[i]*Z[j] for i in range(n) for j in
range(i+1, n)) + c_total
    if e < min_e: min_e = e
    return min_e
except:
    return None

plt.tight_layout()
plt.savefig(filename)
print(f"Grafik konvergensi detail disimpan sebagai '{filename}'.")

def plot_equity_growth(data_clean, value_strategy, value_bench, value_benchmark_idx,
benchmark_ticker, strategy_label, color, filename, value_classic=None):
    plt.figure(figsize=(12, 6))
    plt.plot(data_clean.index[:len(value_strategy)], value_strategy, label=strategy_label,
linewidth=2.5, color=color)
    plt.plot(data_clean.index[:len(value_bench)], value_bench, label='Benchmark (Equal Weight)
', linestyle=':', color='black', alpha=0.6)
    plt.plot(value_benchmark_idx.index, value_benchmark_idx.values, label=f'Indeks ({
benchmark_ticker})', linestyle='-.', color='magenta', linewidth=2)

    if value_classic is not None:
        plt.plot(data_clean.index[:len(value_classic)], value_classic, label='Classic
Markowitz (Mean-Var)', linestyle='--', color='green', linewidth=2)

    plt.title(f'Perbandingan Pertumbuhan Ekuitas: {strategy_label} vs Benchmarks')
    plt.ylabel('Total Ekuitas (Rupiah)')
    plt.xlabel('Tanggal')
    plt.legend(loc='upper left')
    plt.grid(True, linestyle=':', alpha=0.7)
    plt.tight_layout()
    plt.savefig(filename)
    print(f"Grafik {strategy_label} disimpan sebagai '{filename}'.")

def plot_depth_per_window(rebalance_dates, depths_history, filename):
    plt.figure(figsize=(10, 5))
    avg_depth_floor = int(np.floor(np.mean(depths_history)))
    plt.plot(rebalance_dates, depths_history, marker='o', linestyle='-', color='purple', label
='Depth Terpilih')
    plt.axhline(y=avg_depth_floor, color='red', linestyle='--', label=f'Rata-rata Depth (Floor
): {avg_depth_floor}')
    plt.title('Depth Terpilih vs Rebalance Window')
    plt.ylabel('Depth')
    plt.xlabel('Tanggal Rebalance')
    plt.xticks(rotation=45)
    plt.legend()
    plt.grid(True, linestyle=':', alpha=0.7)
    plt.tight_layout()
    plt.savefig(filename)
    print(f"Grafik depth per window disimpan sebagai '{filename}'. Rata-rata (floor): {
avg_depth_floor}")

```

Kode J.1: Skrip utama generator grafik evaluasi portofolio (plot_generator.py)

```

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np
import pennylane as qml
import pandas as pd
import os

def render_vqe_probability_bar(n_qubits, K):
    # 1. Ambil Parameter Terbaik (Theta) dari hasil run sebelumnya
    # Kita asumsikan file 'theta_final_all_depths.csv' ada

```

```

if not os.path.exists('theta_final_all_depths.csv'):
    print("Error: File theta_final_all_depths.csv tidak ditemukan. Jalankan simulasi
    terlebih dahulu.")
    return

df_theta = pd.read_csv('theta_final_all_depths.csv')

# Ambil depth terakhir (max depth) yang tercatat
max_depth_in_file = df_theta['Depth'].max()
params = df_theta[df_theta['Depth'] == max_depth_in_file]['Theta_Value'].values

print(f"Merender Probabilitas untuk Depth: {max_depth_in_file}")

# 2. Definisikan Sirkuit Kuantum untuk mendapatkan Probabilitas
dev = qml.device("default.qubit", wires=n_qubits)

@qml.qnode(dev)
def prob_circuit(p):
    depth = max_depth_in_file
    w = p.reshape((depth + 1, n_qubits, 2))
    for layer in range(depth + 1):
        for q in range(n_qubits):
            qml.RY(w[layer, q, 0], wires=q)
            qml.RZ(w[layer, q, 1], wires=q)
        if layer < depth:
            for q in range(n_qubits - 1):
                qml.CNOT(wires=[q, q + 1])
            qml.CNOT(wires=[n_qubits - 1, 0])
    return qml.probs(wires=range(n_qubits))

# Hitung Probabilitas
probs = prob_circuit(params)
bitstrings = [format(i, f'0{n_qubits}b') for i in range(2**n_qubits)]

# 3. Visualisasi Grafik Batang
plt.figure(figsize=(10, 6))

# Tentukan warna: Hijau untuk yang valid (sum=K), Merah untuk yang melanggar
colors = []
for bs in bitstrings:
    if bs.count('1') == K:
        colors.append('limegreen')
    else:
        colors.append('salmon')

bars = plt.bar(bitstrings, probs, color=colors, edgecolor='black', alpha=0.8)

# Tambahkan Label Probabilitas di atas batang
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01, f'{yval:.3f}', ha='center', va=
    'bottom', fontsize=10)

# Tambahkan keterangan/legend manual
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0], [0], color='limegreen', lw=4, label=f'Valid (K={K})'),
    Line2D([0], [0], color='salmon', lw=4, label='Invalid (Penalty Applied)')
]
plt.legend(handles=legend_elements, loc='upper right')

plt.title(f"Distribusi Probabilitas State VQE (N={n_qubits}, K={K})", fontsize=14)
plt.ylabel("Probabilitas", fontsize=12)
plt.xlabel("Bitstring (Spin Interpretation)", fontsize=12)
plt.ylim(0, 1.1) # Beri ruang untuk label angka
plt.grid(axis='y', linestyle='--', alpha=0.6)

# Tandai Pemenang
winner_idx = np.argmax(probs)
plt.annotate('Kandidat Terpilih\n(Energi Terendah)',
    xy=(winner_idx, probs[winner_idx]),
    xytext=(winner_idx, probs[winner_idx] + 0.15),

```

```

        arrowprops=dict(facecolor='black', shrink=0.05),
        ha='center', fontweight='bold')

filename = "visualisasi_probabilitas_vqe.png"
plt.savefig(filename, dpi=300)
plt.close()
print(f"Grafik probabilitas berhasil disimpan sebagai: {filename}")

if __name__ == "__main__":
    # Gunakan parameter yang sesuai dengan run terakhir Anda
    render_vqe_probability_bar(n_qubits=2, K=1)

```

Kode J.2: Skrip visualisasi distribusi probabilitas status VQE (render_vqe_probs.py)

```

import marimo

__generated_with__ = "0.23.5"
app = marimo.App()

@app.cell
def _():
    import marimo as mo
    import os
    import re

    return mo, os, re

@app.cell
def _(mo, os, re):
    # Fungsi untuk mencari file apa saja yang di-import oleh sebuah file
    def scan_dependencies(file_path, project_files):
        found_deps = set()
        try:
            with open(file_path, 'r', encoding='utf-8') as f:
                content = f.read()
                # Mencari pola 'import nama_file' atau 'from nama_file import ...'
                for other_file in project_files:
                    module_name = other_file.replace('.py', '')
                    # Regex untuk memastikan kita tidak salah mencocokkan kata di dalam string
                    pattern = rf'\b(import|from)\s+{module_name}\b'
                    if re.search(pattern, content):
                        found_deps.add(other_file)
        except:
            pass
        return found_deps

    def draw_project_map():
        root = os.getcwd()
        # Filter: Hanya ambil file .py, abaikan project_graph.py dan main_N...py
        all_py_files = [
            f for f in os.listdir(root)
            if f.endswith('.py')
            and f != 'project_graph.py'
            and not re.match(r'main_N\d+\.py', f)
        ]

        mermaid_lines = ["graph TD"]

        # Tambahkan gaya visual yang lebih kaya
        mermaid_lines.append("classDef core fill:#FF9800,stroke:#E65100,stroke-width:2px,color:white;")
        mermaid_lines.append("classDef strategy fill:#2196F3,stroke:#0D47A1,stroke-width:2px,color:white;")
        mermaid_lines.append("classDef engine fill:#4CAF50,stroke:#1B5E20,stroke-width:2px,color:white;")
        mermaid_lines.append("classDef math fill:#9C27B0,stroke:#4A148C,stroke-width:1px,color:white;")

        # Kategorisasi File
        categories = {

```

```

        'core': ['main.py', 'config.py', 'backtest_runner.py', 'report_generator.py', '
plot_generator.py', 'data_downloader.py'],
        'strategy': ['run_strategy_step.py', 'rebalance_portfolio.py'],
        'engine': ['run_vqe_adaptive.py', 'find_nash_sbr.py', 'brute_force_validator.py',
'find_optimal_lr_spsa.py', 'run_spsa_test.py'],
        'math': [f for f in all_py_files if f.startswith(('calc_', 'compute_', 'build_'))]
    }

    def get_style(filename):
        for cat, files in categories.items():
            if filename in files:
                return f"::{cat}"
        return ""

    has_connections = False
    seen_edges = set()

    for file in all_py_files:
        deps = scan_dependencies(os.path.join(root, file), all_py_files)

        # Styling node berdasarkan kategori
        style = get_style(file)
        mermaid_lines.append(f'    {file.replace(".", "_")}["{file}"]{style}')

        for dep in deps:
            if file != dep:
                src = file.replace(".", "_")
                dst = dep.replace(".", "_")
                edge = (src, dst)
                if edge not in seen_edges:
                    mermaid_lines.append(f'    {src} --> {dst}')
                    seen_edges.add(edge)
                    has_connections = True

    if not has_connections:
        return mo.md("### Tidak ditemukan hubungan import antar file.\nPastikan file Anda
menggunakan `import nama_file_lain`.")

    return mo.vstack([
        mo.md("# Peta Hubungan Kode Projekmu"),
        mo.md("Garis panah menunjukkan file mana yang 'memanggil' file lainnya."),
        mo.mermaid("\n".join(mermaid_lines))
    ])

    draw_project_map()
    return

if __name__ == "__main__":
    app.run()

```

Kode J.3: Skrip visualisasi grafik interaksi antar aset (project_graph.py)

```

import os
import re
import base64
import requests # Pastikan library requests tersedia

def scan_dependencies(file_path, project_files):
    found_deps = set()
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read()
            for other_file in project_files:
                module_name = other_file.replace('.py', '')
                pattern = rf'\b(import|from)\s+{module_name}\b'
                if re.search(pattern, content):
                    found_deps.add(other_file)
    except:
        pass
    return found_deps

def generate_mermaid():

```



```

root = os.getcwd()
all_py_files = [
    f for f in os.listdir(root)
    if f.endswith('.py')
    and f not in ['project_graph.py', 'export_graph.py']
    and not re.match(r'main_N\d+\.py', f)
]

mermaid_lines = ["graph TD"]
mermaid_lines.append("    classDef core fill:#FF9800,stroke:#E65100,stroke-width:2px,color:white;")
mermaid_lines.append("    classDef strategy fill:#2196F3,stroke:#0D47A1,stroke-width:2px,color:white;")
mermaid_lines.append("    classDef engine fill:#4CAF50,stroke:#1B5E20,stroke-width:2px,color:white;")
mermaid_lines.append("    classDef math fill:#9C27B0,stroke:#4A148C,stroke-width:1px,color:white;")

categories = {
    'core': ['main.py', 'config.py', 'backtest_runner.py', 'report_generator.py', 'plot_generator.py', 'data_downloader.py'],
    'strategy': ['run_strategy_step.py', 'rebalance_portfolio.py'],
    'engine': ['run_vqe_adaptive.py', 'find_nash_sbr.py', 'brute_force_validator.py', 'find_optimal_lr_spsa.py', 'run_spsa_test.py'],
    'math': [f for f in all_py_files if f.startswith(('calc_', 'compute_', 'build_'))]
}

def get_style(filename):
    for cat, files in categories.items():
        if filename in files:
            return f"::{cat}"
    return ""

seen_edges = set()
for file in all_py_files:
    style = get_style(file)
    mermaid_lines.append(f'    {file.replace(".", "_")}["{file}"]{style}')
    deps = scan_dependencies(os.path.join(root, file), all_py_files)
    for dep in deps:
        if file != dep:
            src = file.replace(".", "_")
            dst = dep.replace(".", "_")
            if (src, dst) not in seen_edges:
                mermaid_lines.append(f"    {src} --> {dst}")
                seen_edges.add((src, dst))

return "\n".join(mermaid_lines)

def export_png(mermaid_text, output_file="project_graph.png"):
    print("Mengonversi Mermaid ke PNG via mermaid.ink...")
    # Encode ke base64
    graphbytes = mermaid_text.encode("ascii")
    base64_bytes = base64.b64encode(graphbytes)
    base64_string = base64_bytes.decode("ascii")

    url = "https://mermaid.ink/img/" + base64_string

    try:
        response = requests.get(url)
        if response.status_code == 200:
            with open(output_file, "wb") as f:
                f.write(response.content)
            print(f"Berhasil! Gambar disimpan sebagai: {output_file}")
        else:
            print(f"Gagal mengunduh gambar. Status code: {response.status_code}")
    except Exception as e:
        print(f"Terjadi kesalahan: {e}")

if __name__ == "__main__":
    mermaid_str = generate_mermaid()

```

```
export_png(mermaid_str)
```

Kode J.4: Skrip utilitas untuk pengeksporan hasil graf fisis (`export_graph.py`)

Lampiran K

Skrip Utama Orkestrasi dan Algoritma Hibrida Kuantum

Pada bagian ini, dilampirkan kumpulan skrip pemrograman utama yang mengatur alur logika eksekusi algoritma GT-VQE secara terintegrasi.

```
import warnings
import pandas as pd
from data_downloader import download_market_data
from backtest_runner import run_backtest
from report_generator import generate_report
from plot_generator import plot_all
from config import get_config, clean_workspace

warnings.filterwarnings('ignore')

tickers = ['BBCA.JK', 'TLKM.JK', 'SMGR.JK', 'ADRO.JK']
N = len(tickers)

# 0. Bersihkan Workspace
clean_workspace(N)

config = get_config(tickers)

suffix = f"_N{N}"
filenames = {
    'daily_prices_csv': f'harga_harian_saham{suffix}.csv',
    'report_txt': f'laporan_backtest{suffix}.txt',
    'circuit_png': f'rangkaian_kuantum{suffix}.png',
    'convergence_png': f'grafik_konvergensi_detail{suffix}.png',
    'backtest_vqe_png': f'hasil_backtest_vqe{suffix}.png',
    'backtest_nash_png': f'hasil_backtest_nash{suffix}.png',
    'depth_png': f'grafik_depth_per_window{suffix}.png'
}

data_clean, benchmark_data, benchmark_rets = download_market_data(
    config['tickers'], config['benchmark_ticker'],
    config['start_date'], config['end_date'], config['start_bt_date']
)
data_clean.to_csv(filenames['daily_prices_csv'])
print(f>Data harga harian telah diekspor ke '{filenames['daily_prices_csv']}'")

results = run_backtest(data_clean, config['tickers'], config)
value_benchmark_idx = generate_report(results, data_clean, benchmark_data, benchmark_rets,
    config['tickers'], config, filenames['report_txt'])
plot_all(results, data_clean, value_benchmark_idx, config, filenames)
print(f"\nSeluruh proses N={N} selesai.")
```

Kode K.1: Skrip utama orkestrasi keseluruhan parameter dan eksekusi pada sistem N=4 (main_N4.py)

```

import numpy as np
import pandas as pd
import os
import pennylane as qml
from compute_metrics import calculate_entanglement_entropy

def run_vqe_adaptive(H_obj, H_pen, n_qubits, curr_date, ne_bitstring=None, K=2, target_penalty
=5.0,
                    max_depth=4, maxiter=100, max_total_iter=500,
                    batch_size=10, conv_window=4, conv_tol=1e-3,
                    best_a_base=0.1, seed=42, file_config=None):
    """
    Menjalankan VQE dengan Penalty Annealing.
    H_total(t) = H_obj + penalty(t) * H_pen
    """
    if file_config is None:
        file_config = {
            'riwayat_iterasi': 'riwayat_iterasi_vqe.csv',
            'theta_final': 'theta_final_all_depths.csv',
            'depth_vs_energi': 'hasil_depth_vs_energi.csv'
        }

    dev = qml.device("default.qubit", wires=n_qubits)
    rng = np.random.default_rng(seed)

    # --- RESET RIWAYAT ITERASI ---
    if os.path.exists(file_config['riwayat_iterasi']):
        os.remove(file_config['riwayat_iterasi'])

    def make_circuit(depth):
        def ansatz(params):
            w = params.reshape((depth + 1, n_qubits, 2))
            for layer in range(depth + 1):
                for q in range(n_qubits):
                    qml.RY(w[layer, q, 0], wires=q)
                    qml.RZ(w[layer, q, 1], wires=q)
                if layer < depth:
                    for q in range(n_qubits - 1):
                        qml.CNOT(wires=[q, q + 1])
                    qml.CNOT(wires=[n_qubits - 1, 0])

        @qml.qnode(dev)
        def cost_circuit(params, p_scale=1.0):
            ansatz(params)
            # H_total dinamis: H_obj + scale * H_pen
            return qml.expval(H_obj + p_scale * H_pen)

        @qml.qnode(dev)
        def prob_circuit(params):
            ansatz(params)
            return qml.probs(wires=range(n_qubits))

        @qml.qnode(dev)
        def state_circuit(params):
            ansatz(params)
            return qml.state()

        return cost_circuit, prob_circuit, state_circuit

    def run_spsa(cost_circuit, state_circuit, n_params, init_params=None, a_base=0.1, c_base
=0.1, depth_label=1, target_p=5.0):
        a, c = a_base, c_base
        A_coeff = maxiter * 0.1
        alpha_exp = 0.602
        gamma_exp = 0.101

        if init_params is not None:
            params = init_params.copy()
        else:
            params = rng.uniform(0, 2 * np.pi, n_params)

        energy_history = []

```

```

entropy_history = []
grad_var_history = []
total_iters = 0

while total_iters < max_total_iter:
    # --- PENALTY ANNEALING TERKALIBRASI ---
    # Pinalti mencapai 100% pada 70% total budget iterasi.
    # Sisa 30% digunakan untuk "fine-tuning" dengan pinalti konstan.
    anneal_progress = min(1.0, total_iters / (0.7 * max_total_iter))
    p_scale = target_p * (0.1 + 0.9 * anneal_progress)

    current_grad_vars = []
    for _ in range(batch_size):
        k = total_iters
        a_k = a / (A_coeff + k + 1) ** alpha_exp
        c_k = c / (k + 1) ** gamma_exp
        delta = 2 * rng.integers(0, 2, size=n_params) - 1

        cost_plus = float(cost_circuit(params + c_k * delta, p_scale=p_scale))
        cost_minus = float(cost_circuit(params - c_k * delta, p_scale=p_scale))
        grad = (cost_plus - cost_minus) / (2 * c_k * delta)

        # [Phase 3] Gradient Clipping
        grad = np.clip(grad, -1.0, 1.0)

        params = params - a_k * grad
        current_grad_vars.append(np.var(grad))
        total_iters += 1

    current_energy = float(cost_circuit(params, p_scale=p_scale))
    energy_history.append(current_energy)
    grad_var_history.append(np.mean(current_grad_vars))

    # Hitung Entropi
    try:
        current_psi = state_circuit(params)
        current_entropy = calculate_entanglement_entropy(current_psi)
        entropy_history.append(current_entropy)
    except:
        entropy_history.append(0.0)

    iter_df = pd.DataFrame({
        'Depth': [depth_label],
        'Iteration': [total_iters],
        'Energy': [current_energy],
        'Entropy': [entropy_history[-1]],
        'Grad_Var': [grad_var_history[-1]],
        'Penalty_Scale': [p_scale]
    })
    iter_df.to_csv(file_config['riwayat_iterasi'], mode='a', header=not os.path.exists(
        file_config['riwayat_iterasi']), index=False)

    if total_iters >= maxiter and len(energy_history) >= conv_window:
        # [Phase 4] Entropy Protection & Annealing Protection
        # Jangan berhenti jika:
        # 1. Entropi masih tinggi (mencegah superposisi macet)
        # 2. Annealing pinalti belum mencapai 100%
        if (n_qubits <= 4 and entropy_history[-1] > 0.1) or (anneal_progress < 1.0):
            continue

        E_old, E_now = energy_history[-conv_window], energy_history[-1]
        if abs(E_old - E_now) / (abs(E_old) + 1e-12) < conv_tol:
            break
    return params, energy_history[-1], energy_history, entropy_history, grad_var_history,
    total_iters

best_energy = np.inf
best_params, best_depth, best_history, best_ent_hist, best_grad_var_hist = None, 1, [],
[], []
depth_energies = []
prev_params = None
all_theta_data = []

```

```

for depth in range(1, max_depth + 1):
    n_params = n_qubits * 2 * (depth + 1)
    cost_fn, prob_fn, state_fn = make_circuit(depth)

    if prev_params is not None and len(prev_params) < n_params:
        # Warm-start yang diperbaiki: gunakan distribusi normal sangat kecil
        # agar layer baru mendekati identitas, menjaga stabilitas energi.
        old_w = prev_params.reshape((depth, n_qubits, 2)) # (depth, n, 2)
        new_layer = rng.normal(0, 1e-4, (1, n_qubits, 2)) # layer baru (dekat identitas)
    )

    # Sisipkan layer baru sebelum output layer terakhir
    new_w = np.concatenate([old_w[:-1], new_layer, old_w[-1:]], axis=0) # (depth+1, n
, 2)

    # [Phase 2] Asymmetric Jitter: Pemutus simetri agar tidak terjebak superposisi
    asymmetric_jitter = rng.normal(0, 1e-4, n_params) * np.linspace(1, 1.5, n_params)
    init_p = new_w.flatten() + asymmetric_jitter
else:
    if ne_bitstring is not None:
        init_w = np.zeros((depth + 1, n_qubits, 2))
        for q in range(n_qubits):
            init_w[0, q, 0] = np.pi if int(ne_bitstring[q]) == 1 else 0.0

    # [Phase 2] Asymmetric Jitter pada inisialisasi Nash
    asymmetric_jitter = rng.normal(0, 1e-3, n_params) * np.linspace(1, 1.5,
n_params)
    init_p = init_w.flatten() + asymmetric_jitter
else:
    init_p = rng.uniform(0, 2 * np.pi, n_params)

# Skala learning rate dinamis: semakin dalam sirkuit, langkah semakin kecil (presisi)
current_a_base = best_a_base / np.sqrt(depth)

params, energy, e_hist, ent_hist, g_var_hist, n_iters = run_spsa(
    cost_fn, state_fn, n_params, init_params=init_p,
    a_base=current_a_base, c_base=0.1/np.sqrt(depth),
    depth_label=depth, target_p=target_penalty
)
print(f"    [Depth {depth}] Konvergen dalam {n_iters} iterasi | E = {energy:.6f}")
depth_energies.append((depth, energy, n_iters))
prev_params = params

# --- KUMPULKAN THETA (Poin 1) ---
for idx, val in enumerate(params):
    all_theta_data.append({'Depth': depth, 'Theta_Index': idx, 'Theta_Value': val})

if energy < best_energy:
    best_energy, best_params, best_depth, best_history, best_ent_hist,
best_grad_var_hist = energy, params, depth, e_hist, ent_hist, g_var_hist

# --- EKSPOR THETA GABUNGAN (Poin 1) ---
pd.DataFrame(all_theta_data).to_csv(file_config['theta_final'], index=False)

# --- EKSPOR DEPTH VS ENERGI (Poin 3) ---
depth_df = pd.DataFrame(depth_energies, columns=['Depth', 'Energy', 'Iterations'])
depth_df.insert(0, 'Date', curr_date.date())
depth_df.to_csv(file_config['depth_vs_energi'], mode='a', header=not os.path.exists(
file_config['depth_vs_energi']), index=False)

_, prob_fn, _ = make_circuit(best_depth)
probs = prob_fn(best_params)
sorted_indices = np.argsort(probs)[::-1]
best_bitstring = None
for idx in sorted_indices:
    bs = format(idx, f'0{n_qubits}b')
    if bs.count('1') == K:
        best_bitstring = bs
        break
if best_bitstring is None:
    top_k = np.argsort(probs)[-K:]
    bs_list = list('0' * n_qubits)

```

```

        for idx in top_k: bs_list[idx] = '1'
        best_bitstring = ''.join(bs_list)

    return [i for i, bit in enumerate(best_bitstring) if bit == '1'], best_depth, best_energy,
           best_history, best_ent_hist, best_grad_var_hist, depth_energies, probs, best_params

```

Kode K.2: Skrip inti loop VQE adaptif (run_vqe_adaptive.py)

```

import numpy as np
import pandas as pd
import os
from compute_endogenous_lambda import compute_endogenous_lambda
from find_nash_sbr import find_nash_sbr
from brute_force_validator import run_brute_force_window
from build_hamiltonian_total import build_hamiltonian_total
from find_optimal_lr_spsa import find_optimal_lr_spsa
from run_vqe_adaptive import run_vqe_adaptive
from calc_simple_return import calculate_simple_return
from calc_log_return import calculate_log_return
from calc_expected_returns import calculate_expected_simple_return,
calculate_expected_log_return_with_drag
from calc_expected_returns import calculate_expected_simple_return,
calculate_expected_log_return_with_drag

def run_strategy_step(lookback_data, tickers, curr_date, K=2, penalty_A=5.0,
                      max_depth=6, max_iter=100,
                      max_total_iter=500, batch_size=10,
                      conv_window=4, conv_tol=1e-3,
                      use_warm_start=True,
                      file_config=None):
    """
    Eksekusi satu periode pembelajaran dengan pencatatan tanggal.
    """
    if file_config is None:
        file_config = {
            'metrics': 'metrik_return_dan_lambda.csv',
            'bias_h': 'bias_h_total.csv',
            'interaksi_J': 'interaksi_J_total.csv',
            'pencarian_lr': 'hasil_pencarian_lr.csv',
            'parameter_pendamping': 'parameter_pendamping.csv'
        }

    n_assets = len(tickers)
    # ... rest of calculations ...
    # Simple Return dihitung manual via fungsi baru
    simple_rets = calculate_simple_return(lookback_data)
    # Log Return dihitung manual via fungsi baru
    log_rets = calculate_log_return(lookback_data)

    binary_st = (log_rets <= 0).astype(int) # 1 = turun, 0 = naik

    # --- PERHITUNGAN EXPECTED RETURN (Rumus Volatility Drag) ---
    mu_R_daily = calculate_expected_simple_return(simple_rets)
    # Variance log return sebagai dasar volatility drag
    var_r_daily = log_rets.var()
    mu_r_daily = calculate_expected_log_return_with_drag(mu_R_daily, var_r_daily)

    # Menggunakan return periode (126 hari)
    mu_simple_period = mu_R_daily.values * 126
    mu_log_period = mu_r_daily.values * 126

    sigma_log = log_rets.std().values
    sigma_period_log = sigma_log * np.sqrt(126)

    # Matriks kovariansi tetap berbasis log return
    sigma_period_matrix = log_rets.cov().values * 126

    # gamma disini mewakili degree of risk-aversion, menggunakan mu_log_period (dengan drag)
    # dan sigma_period_log
    gamma = compute_endogenous_lambda(mu_log_period, sigma_period_log)
    lam = penalty_A # penalty lambda untuk pembatas kardinalitas
    metrics_df = pd.DataFrame({

```

```

        'Date': [curr_date.date()] * n_assets,
        'Ticker': tickers,
        'Mu_Simple_Period': mu_simple_period,
        'Mu_Log_Period': mu_log_period,
        'Sigma_Log': sigma_log,
        'Sigma_Period_Log': sigma_period_log,
        'Lambda_RiskAversion': [gamma] * n_assets
    })

metrics_df.to_csv(file_config['metrics'], mode='a', header=not os.path.exists(file_config[
'metrics']), index=False)

# --- KONSTRUKSI MATRIKS Q (QUBO) TERPISAH ---
# 1. Komponen Objektif (Return & Risk)
Q_diag_obj = np.zeros(n_assets)
Q_off_obj = np.zeros((n_assets, n_assets))
K_sq = K**2

for i in range(n_assets):
    Q_diag_obj[i] = (gamma * sigma_period_matrix[i, i]) / (2.0 * K_sq) - (mu_simple_period
[i] / K)
    for j in range(i + 1, n_assets):
        Q_val = (gamma * sigma_period_matrix[i, j]) / (2.0 * K_sq)
        Q_off_obj[i, j] = Q_val
        Q_off_obj[j, i] = Q_val

# 2. Komponen Pinalti (Constraint)
Q_diag_pen = np.zeros(n_assets)
Q_off_pen = np.zeros((n_assets, n_assets))
for i in range(n_assets):
    Q_diag_pen[i] = (1.0 - 2.0 * K)
    for j in range(i + 1, n_assets):
        Q_off_pen[i, j] = 1.0
        Q_off_pen[j, i] = 1.0

# --- KONSTRUKSI PARAMETER ISING TERPISAH ---
# Fungsi pembantu untuk konversi Q ke h, J, C
def qubo_to_ising(Q_diag, Q_off, constant=0.0):
    h = np.zeros(n_assets)
    J = np.zeros((n_assets, n_assets))
    for i in range(n_assets):
        sum_Q_ij_half = 0.0
        for j in range(n_assets):
            if i != j:
                sum_Q_ij_half += Q_off[i, j] / 2.0
            if i < j:
                J[i, j] = Q_off[i, j] / 2.0
                J[j, i] = Q_off[i, j] / 2.0
        h[i] = -((Q_diag[i] / 2.0) + sum_Q_ij_half)

    sum_Q_ii_half = np.sum(Q_diag) / 2.0
    sum_Q_ij_half_total = 0.0
    for i in range(n_assets):
        for j in range(i + 1, n_assets):
            sum_Q_ij_half_total += Q_off[i, j] / 2.0
    C = sum_Q_ii_half + sum_Q_ij_half_total + constant
    return h, J, C

h_obj, J_obj, C_obj = qubo_to_ising(Q_diag_obj, Q_off_obj, 0.0)
h_pen, J_pen, C_pen = qubo_to_ising(Q_diag_pen, Q_off_pen, float(K_sq))

# --- EKSPOR BIAS & INTERAKSI TERPISAH ---
h_df = pd.DataFrame({
    'Date': [curr_date.date()] * n_assets,
    'Ticker': tickers,
    'Bias_h_Obj': h_obj,
    'Bias_h_Pen': h_pen
})
h_df.to_csv(file_config['bias_h'], mode='a', header=not os.path.exists(file_config['bias_h
']), index=False)

J_list = []

```



```

for i in range(n_assets):
    for j in range(i + 1, n_assets):
        J_list.append({
            'Date': curr_date.date(),
            'Ticker_i': tickers[i],
            'Ticker_j': tickers[j],
            'Interaction_J_Obj': J_obj[i, j],
            'Interaction_J_Pen': J_pen[i, j]
        })
J_df = pd.DataFrame(J_list)
J_df.to_csv(file_config['interaksi_J'], mode='a', header=not os.path.exists(file_config['interaksi_J']), index=False)

# --- EKSPOR KONSTANTA C ISING TERPISAH ---
c_df = pd.DataFrame({
    'Date': [curr_date.date()],
    'C_Obj': [C_obj],
    'C_Pen': [C_pen],
    'Penalty_A_Target': [penalty_A]
})
c_df.to_csv(file_config['parameter_pendamping'], mode='a', header=not os.path.exists(file_config['parameter_pendamping']), index=False)

# Membangun Hamiltonian terpisah (Diperlukan untuk VQE & LR Finder)
H_obj = build_hamiltonian_total(h_obj, J_obj, n_assets, offset=C_obj)
H_pen = build_hamiltonian_total(h_pen, J_pen, n_assets, offset=C_pen)

# [Tugas 4] Pencarian Nash Equilibrium (Hanya jika Warm-Start aktif)
if use_warm_start:
    ne_bitstring, ne_utility = find_nash_sbr(mu_simple_period, sigma_period_matrix, gamma,
        curr_date, N=n_assets, K=K, history_file=file_config.get('nash_history', 'riwayat_nash_sbr.csv'))
    print(f"    [Nash Eq] Warm-start aktif. Bitstring: {ne_bitstring} | Utility: {ne_utility:.6f}")
    vqe_init_bs = ne_bitstring
else:
    ne_bitstring, ne_utility = "N/A", 0.0
    vqe_init_bs = "0" * n_assets # Mulai dari |0...0>
    print(f"    [VQE] Warm-start dinonaktifkan. Memulai dari $|0...0\\rangle$")

# [Validasi] Jalankan Brute Force Validation per window
# Menggunakan Hamiltonian gabungan (Total) untuk pembandingan energi VQE
h_total = h_obj + penalty_A * h_pen
J_total = J_obj + penalty_A * J_pen
C_total = C_obj + penalty_A * C_pen

print(f"    [Brute Force] Validasi global minimum untuk window {curr_date.date()}...")
bf_bs, bf_e = run_brute_force_window(curr_date, n_assets, h_total, J_total, C_total, K, file_config)
print(f"    [Brute Force] Global Min: {bf_bs} | Energy: {bf_e:.6f}")

# [LR Finder] Menggunakan H gabungan dengan pinalti target bulan ini
H_target = H_obj + penalty_A * H_pen
print(f"    [LR Finder] Mencari Learning Rate optimal...")
best_a, test_a_values, final_energies = find_optimal_lr_spsa(H_target, n_assets, curr_date,
    ne_bitstring=vqe_init_bs, K=K, test_iters=30, file_config=file_config)
print(f"    [LR Finder] Base Learning Rate terpilih: {best_a:.4f}")

# Optimasi VQE dengan Penalty Annealing
selected_indices, depth_used, energy_final, best_history, best_ent_hist, best_gvar_hist,
depth_energies, winning_probs, best_params = run_vqe_adaptive(
    H_obj, H_pen, n_assets, curr_date, ne_bitstring=vqe_init_bs, K=K, target_penalty=penalty_A,
    max_depth=max_depth, max_iter=max_iter,
    max_total_iter=max_total_iter, batch_size=batch_size,
    conv_window=conv_window, conv_tol=conv_tol,
    best_a_base=best_a, file_config=file_config
)

lr_data = (test_a_values, final_energies, best_a)
return selected_indices, depth_used, energy_final, best_history, best_ent_hist,
best_gvar_hist, depth_energies, lr_data, ne_bitstring, ne_utility, winning_probs,

```

```
best_params
```

Kode K.3: Skrip eksekusi langkah strategi yang memadukan teori permainan klasik dengan VQE kuantum (`run_strategy_step.py`)

BIODATA PENULIS



PUTRA NAUFAL TABASSAM lahir di Surabaya tanggal 6 Januari 2004 merupakan anak pertama dari tiga bersaudara. Penulis adalah seorang mahasiswa Program Studi Sarjana Fisika di Institut Teknologi Sepuluh Nopember (ITS) sejak tahun 2022 dengan NRP 5001221120 melalui jalur SBMPTN. Sebelum berada di ITS, penulis menempuh pendidikan formal di SD Negeri Baratajaya Surabaya, SMP Negeri 6 Surabaya, dan SMA Negeri 14 Surabaya. Di samping pendidikan formal, penulis juga pernah mengenyam pendidikan keagamaan di pondok pesantren Ummul Quroo Surabaya. Selain aktif di menjalani pendidikan, penulis pernah aktif di beberapa organisasi seperti Dewan Kerja Cabang Gerakan Pramuka Kota Surabaya Periode 2021-2026, Dewan Kerja Ranting Gerakan Pramuka Kecamatan Rungkut Periode 2020-2022, dan turut aktif menjadi panitia di banyak kegiatan baik organisasi maupun pengabdian masyarakat. Penulis memiliki minat di bidang finansial sejak akhir SMA dan memutuskan untuk mengambil fisika teori untuk mengeksplorasi lebih lanjut tentang kombinasi bidang finansial dan kuantum secara praktis. Jika pembaca memiliki pertanyaan seputar penelitian ini, penulis dapat dihubungi melalui Email: naufal.tabassam@gmail.com